데이터베이스

데이터베이스의 기본 개념부터 설계 및 구현까지 학습하여 향후 데이터베이스를 효율적으로 관리하고 활용할 수 있는 능력을 배양합니다.

해커스 원격평생교육원

본 콘텐츠는 무단 복제 및 배포를 금지합니다.
반드시 수강생 본인의 학습을 위해서만 사용해야 합니다.

목 차 - 데이터베이스

1주차 1차시 정보환경 5
1주차 2차시 데이터베이스 특징 및 구성요소 8

2주차 1차시 데이터베이스 관리 시스템 1 12
2주차 2차시 데이터베이스 관리 시스템 2 15

3주차 1차시 데이터베이스 시스템의 구성요소 1 19
3주차 2차시 데이터베이스 시스템의 구성요소 2 22

4주차 1차시 관계형 데이터 구조 1 28
4주차 2차시 관계형 데이터 구조 2 32

5주차 1차시 관계 데이터 제약 1 38
5주차 2차시 관계 데이터 제약 2 45

6주차 1차시 SQL 이해 1 52
6주차 2차시 SQL 이해 2 58

7주차 1차시 관계 데이터 연산 1 67
7주차 2차시 관계 데이터 연산 2 71

9주차 1차시 데이터 모델링 79
9주차 2차시 ERD 85

10주차 1차시 데이터베이스 설계 92
10주차 2차시 DBMS의 선정 97

11주차 1차시 무결성 규정 ····· 104
11주차 2차시 HTML/XML ····· 109

12주차 1차시 데이터베이스 보안 ····· 116
12주차 2차시 데이터베이스 권한 ····· 121

13주차 1차시 빅데이터의 개요 ····· 128
13주차 2차시 빅데이터의 실제 ····· 132

14주차 1차시 빅데이터 분석기법 ····· 137
14주차 2차시 빅데이터와 인공지능 ····· 142

1주차 - 1차시	정보환경

1. 정보 처리 시스템

1) 정보 처리 시스템 구성 요소

데이터(Data)
관찰이나 측정을 통해서 수집된 사실(Facts)이나 값(Values : 수치, 스트링)

정보(Information)
의사결정을 할 수 있게 하는 데이터의 해석이나 데이터 상호간의 관계 - 의미(Meaning)

데이터(Data) ≠ 정보(Information)

P(D) = I

2) 정보 및 응용 시스템

◆ 정보 시스템(Information System)

- 한 기관을 위해 데이터를 수집, 조직, 저장하고 정보를 생성, 분배하는 수단

◆ 응용 시스템(Application Program)

- 한 부서를 위한 서브 시스템
- 응용 프로그램(Application Program)으로 구성

3) 정보 처리 시스템

◆ 경영 정보 시스템(MIS : Management Information System)

기획(Planning)	운영(Operation)	통제(Control)

◆ 의사 결정 지원 시스템(DSS : Decision Support System)

광의의 정보 시스템	데이터 웨어하우스 (Data Warehouse)
Information Repository, Information Archive	데이터 마이닝 (Data Mining)
Knowledge Discovery for Finding Rules or Patterns	지식 관리 시스템 (Knowledge Management System)

2. 일괄 처리 및 온라인 처리

1) 일괄 처리(Batch Processing)

◆ 사전 준비 작업 필요

- 원시 데이터(Raw Data)의 수집
- 분류 정리하여 화일(File)에 수록

◆ 시스템 중심 처리방법

- 높은 시스템 성능
- 낮은 처리 비용

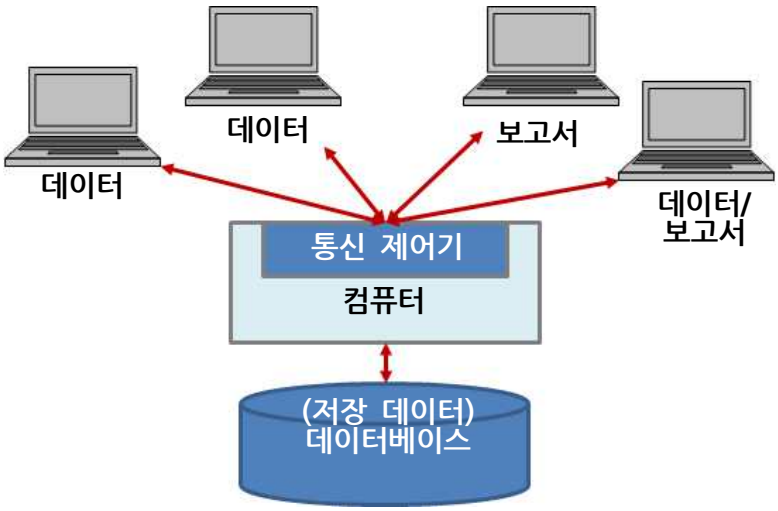
◆ 순차 접근 방법 이용 업무에 유리

2) 온라인 처리(Online Processing)

◆ 실시간 처리

- 사용자 중심 처리방법
- 낮은 시스템 성능
- 높은 처리 비용
- 통신 제어기(Communication Controller)가 필요
- 보수, 유지, 회복의 오버헤드

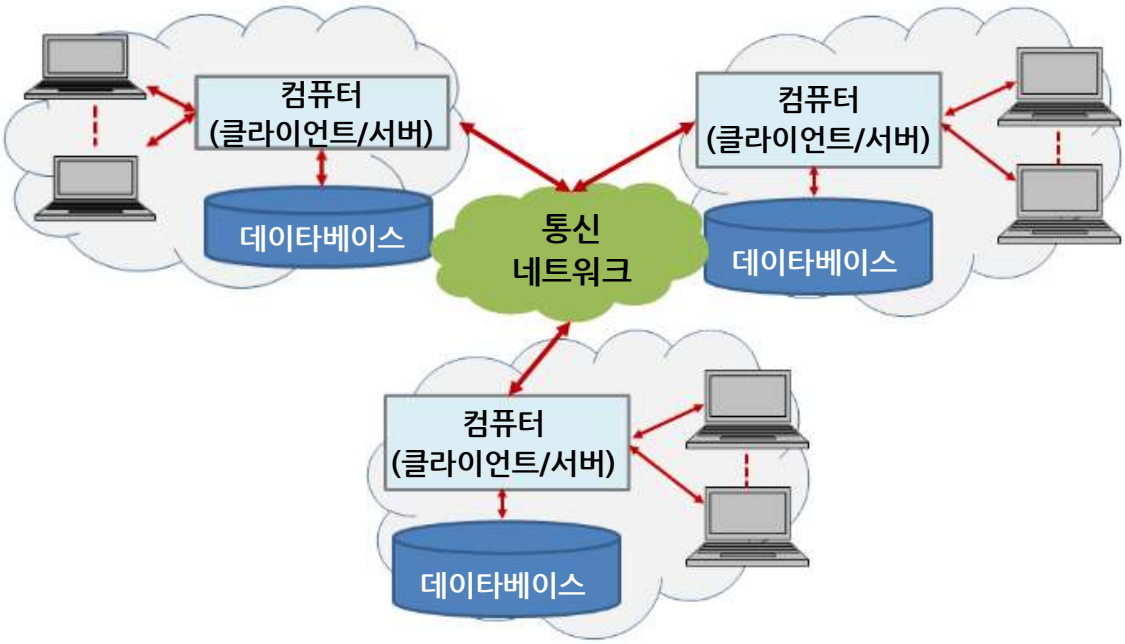
3) 온라인 처리 시스템(Online Processing System)



4) 분산 처리(Distributed Processing)

◆ 분산 처리기(Dispersed Processor) - 컴퓨터 시스템
◆ 분산 데이터베이스(Distributed Database)
◆ 통신 네트워크(Communication Network)
◆ 클라이언트/서버(Client/Server) 시스템 운영 형태

5) 분산 처리 시스템(Distributed Processing System)



1주차 - 2차시	데이터베이스 특징 및 구성요소

1. 데이터베이스의 특징

1) 데이터베이스

1963.6 제1차 SDC 심포지엄
• 컴퓨터 중심의 데이터베이스 개발과 관리 • Development and Management of a Computer-centered Data Base

1965.9 제2차 SDC 심포지엄
• 컴퓨터 중심의 데이터베이스 시스템 • Computer-centered Data Base Systems

- 한 조직(Enterprise)의 여러 응용 시스템들이 공용(Shared)하기 위해 통합(Integrated), 저장(Stored)한 운영 데이터(Operational Data)의 집합

공용 데이터 (Shared Data)	한 조직의 여러 응용 시스템들이 공동으로 소유, 유지, 이용하는데 데이터
통합 데이터 (Integrated Data)	- 최소의 중복(Minimal Redundancy) - 통제된 중복(Controlled Redundancy)
저장 데이터 (Stored Data)	컴퓨터가 접근 가능한 저장 매체에 저장 디스크, 테이프 등
운영 데이터 (Operational Data)	한 조직의 고유 기능을 수행하기 위해 필요한 데이터

2) 데이터베이스의 특징

① 실시간 접근성(Real-time Accessibilities)

- 질의에 대한 실시간 처리(Real-time Processing) 및 응답

② 계속적인 변화(Continuous Evolution)

- 갱신(Update), 삽입(Insert), 삭제(Delete) : 동적(Dynamic) 특성

③ 동시공용(Concurrent Sharing)

- 여러 사용자(Multi-user)가 동시에 사용

④ 내용에 의한 참조(Content Reference)

- 데이터의 위치(Location)나 주소(Address)가 아닌 내용(Contents)에 따라 참조

2. 데이터베이스의 구성요소 및 구조

1) 데이터베이스의 구성요소

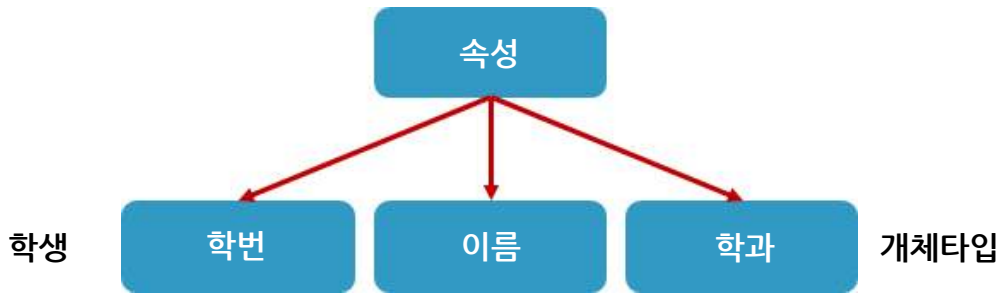
◆ 논리적 구성요소(Logical Components)

사용자의 입장

데이터베이스 = {개체(Entities), 관계(Relationships)}

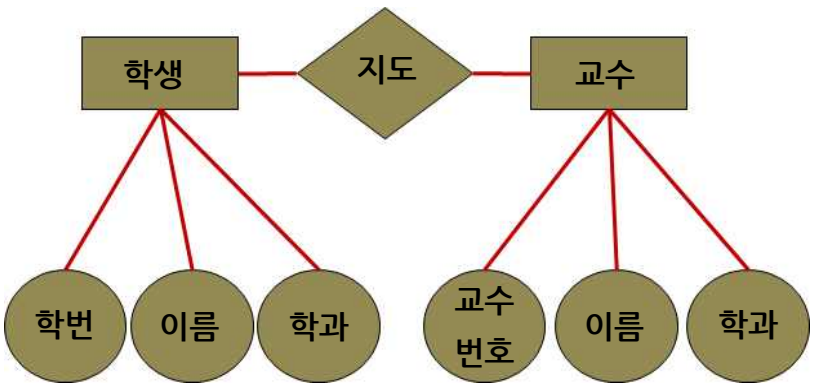
◆ 개체(Entity)

- 표현하려는 유무형 정보의 객체(Object)
- 정보의 단위(Unit)
- 하나 이상의 속성(Attribute)으로 구성
  - 데이터의 가장 작은 논리적 단위(Logical Unit)
- 개체집합(Entity Set)
  - Entity Type vs. Entity Instance
- 일반 레코드와 대응
  - Record Type vs. Record Instance



◆ 관계(Relationship)

속성 관계 (Attribute Relationship)	개체 내(Intra-entity) 관계 : 특성(Properties)
개체 관계 (Entity Relationship)	개체 간(Inter-entity) 관계



2) 데이터베이스의 구조

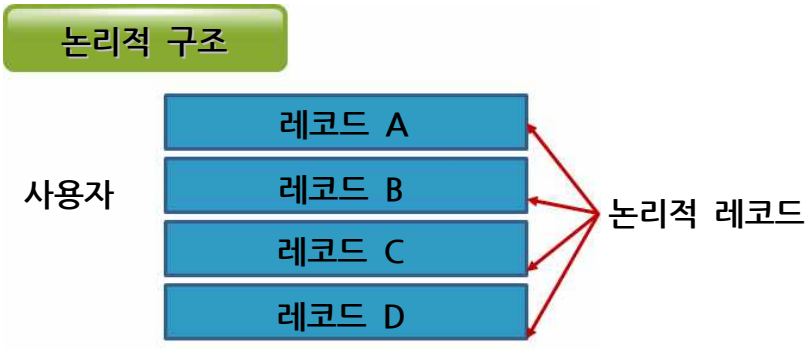
◆ 논리적 구조(Logical Organization)

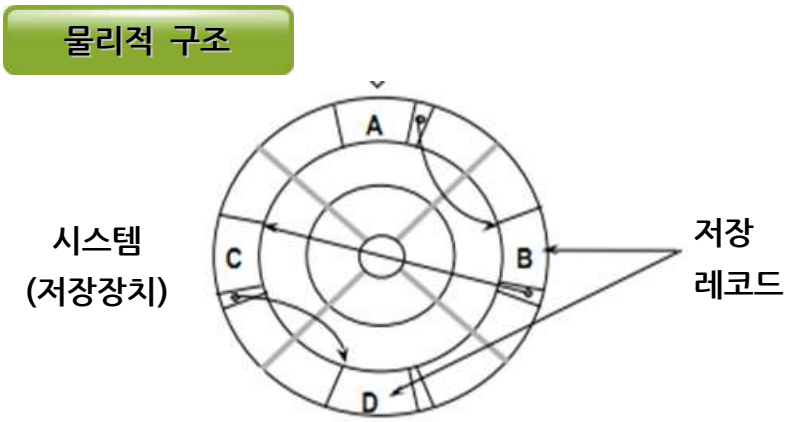
- 사용자의 관점에서 본 데이터의 개념적 구조(Conceptual Structure)
- 데이터의 논리적 배치(Logical Allocation)
- 논리적 레코드(Logical Record)

◆ 물리적 구조(Physical Organization)

- 저장 관점에서 본 데이터의 물리적 배치(Physical Allocation)
- 저장 장치에 저장된 데이터의 실제 구조(Actual Structure)
- 추가 정보를 포함 : 인덱스, 포인터 체인, 오버플로 구역 등
- 저장 레코드(Stored Record)

◆ 논리적 구조와 물리적 구조





2주차 - 1차시	데이터베이스 관리 시스템 1

1. DBMS의 발전 배경 1

1) 데이터베이스 관리 시스템
(DBMS: DataBase Management System)

2) 파일 중심 데이터 처리 시스템(File System)

◆ Operating System이 지원

① 각 응용 프로그램이 논리적 파일 구조를 정의하고 직접 물리적 파일 구조로 구현

② 각 응용 프로그램이 물리적 데이터 구조에 대한 접근 방법을 구현

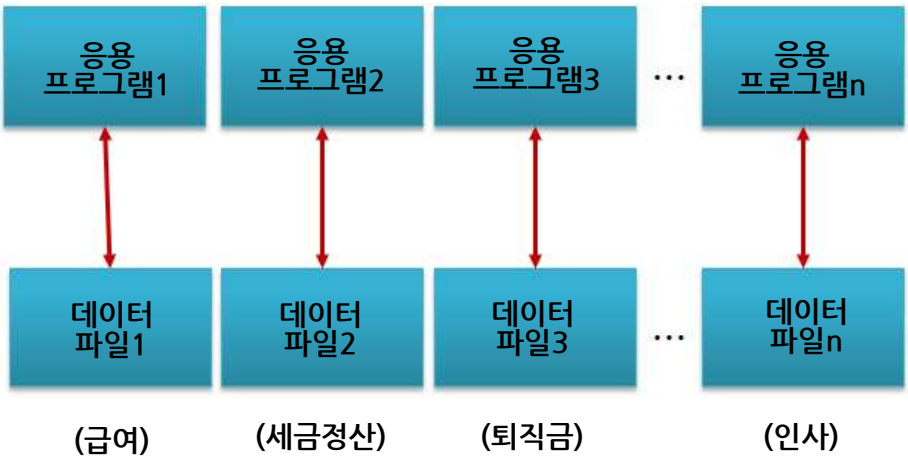
③ 각 사용자가 데이터와 이를 처리하는 프로그램을 모두 관리 유지

④ 응용 간의 데이터 공용이 불가능

→ 데이터의 종속성(Dependency)과 중복성(Redundancy)을 야기

3) 파일 시스템에서 응용 프로그램과 파일과의 관계

◆ 1:1의 관계



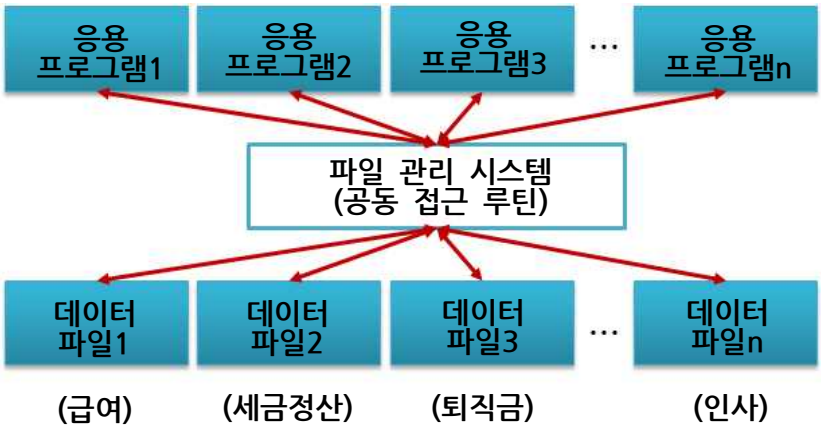
4) 데이터 종속성(Data Dependency)

◆ 응용 프로그램과 데이터간의 상호 의존관계
◆ 데이터의 구성 방법이나 접근 방법의 변경 시 관련 응용 프로그램도 동시에 변경해야 됨
◆ 응용 프로그램 관리가 곤란

5) 데이터 중복성(Data Redundancy)

◆ 한 시스템 내에 같은 내용의 데이터가 여러 파일에 중복 저장되어 관리
◆ 문제점
- 일관성(Consistency)
- 보안성(Security)
- 경제성(Economics)
- 무결성(Integrity)

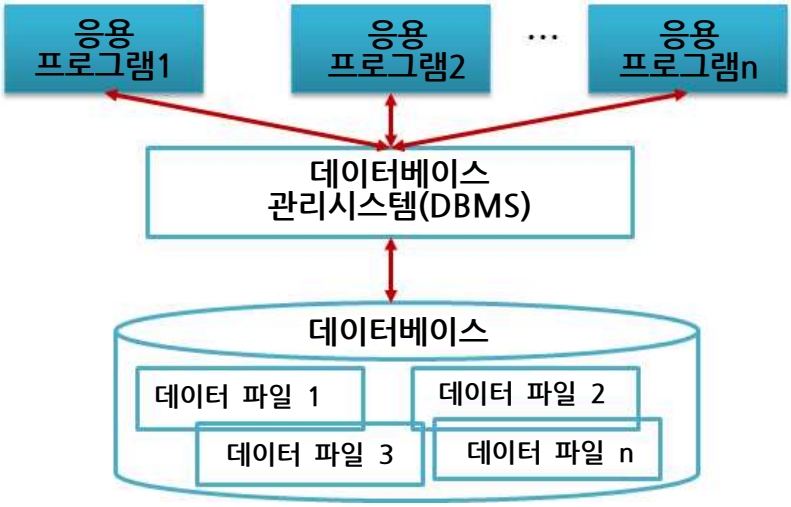
6) 파일 관리 시스템(File Management System)과 응용 프로그램



2. 데이터베이스의 관리 시스템의 정의

1) DBMS(Database Management System)이란?

◆ 응용 프로그램과 데이터 사이의 중재자로서 모든 응용 프로그램(사용자)들이 데이터베이스를 공용할 수 있게 관리해 주는 범용 소프트웨어 시스템



2) DBMS의 필수 기능

◆ 정의(Definition) 기능

- 하나의 저장 구조(Storage Structure)로 여러 사용자의 요구를 지원할 수 있도록 데이터를 조직(Organize)하는 기능
- 정의 기능의 요건
  - 데이터의 논리적 구조(Logical Structure)를 명세
  - 데이터의 물리적 구조(Physical Structure)를 명세
  - 물리적/논리적 사상(Mapping)을 명세

◆ 조작(Manipulation) 기능

- 사용자와 데이터베이스간의 Interface를 위한 수단
- 체계적 데이터베이스 접근 및 조작
  - 검색(Retrieve), 갱신(Update), 삽입(Insert), 삭제(Delete)
- 데이터 언어로 표현
  - 사용하기가 쉽고 자연스러운 도구
  - 원하는 연산의 명세 가능
  - 효율적인 처리

◆ 제어(Control) 기능

- 데이터의 정확성(Correctness)과 보안성(Security)을 유지하는 기능
- 제어기능의 요건
  - 무결성(Integrity) 유지

- 보안(Security), 권한(Authority) 검사
- 병행수행제어(Concurrency Control)
- 복구(Recovery)

2주차 - 2차시	데이터베이스 관리 시스템 2

1. DBMS의 장단점

1) 장점

- 데이터 중복(Redundancy)의 최소화
- 데이터의 공용(Sharing)
- 일관성(Consistency) 유지
- 무결성(Integrity) 유지
- 보안(Security) 보장
- 표준화(Standardization) 용이
- 기관 전체 데이터 요구의 조정

2) 단점

- 운영비의 Overhead
- 특정 응용 프로그램의 복잡화
- 복잡한 Backup과 Recovery
- 시스템의 취약성

3) DBMS의 궁극적 목적

◆ DBMS의 궁극적 목적은 데이터 독립성(Data Independency)

① 논리적 데이터 독립성(Logical Data Independency)

- 응용 프로그램에 영향을 주지 않고 논리적 데이터 구조의 변경이 가능
- 응용 프로그램의 효율적 개발이 가능

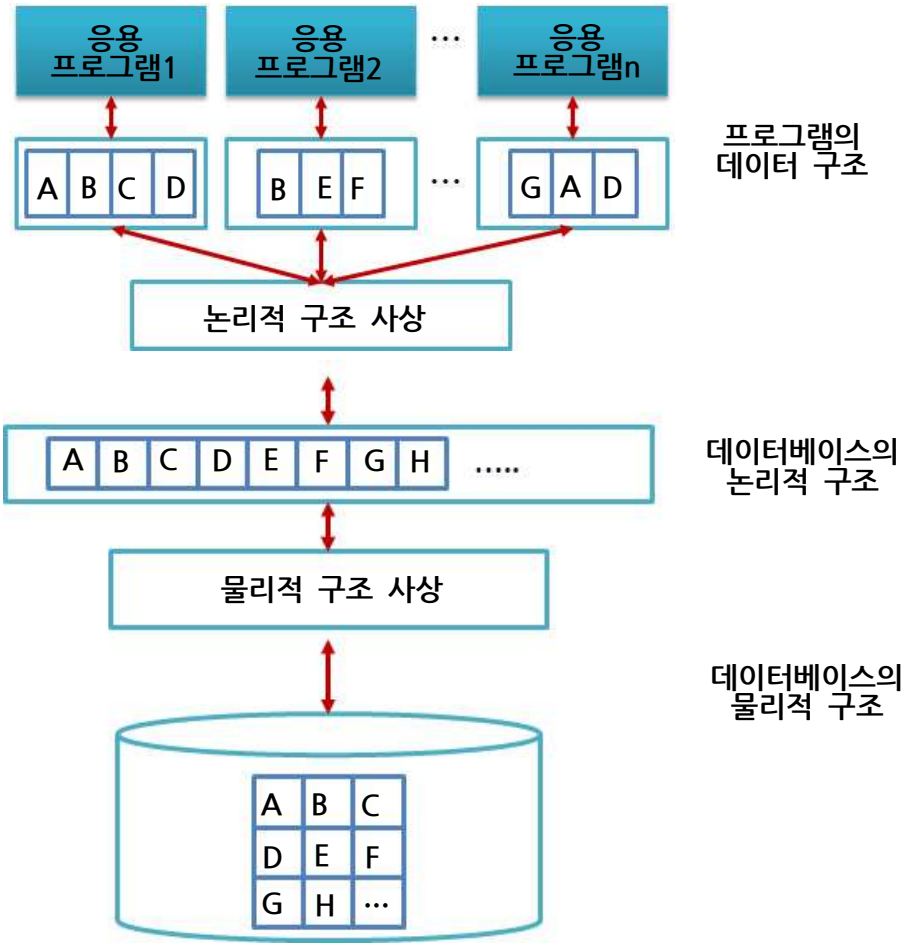
② 물리적 데이터 독립성(Physical Data Independency)

- 응용 프로그램과 논리적 데이터 구조에 영향을 주지 않고 물리적 데이터 구조의 변경이 가능
- 저장 장치의 효율적 개발이 가능

◆ 데이터 독립성 구현 기법

• 사상

4) 데이터 구조 간의 사상과 데이터 독립성



2. DBMS의 발전 과정

1) 제 1세대 DBMS(First-generation DBMS)

◆ IDS(Integrated Data Store)
- 최초의 범용 DBMS
- 1960년대 초 General Electric의 Charles Bachman에 의해 설계
- 네트워크 데이터 모델(Network Data Model)의 기초
- Bachman은 또한 데이터 구조도(Data Structure Diagram)도 제안

◆ IMS(Information Management System) DBMS
- 1960년대 후반 IBM이 개발
- 계층 데이터 모델(Hierarchical Data Model)의 기초
- 1970년대 초에는 대형 컴퓨터 회사들이 DBMS를 자체 제작 판매

2) 제 2세대 DBMS

① 관계 데이터 모델(Relational Data Model)
- IBM의 E.F.Codd가 제안
- Relational Database 이론의 기초
- 1980년대 DBMS의 주류가 되었고 계속 확장

② SQL
- IBM이 관계 DBMS의 일부로 개발
- 세계 표준 데이터베이스 언어

③ 주요 상용 DBMS
- DB2, Oracle, Ingres, Sybase, Informix

④ PC기반 DBMS
- Access, FoxPro, dBase, Paradox, SQL Server

3) 제 3세대 DBMS

① 사용자의 데이터베이스 응용에 대한 복잡성(Complexity) 증대
- Engineering, Images, Videos, Spatial, Time Series, Data Mining

② 사용자의 요구에 대처하기 위해 새로운 data model을 기반으로 시스템 개발
- 객체 지향 DBMS(OODBMS : Object-Oriented DBMS)
- Object-oriented Programming-basis

③ 현재는 제 2세대 DBMS + 제 3세대 DBMS

- 객체 관계 DBMS(O-RDBMS : Object-Relational DBMS)

3주차 - 1차시	데이터베이스 시스템의 구성요소 1

1. 데이터베이스 시스템

① Database System

② 데이터를 데이터베이스(Database)로 저장 관리하여 정보를 생성하는 컴퓨터 중심 시스템(Computer-centered System)

1) 구성요소

- 3-level Database
- Data Languages
- Users
- DBMS
- Database Administrator
- Database Computer

2. 3단계 데이터베이스

1) DB의 3단계(3-level) 표현

◆ DB의 3단계(3-level) 표현

- DB에 대한 3단계 관점(View)
  - 개인, 기관 전체, 저장장치

외부 단계 (External Level)	개념 단계 (Conceptual Level)	내부 단계 (Internal Level)

◆ 스키마

- DB의 구조(개체, 관계)와 제약조건(Constraints)을 명세
  - 3-level DB 구조에 기반

외부 스키마 (External Schema)	개념 스키마 (Conceptual Schema)	내부 스키마 (Internal Schema)

2) 스키마

① 외부 스키마(External Schema)

- 개개인의 사용자 관점에서 정의한 DB 스키마
- 전체 DB의 한 논리적인 부분
- Subschema

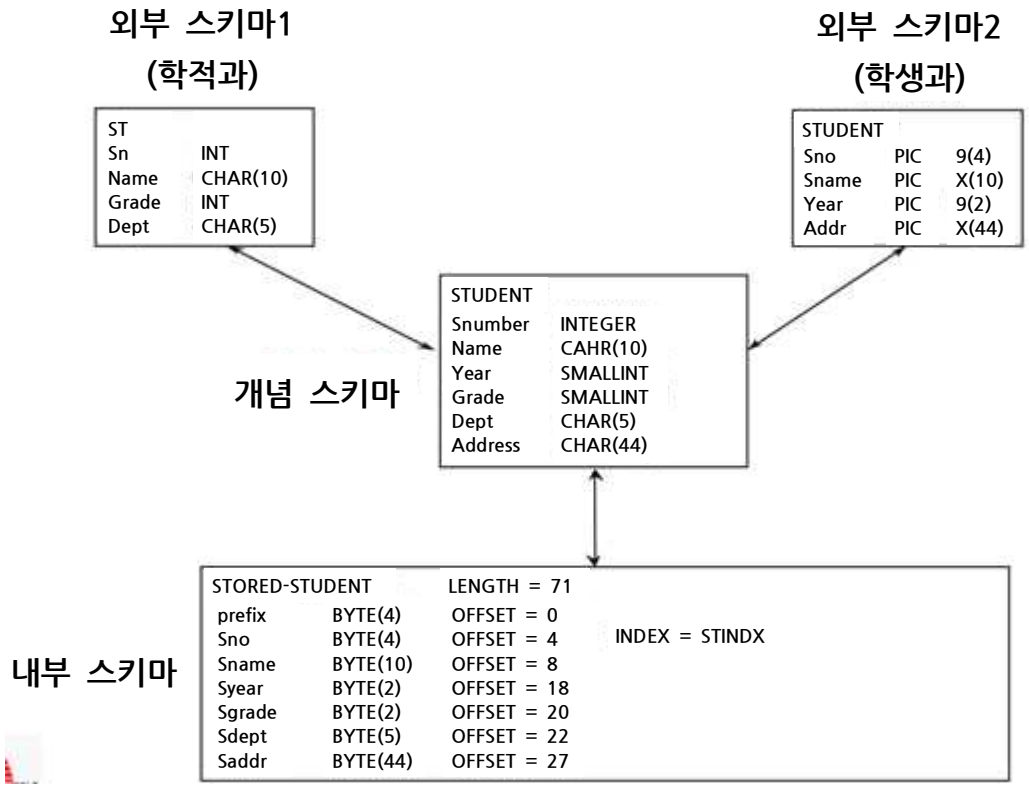
② 개념 스키마(Conceptual Schema)

- 범 기관적인 관점에서 정의한 DB 스키마
- 모든 응용에 대한 전체적인 통합된 데이터 구조
- Schema

③ 내부 스키마(Internal Schema)

- 저장장치 관점에서의 정의한 DB 스키마
- 개념 스키마에 대한 저장구조(Storage Structure)를 정의

3) 3단계 STUDENT Database 예



4) 3단계간의 사상(Mapping)

◆ External/Conceptual 단계 간의 사상

- 외부 스키마와 개념 스키마간의 대응 관계를 정의
- 응용 인터페이스(Application Interface)
- 논리적 데이터 독립성을 제공

◆ Conceptual/Internal 단계 간의 사상

- 개념 스키마와 내부 스키마간의 대응 관계를 정의
- 저장 인터페이스(Storage Interface)

내부 스키마

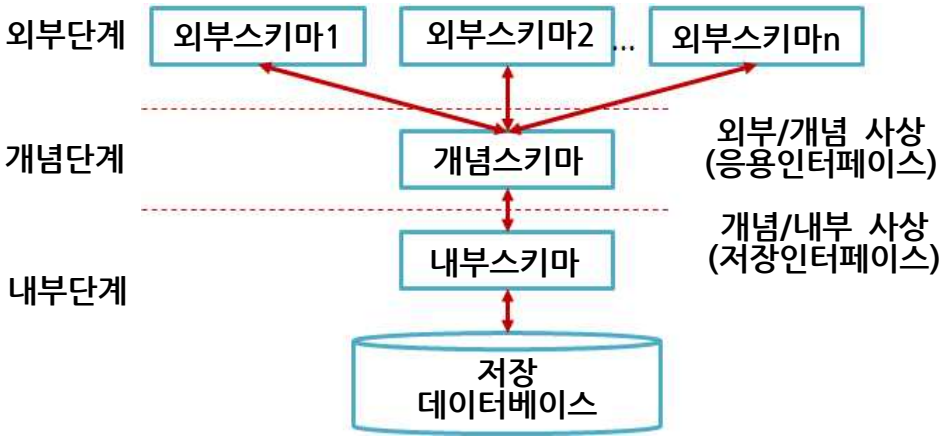
ㅣ터 독립성을 제공

ㅣ터 독립성(Data Independency)의 구현 방법

컴파일되어 카탈로그(Catalog)에 저장

그래픽 형태로 표현한 것이 스키마 다이어그램(Schema Diagram)

◆ 3단계 간의 사상(Mapping)



◆ 카탈로그와 데이터 디렉토리

- 카탈로그(Catalog)
  - 시스템 내의 모든 객체들에 대한 정의(Definition)나 명세(Specification)에 대한 정보를 수록
  - 시스템 DB, 메타 데이터베이스(Meta-database)
  - DB 관리자(DBA)가 사용하는 도구
  - 사용자와 시스템 모두 사용
  - 데이터 사전(Data Dictionary)이라고도 함
- 데이터 디렉토리(Data Directory)

- DB에 저장된 데이터를 참조(Reference)하는데 필요한 정보를 수록
- 시스템만 사용

3주차 - 2차시	데이터베이스 시스템의 구성요소 2

1. 데이터 언어(Data Language)

데이터베이스를 정의(Definition), 조작(Manipulation), 제어(Control)하기 위한 시스템과의 통신 수단

1) 데이터 정의어

◆ DDL

- Data Definition Language
- 데이터베이스 구조를 정의(Definitions)하고 변경(Alteration)하는데 사용

◆ 데이터 정의의 내용

- 논리적 데이터 구조의 정의
  - 스키마, 외부 스키마 명세
- 물리적 데이터 구조의 정의
  - 내부 스키마 명세
  - 데이터 저장 정의어(Data Storage Definition Language)
- 논리적 데이터 구조와 물리적 데이터 구조 간의 사상을 정의

2) 데이터 조작어 1

◆ DML

- Data Manipulation Language
- 사용자(응용 프로그램)와 DBMS 사이의 통신 수단
- 데이터 처리 연산(Operations)의 집합
  (데이터의 검색, 삽입, 삭제, 변경 연산)

◆ 절차적(Procedural) DML
- 저급 데이터 언어(Low-level Data Language)
- what과 how를 명세
- 한 번에 하나의 레코드만 처리(One record at a time)
- 응용 프로그램 속에 삽입(Embedded)되어 사용
- DML 예비 컴파일러(Precompiler)에 의해 처리

3) 데이터 조작어 2

◆ 비절차적(Non-procedural) DML
- 고급 데이터 언어(High-level Data Language)
- “What”만 명세(Declarative)하고 “How”는 시스템에 위임
- 한 번에 여러 개의 레코드를 처리(Set of records at a time)
- 질의어(Query Language)
  - 독자적이고 대화식 : Command 타입
  - 프로그램의 추상적 표현
- 데이터 부속어(DSL, Data Sublanguage)
  - Host Program 속에 삽입되어 사용되는 DML
  - 비절차적 또는 절차적 DML
- 데이터 언어의 완전성(Completeness of Data Language)
  - 사용자가 원하면 어떤 객체도 추출할 수 있고, 관계도 표현할 수 있는 능력

4) 데이터 제어어

◆ DCL
- Data Control Language
- 공용 데이터베이스 관리를 위해 데이터 제어(Data Control)를 정의하고 기술

◆ 데이터 제어 내용
- 데이터 보안(Security)
- 데이터 무결성(Integrity)
- 데이터 회복(Recovery)

- 병행수행(Concurrency)

◆ DB 관리 목적으로 DBA가 사용

5) 데이터 제어어

◆ 3단계 간의 사상(Mapping)

① External/Conceptual 단계 간의 사상

- 외부 스키마와 개념 스키마간의 대응 관계를 정의
- 응용 인터페이스(Application Interface)
- 논리적 데이터 독립성을 제공

② Conceptual/Internal 단계 간의 사상

- 개념 스키마와 내부 스키마간의 대응 관계를 정의
- 저장 인터페이스(Storage Interface)
- 물리적 데이터 독립성을 제공
- 구체적 데이터 독립성(Data Independency)의 구현 방법
- 스키마는 컴파일되어 카탈로그(Catalog)에 저장
- 스키마를 그래픽 형태로 표현한 것이 스키마 다이어그램(Schema Diagram)

◆ 사용자

→ 데이터베이스를 이용하기 위해 접근하는 사람

① 일반 사용자(End User)

- 비절차적 DML(Query Language)을 통해서 데이터베이스를 접근
- Menu, Form, Graphics
- 데이터의 Insertion, Deletion, Update, Retrieval

② 응용 프로그래머(Application Programmer)

- 응용 프로그램(Host 언어 + DML(DSL))을 통해서 DB를 접근
- Host 언어 : PL/I, COBOL, PASCAL, C, Java

③ DB 관리자(DBA : Database Administrator)

- DDL과 DCL 을 통해 DB를 정의하고 제어하는 사람

2. DBMS

DBMS는 Database를 관리하고 사용자가 요구하는 모든 연산을 수행

① 사용자의 접근 요구(Access Request)를 접수/분석
② 시스템이 이해할 수 있는 형태로 사용자 요구를 변환(Transform)
③ 외부/개념/내부/저장 구조간의 사상(Mapping)을 수행하여 저장 데이터를 접근
④ 저장 DB에 대해 필요한 연산(Operation)을 실행

1) DBMS의 구성요소 1

① DDL 컴파일러(DDL Compiler)
- DDL로 명세된 Schema를 내부 형태로 변환하여 Catalog에 저장
- 메타 데이터베이스(Meta-database) : 데이터의 데이터

② 질의어 처리기(Query processor)
- 질의문을 Parsing, Analysis, Compile하여 DB를 접근하기 위한 Object Code를 생성

③ 예비 컴파일러(Precompiler)
- 응용 프로그램에 삽입된 DML(DSL)을 추출하고 그 자리에 Procedure Call로 대체
- 추출된 DML은 DML 컴파일러로 전달
- 수정된 응용 프로그램은 Host 프로그램 컴파일러로 전달

④ DML 컴파일러(DML Compiler)
- DML 명령어를 Object Code로 변환

2) DBMS의 구성요소 2

① 런타임 데이터베이스 처리기(Runtime Database Processor)
- 실행 시간에 데이터베이스를 접근
- DB 연산(Operations)을 저장 데이터 관리자(Stored Data Manager)를 통해 수행

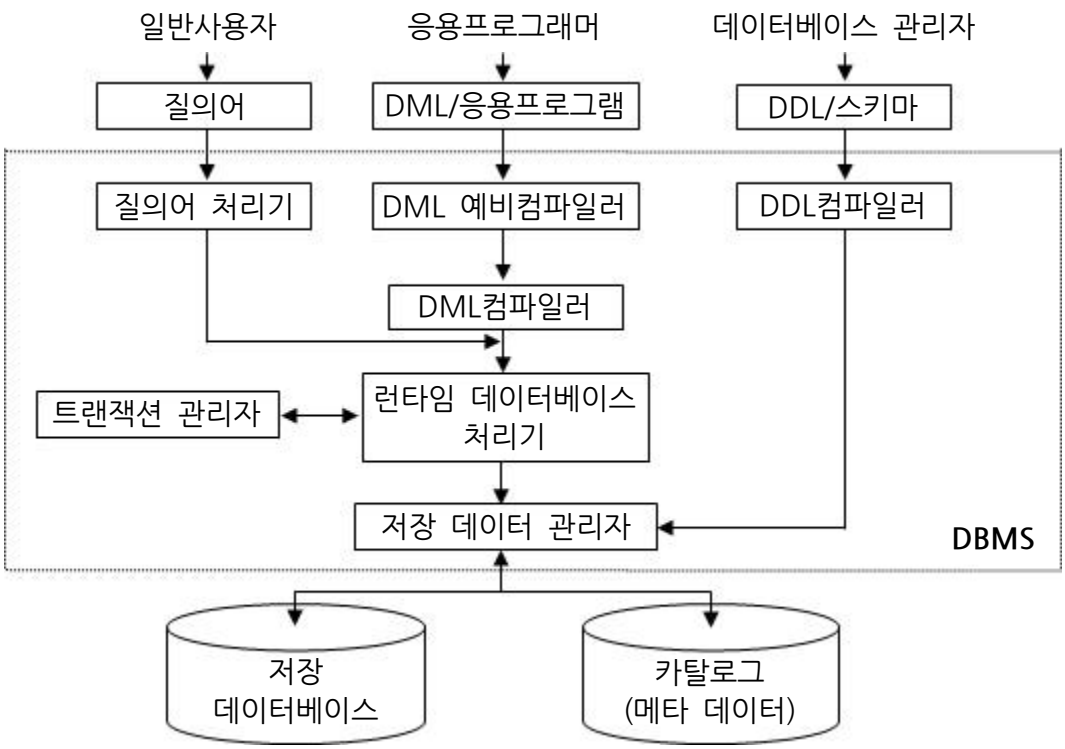
② 트랜잭션 관리자(Transaction Manager)

- 트랜잭션(Transaction) 단위로 작업을 수행
- DB 접근 과정에서 무결성(Integrity)과 권한(Authorization) 제어
- 병행 제어(Concurrency Control)와 회복(Recovery) 작업

③ 저장 데이터 관리자(Stored Data Manager)

- 디스크에 있는 사용자 DB나 카탈로그 접근을 제어
- 기본 OS Module(File Manager, Disk Manager)을 이용
  - File Manager를 통해 화일의 페이지 결정
  - Disk Manager가 데이터 전송을 수행
  - 메모리 내에서는 Buffer Manager가 관리

3) DBMS의 구성요소 3



4) 데이터베이스 관리자(DBA)

◆ DBA

- Database Administrator
- DB 시스템의 관리, 운영에 대한 모든 책임을 지는 사람

◆ DBA의 업무

- DB 설계와 운영
  - DB의 구성요소(Components)를 결정
  - 스키마 정의
  - 저장 구조와 접근 방법 설정
  - 보안 정책 수립, 권한부여, 유효성 검사
  - 백업(Backup), 회복(Recovery) 절차의 수립
  - DB의 무결성 유지
  - 성능 향상과 새로운 요구에 대응한 DB의 재구성
  - Catalog의 유지 관리
- 행정 및 불평 해결
  - 데이터의 표현과 시스템의 문서화에 표준 설정
  - 사용자의 요구 및 불평 해결
- 시스템 감시(Monitoring) 및 성능 분석
  - 시스템 성능의 청취
  - 자원의 이용도, 병목현상(Bottleneck), 장비 성능 등
  - 사용자 요구의 변화, 데이터 사용 추세, 각종 통계의 종합 분석

◆ 하드웨어 : 데이터베이스 컴퓨터

- 대규모 Database(VLDB) + DB 규모의 대형화
  - GB(Gigabyte) : 1024MB
  - TB(Terabyte) : 1024GB
  - PB(Petabyte) : 1024TB
  - EB(Exabyte) : 1024PB
- DB 기능만을 효율적으로 수행하기 위한 특수 목적의 하드웨어
  - Backend Computer(후위 컴퓨터)
  - 지능형 저장 장치
  - 내용식 메모리(Associative Memory)
  - 병렬 처리(Parallel Processing)

4주차 - 1차시	관계형 데이터 구조 1

1. 관계형 데이터 모델 1

1) 관계 데이터 모델(Relational Data Model) 1

◆ 관계 데이터 모델(Relational Data Model)의 탄생

- 1970년에 IBM의 E. F. Codd에 의해 제안

◆ 관계 데이터 모델의 특성

- 수학에서의 릴레이션(Relation)과 집합(Set) 이론에 기초
- 일반 사용자는 테이블(Table) 형태로 생각
  - 통상적인 테이블의 개념과는 다름
  - 관계 데이터 모델의 직관적인 이해에 도움
  - 테이블의 열(Column) = 필드(Field) 혹은 아이템(Item)
    → 관계 데이터 모델의 애트리뷰트(Attribute)
- 테이블의 행(Row) = 레코드(Record)
  → 관계 데이터 모델의 투플(Tuple)

2) 관계 데이터 모델 2

◆ 학생(STUDENT) 테이블 : 릴레이션

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

◆ 애트리뷰트와 도메인

테이블(Table) ≒ 릴레이션(Relation)

도메인(Domain)	**애트리뷰트가 취할 수 있는 값(Value)들의 집합**
애트리뷰트 (Attribute)	**- 도메인의 역할(Role) 이름** **- 한 릴레이션 내에서 애트리뷰트 이름들은 모두 달라야 함**

- 단순 도메인(Simple Domain) : 단순 애트리뷰트 : 원자 값
- 복합 도메인(Composite Domain) : 복합 애트리뷰트 : 복합 값
  연, 월, 일 → 날짜 : <연,월,일>

→ 애트리뷰트 이름과 도메인 이름은 같을 수도 있음
: 역할 이름을 도메인 이름으로 지정

2. 관계형 데이터 모델 2

1) 릴레이션

◆ 릴레이션의 개념

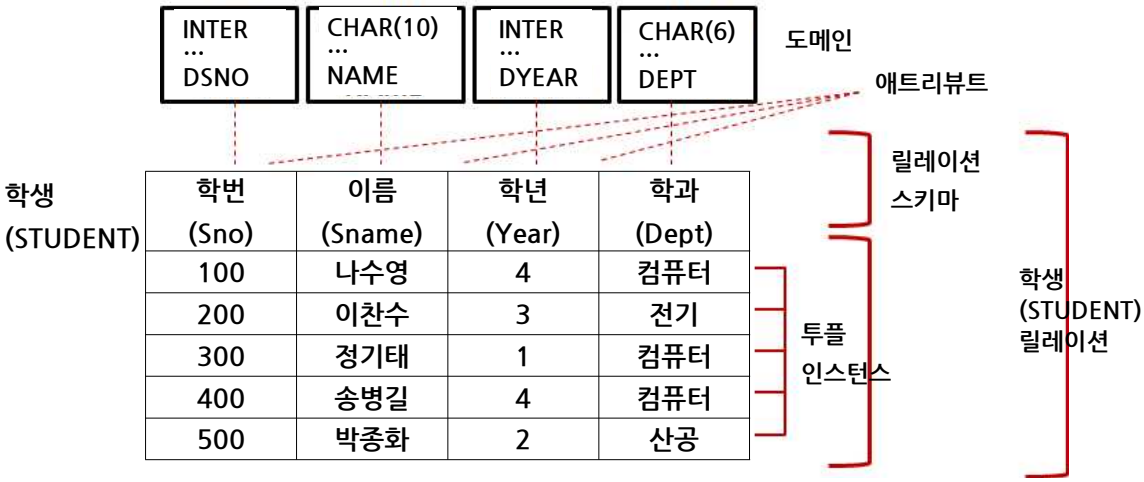
- STUDENT 릴레이션의 정의

```
DCL DOMAIN DSNO INTEGER;
DCL DOMAIN NAME CHAR(10);
DCL DOMAIN DYEAR INTEGER;
DCL DOMAIN DEPT CHAR(6);
DCL RELATION STUDENT
(Sno DOMAIN DSNO,
Sname DOMAIN NAME,
Year DOMAIN DYEAR,
Dept DOMAIN DEPT);
```

- 도메인 명세가 생략된 STUDENT 릴레이션 정의

```
DCL RELATION STUDENT
(Sno INTEGER,
Sname CHAR(10),
Year INTEGER,
Dept CHAR(6));
```

◆ 릴레이션의 예



◆ 릴레이션 스키마(Relation Schema)

- 릴레이션 내포(Relation Intension) 또는 릴레이션 스킴(Relation Scheme)이라고도 함
- 릴레이션 이름 + 애트리뷰트 이름
  - R(A1, A2, … , An), Ai ⇔ Di⏲ R({A1, A2, … , An})
- 정적 성질
  - 시간에 무관 + 시간에 따라 변경되지 않음
  - 릴레이션 타입과 같은 의미

◆ 릴레이션 인스턴스(Relation Instance)

- 릴레이션 외연(Relation Extension)이라고도 함
- 릴레이션 R의 인스턴스
  - 어느 한 시점에 릴레이션 R이 포함하고 있는 투플들의 집합
    {< V1, V2, … , Vn >} Vi ∈ Di

- 릴레이션의 내용, 상태, Snapshot

• 투플 : {(attr1=V1, attr2=V2, ⋯ , attrn=Vn)}

• 동적 성질

- 삽입, 삭제, 갱신으로 시간에 따라 변함
- 릴레이션 값(보통 릴레이션)

◆ 릴레이션(Relation) R

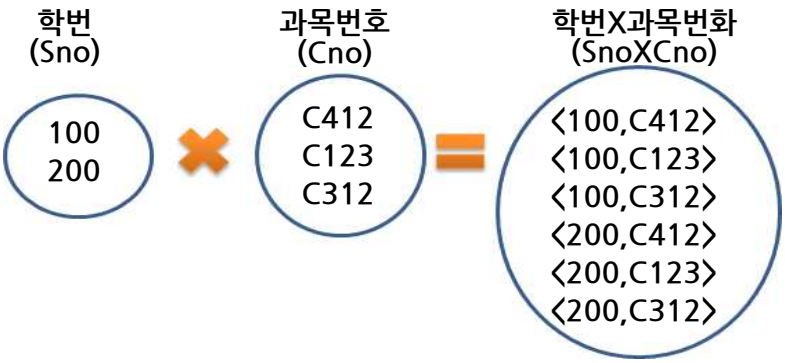
• 수학적 정의

- 릴레이션 R : 카티션 프로덕트(Cartesian Product)의 부분집합
- R ⊆ D1× D2× ... × Dn , 단 Di : i번째 도메인
- 즉 n-투플, <d1, d2, ... , dn>의 집합

di ∈ Di, i = 1,2, ... ,n

n : R의 차수(Degree : 1차, 2차, 3차, ... , n차)

투플의 수 : 카디널리티(Cardinality)



◆ 개념적 정의

◆ 릴레이션의 특성 1

① 투플의 유일성(Uniqueness of Tuples)

릴레이션 = 투플들의 "집합(Set)"

② 투플의 무순서성 (No Ordering of Tuples)

릴레이션 : 추상적 개념 ⇐ 투플들의 집합(Set)

테이블 : 구체적 개념

③ 애트리뷰트의 무순서성(No Ordering of Attributes)

릴레이션 스키마 → 애트리뷰트들의 "집합"

투플 : <Attribute: Value>쌍의 집합

◆ 릴레이션의 특성 2

④ 애트리뷰트의 원자성(Atomicity)

- 애트리뷰트 값은 원자 값(Atomic Value) : 논리적으로 분해 불가능
- 정규화 릴레이션(Normalized Relation)
  - ▸ 애트리뷰트 값으로 원자 값만 허용되는 릴레이션
  - ▸ 비정규화 릴레이션은 분해(Decomposition)를 통해 정규화
  - ▸ 동등한 의미를 유지
- 널 값(Null Value)도 원자 값으로 취급 : Unknown, Inapplicable
- 도메인
  - ▸ 단순 도메인
  - ▸ 복합 도메인 : 값을 하나의 단위로 취급

4주차 - 2차시	관계형 데이터 구조 2

1. 관계형 데이터 모델 1

1) 릴레이션의 정규화

등록 1(ENROL 1)
(a)비정규릴레이션

학번 (Sno)	과목성적 (Cgrade)	
	과목번호 (Cno)	성적 (Grade)
100	C413	A
	E412	A
200	C123	B
300	C312	A
	C324	C
	C413	A
400	C312	A
	C324	A
	C413	B
	E412	C
500	C312	B

등록 (ENROL 1)
(b)정규릴레이션

학번 (Sno)	과목번호 (Cno)	성적 (Grade)
100	C413	A
100	E412	A
200	C123	B
300	C312	A
300	C324	C
300	C413	A
400	C312	A
400	C324	A
400	C413	B
400	E412	C
500	C312	B

2) 관계 데이터베이스 개념

◆ 관계 데이터베이스(Relational Database)

- 테이블들의 집합
- 데이터베이스를 시간에 따라 그 내용(상태)이 변할 수 있는 테이블 형태로 표현

◆ 관계 데이터베이스 스키마(Relational Database Schema)

{릴레이션 스키마} ✚ {무결성 제약조건}

- 관계 데이터 모델 ⇔ 프로그래밍 시스템

릴레이션	⇔	화일
투플	⇔	레코드 (레코드어커런스)
애트리뷰트	⇔	필드(필드 타입)

→ 관계 데이터베이스라고 할 때 데이터가 물리적 테이블 형태로 저장된다는 것을 의미하지는 않음

◆ Example 1 : 대학(University) 관계 데이터베이스

- 학생(STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

- 과목(COURSE)

과목번호 (Cno)	과목이름 (Cname)	학점 (Credit)	학과 (Dept)	담당교수 (Prname)
C123	프로그래밍	3	컴퓨터	김성국
C312	자료구조	3	컴퓨터	황수관
C324	화일구조	3	컴퓨터	이규찬
C413	데이터베이스	3	컴퓨터	이일로
E412	반도체	3	전자	홍봉진

◆ Example 2 : 대학(University) 관계 데이터베이스(cont'd)

- 등록(ENRol)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	95

2. 관계형 데이터 모델 2

1) 데이터베이스 키(Key)

◆ 키(Key)

- 각 투플을 유일하게 식별할 수 있는 애트리뷰트 집합(Set of Attributes)
- 후보 키(Candidate Key)
  - 릴레이션 R(A1, A2, ..., An)에 대한 애트리뷰트 집합, K({Ai , Aj , ..., Ak})로서 다음 두 성질을 만족

① 유일성(Uniqueness)

▸ 각 투플에 대해 K({Ai , Aj , ... , Ak})의 값(< vi , vj , ... , vk >)은 유일

② 최소성(Minimality)

▸ K는 각 투플을 유일하게 식별하는데 필요한 애트리뷰트만 포함

◆ 슈퍼 키(Super Key)

- 유일성(Uniqueness)은 만족하지만 최소성(Minimality)은 만족하지 않는 애트리뷰트의 집합

◆ 기본 키(Primary Key)

- 후보 키(Candidate Key) 중에서 지정된 하나의 키

- 데이터베이스 설계자가 지정

• 각 투플에 대한 기본 키 값은 항상 유효한 값이어야 함

- null 값이 허용되지 않음

◆ 대체 키(Alternate Key)

• 후보 키 중에서 기본 키를 제외한 나머지 후보 키

◆ 외래 키(Foreign Key)

• 릴레이션 R의 애트리뷰트 집합 FK가 릴레이션 S의 기본 키일 때 이 FK는 R의 외래 키

• (FK의 도메인) = (S의 기본 키의 도메인)

• FK의 값은 S에 존재하는 값이거나 null

• R과 S가 같은 릴레이션일 수도 있음

• R을 참조 릴레이션(Referencing Relation), S를 피참조 릴레이션 (Referenced Relation)이라 함

- 릴레이션 R은 FK를 통해 릴레이션 S를 참조

◆ 외래 키(Foreign Key) 2

• R ≠ S인 경우

교수 (교수번호, 교수이름, 학과번호, 직급)
학과 (학과번호, 학과이름, 학과장교수번호, 학생수)
PK FK
학생 (학번, 이름, 학년, 학과)
과목 (과목번호, 과목이름, 학점, 학과, 담당교수)
등록 (학번, 과목번호, 성적)
FK FK

• R = S인 경우

교수1 (교수번호, 교수이름, 학과번호, 학장교수번호)
PK FK

2) 무결성 제약(Integrity Constraints)

① 개체 무결성(Entity Integrity)

- 기본 키 값은 언제 어느 때고 null 값을 가질 수 없음
- 정보 부재를 명시적으로 표현하는 특수한 데이터 값

(1) 알려지지 않은 값(Unknown Value)

(2) 해당 없음(Inapplicable)

② 참조 무결성(Referential Integrity)

- 외래 키 값은 반드시 피참조 릴레이션의 기본 키 값이거나 null임
- 이 무결성 제약조건은 데이터베이스 상태(Database State)가 항상 만족시켜야 될 제약조건임

③ 데이터베이스 상태(Database State)

- 어느 한 시점에 데이터베이스에 저장되어 있는 모든 데이터 값(투플)
- 데이터베이스 인스턴스(Database Instance)
  - 데이터베이스 스키마에 포함되어 있는 모든 릴레이션들의 인스턴스 집합
- 데이터베이스 상태의 계속적인 변화 : 삽입, 삭제, 변경 연산

→ DBMS는 데이터베이스 상태의 변화에도 항상 무결성 제약을 만족시키도록 해야 함

5주차 - 1차시	관계 데이터 제약 1

1. 관계형 데이터 1

1) 관계 데이터 연산

◆ 연산(Operation)과 데이터 언어(Data Language)

연산	데이터 언어
시스템 입장	사용자 입장

◆ 관계데이터 언어(Relational Data Language)

① 정형어(Formal Language)

- 관계대수(Relational Algebra)
  - ▸ 절차언어 : How, What
- 관계해석(Relational Calculus)
  - ▸ 비절차 언어 : What
  - ▸ 투플관계 해석(Tuple Relational Calculus)
  - ▸ 도메인관계 해석(Domain Relational Calculus)

◆ 관계 해석과 관계 대수는 표현과 기능 면에서 동등

2) 관계 대수

◆ 릴레이션을 처리하기 위한 연산의 집합

- 릴레이션 : 투플의 집합

기본연산	
일반 집합 연산자	- 합집합(UNION, ∪) - 교집합(INTERSECT, ∩) - 차집합(DIFFERENCE, -) - 카티션프로덕트(CARTESIAN PRODUCT, ×)
순수 관계 연산자	- 실렉트(SELECT, σ) - 프로젝트(PROJECT, Π) - 조인(JOIN, ⋈) - 디비전(DIVISION, ÷)

- 폐쇄성질(Closure Property)
  - ▸ 피연산자와 연산 결과가 모두 릴레이션
  - ▸ 중첩(Nested)된 수식의 표현이 가능

3) 일반 집합 연산자 1

① 합집합 (Union, ∪)

- $R \cup S = \{ t \mid t \in R \vee t \in S \}$

  $|R \cup S| \leq |R| + |S|$

② 교집합 (Intersect, ∩)

- $R \cap S = \{ t \mid t \in R \wedge t \in S \}$

  $|R \cap S| \leq \min\{ |R|, |S| \}$

③ 차집합 (Difference, -)

- $R - S = \{ t \mid t \in R \wedge t \notin S \}$

  $|R - S| \leq |R|$

④ 카티션 프로덕트 (Cartesian Product, ×)

- $R \times S = \{ r \cdot s \mid r \in R \wedge s \in S \}$

  ·: 접속(Concatenation)

  $|R \times S| = |R| \times |S|$

  차수(Degree) = R의 차수 + S의 차수

4) 일반 집합 연산자 2

① 합병가능(Union-compatible)한 릴레이션

- ∪, ∩, - 연산의 피연산자들은
  1. 차수가 같아야 함
  2. 대응 애트리뷰트 쌍 별로 도메인이 같아야 함

② ∪, ∩, × 연산은 결합적(Associative)임

- R ∪ S ∪ T = (R ∪ S) ∪ T = R ∪ (S ∪ T)
  R ∩ S ∩ T = (R ∩ S) ∩ T = R ∩ (S ∩ T)
  R × S × T = (R × S) × T = R × (S × T)

③ ∪, ∩, × 연산은 교환적(Commutative)임

- R ∪ S = S ∪ R
  R ∩ S = S ∩ R
  R × S = S × R

2. 관계형 데이터 2

1) 실렉트(SELECT: σ:sigma) 1

- 실렉트(σ)의 결과는 선택 조건을 만족하는 릴레이션의 수평적 부분집합(Horizontal Subset)
- Example
  - σ학과 = '컴퓨터'(학생)
    σ학번 = 300 ∧ 과목번호='C312'(등록)
    σ중간성적<기말성적 (등록)
- 데이터 언어 형식의 표현
  - R WHERE 조건식
- σ조건2(σ조건1(R)) = σ조건1(σ조건2(R))= σ(조건1 ∧ 조건2)(R)
- 선택도(Selectivity) :
  - 선택 조건에 의해 검색되는 투플의 비율

2) 프로젝트(PROJECT: Π: pi)

- 릴레이션의 수직적 부분집합(Vertical Subset)
- 결과에 투플이 중복되는 경우에는 제거
  - 결과도 투플의 집합(릴레이션) - 폐쇄성
- Example
  - 학생(학번, 이름, 학년, 학과)에서 Π이름, 학과(학생)

3) 릴레이션의 예

◆ 학생 릴레이션

- 학생(STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

◆ 등록 릴레이션

- 등록(ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80

◆ 프로젝트 연산 예

Π이름,학과(학생)

이름	학과
나수영	컴퓨터
이찬수	전기
정기태	컴퓨터
송병길	컴퓨터
박종화	컴퓨터

∩과목이름,담당교수(과목)

이름	학과
프로그래밍	김성국
자료구조	황수관
화일구조	이규찬
데이터베이스	이일로
반도체	홍봉진

4) 조인(JOIN:⋈) 1

◆ 결과 릴레이션의 애트리부트는 원 소속 릴레이션 이름을 애트리뷰트 앞에 한 정어로 붙여 일관성을 유지

- 예) 학생$\bowtie_{학번=학번}$ 등록
  - 이 연산의 결과 릴레이션이 갖는 애트리뷰트는 학생.학번, 학생.이름, 학생.학년, 학생.학과, 등록.과목번호, 등록.성적, 등록.중간성적, 등록.기말성적으로 표현
  - 그러나 상이한 애트리뷰트는 보통 그 이름만 사용

◆ 릴레이션 학생과 등록의 동일 조인 예

- 학생⋈학번=학번 등록

학생, 학번	이름	학년	학과	등록, 학번	과목번호	성적	중간성적	기말성적
100	나수영	4	컴퓨터	100	C413	A	90	95
100	나수영	4	컴퓨터	100	E412	A	95	95
200	이찬수	3	전기	200	C123	B	85	80
300	정기태	1	컴퓨터	300	C312	A	90	95
300	정기태	1	컴퓨터	300	C324	C	75	75
300	정기태	1	컴퓨터	300	C413	A	95	95
400	송병길	4	컴퓨터	400	C312	A	90	95
400	송병길	4	컴퓨터	400	C324	A	95	90
400	송병길	4	컴퓨터	400	C413	B	80	85
400	송병길	4	컴퓨터	400	E412	C	65	75
500	박종화	2	산공	500	C312	B	85	80

5) 자연조인

◆ 자연조인(Natural Join : ⋈N)

- 동일조인의 결과에서 중복되는 애트리뷰트를 제거
- 일반적으로 조인이라 하면 자연 조인을 의미
- 학생⋈N등록

학생, 학번	이름	학년	학과	등록, 학번	과목번호	성적	중간성적	기말성적
100	나수영	4	컴퓨터	100	C413	A	90	95
100	나수영	4	컴퓨터	100	E412	A	95	95
200	이찬수	3	전기	200	C123	B	85	80
300	정기태	1	컴퓨터	300	C312	A	90	95
300	정기태	1	컴퓨터	300	C324	C	75	75
300	정기태	1	컴퓨터	300	C413	A	95	95
400	송병길	4	컴퓨터	400	C312	A	90	95
400	송병길	4	컴퓨터	400	C324	A	95	90
400	송병길	4	컴퓨터	400	C413	B	80	85
400	송병길	4	컴퓨터	400	E412	C	65	75
500	박종화	2	산공	500	C312	B	85	80

6) 디비전 연산 Example

학과목(SC)

학번 (Sno)	과목번호 (Cno)
100	C413
100	E412
200	C123
300	C312
300	C324
300	C413
400	C312
400	C324
400	C413
400	E412
500	C312

과목1(C1)

과목번호 (Cno)
C413

과목2(C2)

과목번호 (Cno)
C312
C413

과목3(C3)

과목번호 (Cno)
C312
C413
E412

SC ÷ C1

학번 (Sno)
100
300
400

SC ÷ C2

학번 (Sno)
300
400

SC ÷ C3

학번 (Sno)
400

7) 관계 대수의 질의문 표현 1

① 모든 학생의 ‘이름’과 ‘학과’를 검색하라.

- Π이름, 학과(학생)

② ‘과목번호’가 C413인 과목에 등록한 학생의 ‘이름’과 ‘성적’은 무엇인가?

- Π이름, 성적(σ과목번호='C413'(학생⋈N등록))

③ ‘화일구조’ 과목을 가르치는 ‘담당교수’를 검색하라.

- Π담당교수(σ과목이름='화일구조'(과목))

5주차 - 2차시	관계 데이터 제약 2

1. 관계 해석

① Predicate Calculus에 기반

Predicate

- A Function Whose Value Is True Or False
- 실행 결과가 반드시 참(True)이나 거짓(False)인 함수

② 관계 데이터 모델의 연산 표현 방법

③ 비절차적(Non-procedural)

- 원하는 정보가 무엇이라는 것만 선언
- 투플 관계 해석(Tuple Relational Calculus)
- 도메인 관계 해석(Domain Relational Calculus)

1) 투플 관계 해석(Tuple Relational Calculus) 1

◆ 원하는 릴레이션을 투플 해석식(Tuple Calculus Expression)으로 정의하는 표기법

◆ 투플 해석식의 구성 요소

① 투플 변수(Tuple Variable) 또는 범위 변수(Range Variable) : t

- 범위식(Range Formula) : R(t)

  R : t의 범위 릴레이션(Range Relation)

② 한정 애트리뷰트(Qualified Attribute) : t.A 또는 t[A]

- 투플 변수 t가 나타내는 투플의 애트리뷰트 A의 값

  STUDENT(s)

s.Sno

2) 투플 관계 해석(Tuple Relational Calculus) 2

◆ 투플 해석식의 구성 요소(con"t)

③ 원자식(Atomic Formula)

(1) 범위식 : R(t)

▸ t : 투플 변수

R : t의 범위 릴레이션

(2) 조건식 : t.A θ u.B

▸ A, B : 애트리뷰트

θ : 비교연산자(=, ≠, 〈, ≤, 〉,≥)

(3) 조건식 : t.A θ c

▸ c : 상수

→ 원자식의 실행 결과는 반드시 참(True) 또는 거짓(False)

3) 투플 관계 해석(Tuple Relational Calculus) 3

◆ 투플 해석식의 구성 요소(con"t)

④ 정형식(WFF, Well-formed Formula)

- 원자식, 불리언 연산자(∧,∨, ¬), 정량자(∀,∃)가 다음 규칙에 따라 결합된 식

(1) 모든 원자식은 WFF

(2) F가 WFF이면, (F)와 ¬F도 WFF

(3) F와 G가 WFF이면, F∧G와 F∨G도 WFF

(4) 투플 변수 t가 자유 변수로 사용된 F(t)가 WFF이면, ∀t(F(t))와 ∃t(F(t))도 WFF

(5) 위의 규칙만을 적용해서 만들어진 식은 WFF

- [정형식의 예]

s.Sno = 100

c.Cno ≠ e.Cno

s.Sno = e.Sno ∧ e.Cno ≠ c.Cno

(∃e)(e.Sno = s.Sno ∧ e.Cno = 'C413')

4) 자유변수와 속박변수

① 자유변수(Free Variable)

- 정량자(Quantifier)로 한정되지 않는 투플변수
- ∀: "for all", 전칭정량자(Universal Quantifier)
- ∃: "there exists", "for some", 존재정량자(Existential Quantifier)

② 속박변수(Bound Variable)

- 정량자(∀t ,∃t)로 한정된 투플 변수
- 프리닉스 정규형(Prenex Normal Form)

  정량자만 앞에 먼저 기술한 WFF

  1) (∀t)(F(t))∧(∀t)(H(t)) = (∀t)(F(t)∧H(t))

  2) (∃t)(F(t))∨(∃t)(H(t)) = (∃t)(F(t)∨H(t))

5) 투플 해석식(Tuple Calculus Expression)

① 투플 해석식의 형식

- { t1.A1, t2.A2, …, tn.An | F(t1, … tn, tn+1, …, tn+m) }
  - ti : 투플변수
  - F(t1,…, tn, tn+1,…, tn+m) : ti가 연관된 정형식으로 조건을 명세
  - 막대(|) 왼편에 명세된 한정 애트리뷰트들은 목표 리스트(Target List)로서 막대(|) 오른편에 명세된 조건을 만족하는 투플로부터 추출됨

② Example

- {s.Sname | STUDENT(s)}
- {s.Sname | STUDENT(s)∧ s.Dept='컴퓨터'}
- {s.Sname, s.Dept | STUDENT(s)∧(∃e)

  (ENROL(e)∧s.Sno=e.Sno∧e.Grade='A')}

  → 목표 리스트에는 자유 변수만 나올 수 있음

6) 투플 해석식의 질의문 표현 1

- 등록(ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80

- 과목 C413에서 성적이 A인 학생의 학번을 모두 검색하라
  - {e.Sno | ENROL(e) ∧e.Cno='C413'∧ e.Grade='A' }
- 과목 C413을 등록한 학생의 이름과 학과를 모두 검색하라
  - { s.Sname, s.Dept | STUDENT(s) ∧∃e(ENROL(e) ∧ s.Sno=e.Sno ∧ e.Cno='C413') }

7) 투플 해석식의 질의문 표현 2

- 등록(ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80

- 모든 과목에 등록한 학생의 이름을 전부 검색하라.
  - { s.Sname | STUDENT(s) ∧(∀c)(∃e)(COURSE(c) ∧ ENROL(e) ∧ e.Sno = s.Sno ∧ e.Cno =c.Cno) }
- 과목'C413"에 등록하지 않은 학생의 이름을 전부 검색하라.
  - {s.Sname | STUDENT(s) ∧ (¬∃e)(ENROL(e) ∧ s.Sno=e.Sno∧e.Cno='C413') }

2. 도메인 관계 해석

1) 도메인 관계 해석(Domain Relational Calculus) 1

- 원하는 릴레이션을 도메인 해석식(Domain Calculus Expression)으로 정의하는 표기법
- 도메인 해석식의 구성 요소

① 도메인 변수(Domain Variable)

▸ 지정된 애트리뷰트도 메인의 한 원소만을 취하는 변수

xSno, xSname, xDept, xYear, …

▶ 범위식

· 도메인 변수 선언 식

· STUDENT(xSno, xSname, xDept, xYear)

2) 도메인 관계 해석(Domain Relational Calculus) 2

- 도메인 해석식의 구성요소(con't)

② 원자식(Atomic Formula)

(1) 범위식 : R(t)

· xi : 도메인 변수

R : xi의 범위 릴레이션

〈x1,x2,…,xn〉에 대응하는 값의 리스트는 릴레이션 R의 투플

(2) 조건식, x θ y

· x, y : 도메인 변수

θ : 비교 연산자(=, ≠, 〈, ≤, 〉,≥)

(3) 조건식, x θ c

· x : 도메인 변수

c : x가 정의된 도메인 값의 상수

- 원자식의 실행 결과는 반드시 참(True) 또는 거짓(False)

3) 도메인 관계 해석(Domain Relational Calculus) 3

- 도메인 해석식의 구성요소(con't)

③ 정형식(WFF, Well-formed Formula)

▶ 원자식, 불리언 연산자(∧,∨, ¬), 정량자(∀,∃)를 다음 규칙에 따라 결합하여 표현한 식

(1) 모든 원자식은 WFF

(2) F가 WFF이면, (F)와 ¬ F도 WFF

(3) F와 G가 WFF이면, F∧G와 F∨G도 WFF

(4) 도메인 변수 x가 자유변수로 사용된 F(x)가 WFF이면, (∀x)(F(x))와 (∃x)(F(x))도 WFF

(5) 위의 규칙만을 적용해서 만들어진 식은 WFF

4) 도메인 해석식(Domain Calculus Expression)

- 도메인 해석식의 형식
  - { x1,x2,⋯,xn | F(x1,⋯,xn,xn+1,⋯,xn+m) }
  - xi : 도메인변수
    F(x1,⋯,xn,xn+1,⋯,xn+m) : xi가 관련된 정형식
- 막대(|) 왼편에 명세된 n 도메인 변수들은 목표 리스트로서 막대(|) 오른편에 명세된 조건(정형식)을 만족하는 도메인 값으로 만들어지는 투플을 표현
- Example

① { xSname | STUDENT(xSno, xSname, xYear, xDept)}

② { xSname | (∃xDept) (STUDENT(xSno, xSname, xYear, xDept)∧ xDept='컴퓨터') }

③ { xSno, xDept | STUDENT(xSno, xSname, xYear,xDept)∧(∃xxSno)(∃xGrade) (ENROL(xxSno, xCno, xGrade, xMidterm, xFinal)∧xSno=xxSno∧xGrade='A') }

5) 도메인 해석식의 질의문 표현

- 컴퓨터학과 3,4 학년(Year)의 이름(Sname)을 검색하라.
  - { xSname | (∃xYear)(∃xDept) (STUDENT(xSno, xSname, xYear, xDept)∧xYear ≥ 3∧ xDept='컴퓨터') }
- 과목 번호(Cno) C413에서 성적(Grade)이 A인 학생의 학번(Sno)을 모두 검색하라.
  - { xSno | (∃xCno)(∃xGrade) (ENROL(xSno, xCno, xGrade, xMidterm, xFinal) ∧xCno= 'C413' ∧xGrade='A') }

6주차 - 1차시	SQL 이해 1

1. 데이터베이스 정의문

1) SQL의 역사

① SEQUEL(Structured English QUEry Language)에 연유

- 1974년, IBM 연구소에서 발표
- IBM은 'SYSTEM R' 의 인터페이스로 설계 구현
  : 실험적 관계 데이터베이스 시스템의 인터페이스

② SQL(Structured Query Language)

③ 표준 SQL

- 1986년, SQL-86 또는 SQL1
- 1992년 개정, SQL/92, SQL-92 또는 SQL2
- 1999년 : SQL3, SQL-99
- 2003년 : SQL4, SQL-2003, 객체 개념을 지원하는 기능 추가

④ 현재

- 미국 표준 연구소(ANSI)와 국제 표준 기구(ISO)에서 관계 데이터베이스의 표준 언어로 채택
- 상용 DBMS인 IBM의 DB2와 SQL/DS의 데이터 언어로 사용
- ORACLE, INFORMIX, SYBASE 등과 같은 모든 주요 제품에서 채택

2) SQL(Structured Query Language)

◆ SQL의 의미

- 구조화 질의어
- 종합 데이터베이스 언어 역할
  - 단순히 검색만을 위한 데이터 질의어가 아님
- 데이터 정의어(DDL), 데이터 조작어(DML), 데이터 제어어(DCL)의 기능 모두 제공

◆ SQL의 특징(1)

- 관계 대수 + 확장된 투플 관계 해석 기초
- 고급, 비절차적 데이터 언어
  - 사용자 친화적인 인터페이스 제공
- SQL의 표준화
  - 상용 RDBMS간의 전환이 용이
  - 여러 관계 데이터베이스를 접근하는 데이터베이스 응용 프로그램의 작성을 지원
- 터미널을 통해 대화식 질의어로 사용

◆ SQL의 특징(2)

- 응용 프로그램에 삽입된 형태로도 사용 가능
  - Java, COBOL, C, C++ 등과 같은 범용 프로그래밍 언어로 된 응용 프로그램
- 개개의 레코드 단위로 처리하기 보다는 레코드 집합 단위로 처리
- 선언적 언어
  - SQL 명령문에는 데이터 처리를 위한 접근 경로(Access Path)에 대한 명세가 불필요
- 관계 모델의 공식 용어 대신 일반 용어를 사용
- 릴레이션 - 테이블, 투플 - 행, 애트리뷰트 - 열

◆ SQL 데이터 정의문

- 스키마와 카탈로그

스키마 (Schema)	- 하나의 응용(사용자)에 속하는 테이블과 기타 구성요소 등의 그룹 - 포함 내용: 스키마 이름(Schema Name), 스키마를 소유하는 허가권자(Authorization Identifier), 테이블, 뷰, 도메인, 기타 내용에 대한 기술자(Descriptor)

  - 권한 식별자(Authorization Identifier), 테이블, 뷰, 도메인, 기타 내용에 대한 기술자(Descriptor) 카탈로그(Catalog)
    - ▸ SQL 시스템 내에서의 한 스키마 집합
    - ▸ 하나의 특별한 스키마 Information_schema를 포함

: 그 카탈로그에 있는 모든 스키마에 대한 정보를 제공

◆ 도메인 정의문(1)

• 일반 형식

- CREATE DOMAIN 도메인_이름 데이터_타입

```
[ 기정 값_정의 ]
[ 도메인_제약조건_정의리스트 ];
   ex) CREATE DOMAIN DEPT CHAR(4)
DEFAULT '???'
CONSTRAINT VALID-DEPT
CHECK( VALUE IN
('COMP', 'ME', 'EE', 'ARCH', '???'));
```

- ALTER DOMAIN 도메인_이름 <변경 내용>
- DROP DOMAIN 도메인_이름 RESTRICT | CASCADE ;
  - ▸ RESTRICT : 다른 곳에서 이 도메인을 참조하고 있지 않을 때 삭제
  - ▸ CASCADE : 이 도메인을 참조하고 있는 뷰나 제약조건도 삭제

◆ 도메인 정의문(2)

• 데이터 타입

- 숫자
  - ▸ INT(EGER), SMALLINT : 정수
  - ▸ FLOAT(n), REAL, DOUBLE PRECISION : 실수
  - ▸ DECIMAL(i, j), NUMERIC(i, j) : 정형 숫자
- 문자 스트링
  - ▸ CHAR(n) : 고정 길이 문자
  - ▸ VARCHAR(n) : 가변 길이 문자
- 비트 스트링
  - ▸ BIT(n), BIT VARYING(n)
- 날짜 스트링
  - ▸ DATE : YY-MM-DD
- 시간
  - ▸ TIME : hh:mm:ss

- ▸ TIMESTAMP : DATE와 TIME 포함
- ▸ INTERVAL : DATE, TIME, TIME STAMP 포함

2. 데이터베이스, 테이블 등의 정의

1) 기본 테이블의 생성 (1)

◆ 테이블의 종류

- 기본 테이블(Base Table)
  - CREATE TABLE 문으로 만들어지는 테이블
  - DBMS의 화일로 생성되고 저장
- 가상 테이블(Virtual Table)
  - CREATE VIEW 문으로 만들어지는 테이블
  - 어떤 기본 테이블로부터 유도되어 만들어지는 테이블
  - 독자적으로 존재 불가
- 임시 테이블(Temporary Table)
  - 질의문 처리 과정의 중간 결과로 만들어지는 테이블
  - DDL 문으로 만들어지는 것이 아님

2) 기본 테이블의 생성 (2)

◆ 일반형식

- CREATE TABLE 테이블이름

```
({열이름 데이터타입 [NOT NULL] [DEFAULT 값],}+
[PRIMARY KEY (열이름_리스트),]
{[UNIQUE (열이름_리스트),]}*
{[FOREIGN KEY(열이름_리스트)
REFERENCES 기본테이블[(열이름_리스트)]
[ON DELETE 옵션]
[ON UPDATE 옵션] ,]}*
[CONSTRAINT 이름] [CHECK(조건식)]);
```

3) 기본 테이블의 생성 (3)

- PRIMARY KEY : 기본 키와 개체 무결성 제약조건을 명세
- UNIQUE : 대체키를 명세, 후보 키
- NOT NULL : 애트리뷰트 값 제약조건
- FOREIGN KEY : 외래 키와 참조 무결성 제약조건을 명세
  - SQL에서는 참조된 테이블의 후보 키도 외래 키로 될 수 있음
- 옵션에는 SET NULL, SET DEFAULT, CASCADE가 있음
- ON DELETE SET NULL
  - 참조 투플(열 값)이 삭제되면 NULL로 설정
- ON UPDATE CASCADE
  - 참조 투플(열 값)이 갱신되면 갱신된 값이 파급적으로 갱신됨
- CHECK : 제약조건을 명세

4) 기본 테이블의 생성 (4)

◆ 예

- CREATE TABLE ENROL

```
( Sno INTEGER NOT NULL,
Cno CHAR(6) NOT NULL,
Grade INTEGER,
PRIMARY KEY(Sno,Cno),
FOREIGN KEY(Sno) REFERENCES STUDENT(sno)
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY(Cno) REFERENCES COURSE
ON DELETE CASCADE
ON UPDATE CASCADE,
CHECK(Grade ¤ 0 AND Grade ⓞ 100));
```

5) 기본 테이블의 제거와 변경 (1)

◆ 기본 테이블의 제거

- 일반 형식

```
DROP TABLE 기본_테이블_이름
{ RESTRICT | CASCADE } ;
- RESTRICT: 참조하는 뷰나 제약조건이 있으면 실행 실패
- CASCADE: 참조 뷰나 제약조건도 모두 삭제
DROP TABLE COURSE CASCADE;
```

◆ 스키마 제거

- 일반 형식

```
DROP SCHEMA 스키마_이름
{ RESTRICT | CASCADE };
DROP SCHEMA UNIVERSITY CASCADE;
```

6) 기본 테이블의 제거와 변경 (2)

◆ 기본 테이블의 변경

- 일반 형식

```
ALTER TABLE 기본_테이블_이름
([ADD 열_이름 데이터_타입] [DEFAULT 기정 값] |
[DROP 열_이름] [CASCADE] |
[ALTER 열_이름 (DROP DEFAULT |
SET DEFAULT 기정 값)]);
```

- 예
  - ALTER TABLE ENROL

```
ADD Final CHAR DEFAULT 'F';
```

  - ALTER TABLE ENROL

```
DROP Grade CASCADE;
```

- DROP 열_이름
  - RESTRICT : 참조하는 뷰나 제약조건이 있으면 실행 실패
  - CASCADE : 참조 뷰나 제약조건도 모두 삭제

6주차 - 2차시	SQL 이해 2

1. 데이터베이스 조작문

1) 예

◆ 대학(University) 관계 데이터베이스

- 학생(STUDENT)

학번 (Sno)	이름 (Sname)	학년 (Year)	학과 (Dept)
100	나수영	4	컴퓨터
200	이찬수	3	전기
300	정기태	1	컴퓨터
400	송병길	4	컴퓨터
500	박종화	2	산공

- 과목(COURSE)

과목번호 (Cno)	과목이름 (Cname)	학점 (Credit)	학과 (Dept)	담당교수 (PRname)
C123	프로그래밍	3	컴퓨터	김성국
C312	자료 구조	3	컴퓨터	황수관
C324	화일 구조	3	컴퓨터	이규찬
C413	데이터베이스	3	컴퓨터	이일로
E412	반도체	3	전자	홍봉진

◆ 대학(University) 관계 데이터베이스(cont'd)

- 등록(ENROL)

학번 (Sno)	과목번호 (Cno)	성적 (Grade)	중간성적 (Midterm)	기말성적 (Final)
100	C413	A	90	95
100	E412	A	95	95
200	C123	B	85	80
300	C312	A	90	95
300	C324	C	75	75
300	C413	A	95	90
400	C312	A	90	95
400	C324	A	95	90
400	C413	B	80	85
400	E412	C	65	75
500	C312	B	85	80

2) 데이터 검색

◆ 기본 구조

```
SELECT 열_리스트
FROM 테이블_리스트
WHERE 조건 :
```

실행 결과

Sname	Sno
나수영	100
정기태	300
송병길	400

◆ 예

```
SELECT Sname, Sno
FROM STUDENT
WHERE Dept = '컴퓨터';
SELECT STUDENT.Sname, STUDENT.Sno
FROM STUDENT
WHERE STUDENT.Dept = '컴퓨터';
```

2. 데이터베이스 검색, 삽입, 삭제문, 뷰

1) 데이터 검색

◆ 폐쇄 시스템(Closed System)

- 테이블 처리 결과가 또 다시 테이블이 되는 시스템
- 중첩 질의문(Nested Query)을 구성할 수 있는 이론적 기초

◆ SQL과 이론적 Relational Model의 차이점

- SQL의 테이블
  - 한 테이블 내에 똑같은 레코드(행) 중복 가능
  - 기본 키를 반드시 가져야 하는 것은 아님
  - 이론상 SQL의 테이블은 투플의 집합이 아님
  - 같은 원소의 중복을 허용하는 다중 집합(Multiset) 또는 백(Bag)
    → DISTINCT 명세 : 집합과 같은 결과를 만듦
- 일반적인 형식

```
SELECT [ALL | DISTINCT] 열_리스트
FROM 테이블_리스트
[WHERE 조건]
[GROUP BY 열_리스트
[HAVING 조건]]
[ORDER BY 열_리스트 [ASC | DESC]];
```

- 검색 결과에 레코드의 중복 제거
  - SELECT DISTINCT Dept
    FROM STUDENT;
- 테이블의 열 전부를 검색하는 경우
  - SELECT *
    FROM STUDENT;
- 조건 검색
  - SELECT Sno, Sname

FROM STUDENT
WHERE Dept = '컴퓨터' AND Year = 4;

- 순서를 명세하는 검색
  - SELECT Sno, Cno
    FROM ENROL
    WHERE Midterm ⌑ 90
    ORDER BY Sno DESC, Cno ASC;
- 산술식과 문자 스트링이 명세된 검색(이름 변경)
  - SELECT Sno AS 학번, '중간시험 = ' AS 시험,
    Midterm + 3 AS 점수
    FROM ENROL
    WHERE Cno = 'C312';
- 복수 테이블로부터의 검색(조인)
  - SELECT S.Sname, S.Dept, E.Grade
    FROM STUDENT S, ENROL E
    WHERE S.Sno = E.Sno AND E.Cno = 'C413';
- 자기 자신의 테이블에 조인하는 검색
  - 같은 학과 학생의 학번을 쌍으로 검색하라.
    단 첫 번째 학번은 두 번째 학번 보다 작다
  - SELECT S1.Sno, S2.Sno
    FROM STUDENT S1, STUDENT S2
    WHERE S1.Dept = S2.Dept
    AND S1.Sno < S2.Sno;
- FROM 절에 조인 명세
  - SELECT Sname, Dept, Grade
    FROM STUDENT JOIN ENROL ON
    (STUDENT.Sno=ENROL.Sno)
    WHERE ENROL.Cno = 'C413';
  - WHERE ENROL.Cno = 'C413';
    SELECT Sname, Dept, Grade

```
  FROM STUDENT JOIN ENROL USING(Sno)
  WHERE ENROL.Cno = 'C413';
- SELECT Sname, Dept, Grade
  FROM STUDENT NATURAL JOIN ENROL
  WHERE ENROL.Cno = 'C413';
```

- 집계 함수(aggregate function)를 이용한 검색
  - 집계 함수 : COUNT, SUM, AVG, MAX, MIN
    - ▸ SELECT COUNT(*) AS 학생 수
      FROM STUDENT;
    - ▸ SELECT COUNT(DISTINCT Cno)
      FROM ENROL
      WHERE Sno = 300;
    - ▸ SELECT AVG(Midterm) AS 중간평균
      FROM ENROL
      WHERE Cno = ‘C413’;
- GROUP BY를 이용한 검색
  - SELECT Cno, AVG(Final) AS 기말평균
    FROM ENROL
    GROUP BY Cno;
- HAVING을 사용한 검색
  - SELECT Cno, AVG(Final) AS 평균
    FROM ENROL
    GROUP BY Cno
    HAVING COUNT(*) >= 3;

2) 데이터 삽입

- 일반 형식
  - INSERT
    INTO 테이블 [(열_이름_리스트)]
    VALUES (열값_리스트);

- INSERT
  INTO 테이블 [(열_이름_리스트)]
  SELECT 문;

• 레코드의 직접 삽입
- INSERT
  INTO STUDENT(Sno, Sname, Year, Dept)
  VALUES (600, '박상철', 1, '컴퓨터');
  INSERT
- INTO STUDENT
  VALUES (600, '박상철', 1, '컴퓨터');
- INSERT
  INTO STUDENT(Sno, Sname, Year)
  VALUES (600, '박상철', 1);

• 부속 질의문을 이용한 레코드 삽입
- INSERT
  INTO COMPUTER(Sno, Sname, Year)
  SELECT Sno, Sname, Year
  FROM STUDENT
  WHERE Dept = '컴퓨터';

3) 데이터 삭제

• 일반 형식
- DELETE
  FROM 테이블
  [WHERE 조건];

• 하나의 레코드 삭제
- DELETE
  FROM STUDENT
  WHERE Sno = 100;
  Note : 참조 무결성 문제 발생 가능

- 복수의 레코드 삭제
  - DELETE
    FROM ENROL;
- 부속 질의문을 사용한 삭제
  - DELETE
    FROM ENROL
    WHERE Cno = 'C413' AND Final < 60
    AND ENROL.Sno IN
    (SELECT Sno
    FROM STUDENT
    WHERE Dept = '컴퓨터');

4) SQL 뷰(View)

- 부속 질의문을 이용한 레코드 삽입
  - INSERT
    INTO COMPUTER(Sno, Sname, Year)
    SELECT Sno, Sname, Year
    FROM STUDENT
    WHERE Dept = '컴퓨터';

5) 뷰의 생성

- 일반형식
  - CREATE VIEW 뷰_이름[(열_이름 리스트)]
    AS SELECT문
    [WITH CHECK OPTION];
    ▸ WITH CHECK OPTION : 갱신이나 삽입 연산 시 뷰 정의 조건을 검사
- 예

```
- CREATE VIEW CSTUDENT(Sno, Sname, Year)
  AS SELECT Sno, Sname, Year
  FROM STUDENT
  WHERE Dept = '컴퓨터'
  WITH CHECK OPTION;
```

6) 뷰의 제거

- 일반형식
  - DROP VIEW 뷰_이름 { RESTRICT | CASCADE };
    - ▸ RESTRICT
    - ▸ 다른 곳에서 참조되고 있지 않는 한 데이터베이스에서 제거
    - ▸ CASCADE
    - ▸ 이 뷰가 사용된 다른 모든 뷰나 제약 조건이 함께 제거
- 예
  - DROP VIEW DEPTSIZE RESTRICT;

7) 뷰의 조작연산

- 기본 테이블에 사용 가능한 검색(SELECT)문은 모두 뷰에 사용 가능
- 변경(삽입, 삭제, 갱신) 연산은 제약
  - 열 부분 집합 뷰(Column Subset View)
    - ▸ CREATE VIEW STUDENT_VIEW1
      AS SELECT Sno, Dept
      FROM STUDENT;
      → 기본 키 포함 : 이론적으로 삽입, 삭제, 갱신 가능
    - ▸ CREATE VIEW STUDENT_VIEW2
      AS SELECT Sname, Dept
      FROM STUDENT;
      → 기본 키 불포함 : 이론적으로 삽입, 삭제, 갱신 불가
- 행 부분 집합 뷰(Row Subset View)
    - ▸ CREATE VIEW STUDENT_VIEW3

```
AS SELECT Sno, Sname, Year, Dept
FROM STUDENT
WHERE Year=4;
```

→ 기본 키 포함 : 이론적으로 삽입, 삭제, 갱신 가능

8) 뷰의 장단점

- 뷰의 장점
  - 논리적 독립성을 제공(확장, 구조 변경)
  - 데이터의 접근을 제어(보안)
  - 사용자의 데이터 관리를 단순화
  - 여러 사용자에 다양한 데이터 요구를 지원
- 뷰의 단점
  - 정의를 변경할 수 없음
  - 삽입, 삭제, 갱신 연산에 제한이 많음

7주차 - 1차시	관계 데이터 연산 1

1. 이상(Anomaly)

1) 데이터의 논리적 표현

◆ 릴레이션 스키마(Relation Schema)의 설계

- 논리적 데이터베이스 설계
- 관계 모델을 이용하여 어떻게 실세계를 정확히 표현할 것인가?
  - 애트리뷰트, 엔티티, 관계성을 파악
  - 관련된 애트리뷰트들을 릴레이션으로 묶음

고려사항	① 애트리뷰트들간의 관계성(Relationship) : 데이터 종속성(Data Dependency) ② 효율적인 데이터 처리 ③ 데이터의 일관성

  - 변칙적 성질의 예방
  - 데이터 변경 시의 이상(Anomaly)

2) 이상(Anomaly)

◆ Example : 수강 릴레이션

- 수강
  - 기본 키 : {학번, 과목 번호}

학번	과목번호	성적	학년
100	C413	A	4
100	E412	A	4
200	C123	B	3
300	C312	A	1
300	C324	C	1
300	C413	A	1
400	C312	A	4
400	C324	A	4
400	C413	B	4
400	E412	C	4
500	C312	B	2

◆ 삭제 이상(Deletion Anomaly)

- 200번 학생이 'C123'의 등록을 취소

  ⇒ 3학년이라는 정보도 함께 삭제됨

- 연쇄 삭제(Triggered Deletion)에 의한 정보 손실(Loss of Information)

◆ 삽입 이상(Insertion Anomaly)

- 600번 학생이 2학년이라는 사실을 삽입

  ⇒ 어떤 과목을 등록하지 않는 한 삽입이 불가능

  (∵ 과목 번호가 기본 키)

- 원하지 않는 정보의 강제 삽입

◆ 갱신 이상(Update Anomaly)

- 400번 학생의 학년을 4에서 3으로 변경

  ⇒ 학번 400에 대한 4개의 투플 모두를 변경해야 함

- 중복 데이터의 일부 갱신으로 정보의 모순성(Inconsistency)이 발생

3) 이상의 원인과 해결책

◆ 이상의 원인

- 하나의 개체에 속한 애트리뷰트들 간에 존재하는 여러 개의 종속관계(Dependency)를 하나의 릴레이션으로 표현하기 때문

◆ 이상의 해결

- 애트리뷰트들 간의 여러 종속관계를 분해(Decomposition) 하여 각각

하나의 릴레이션으로 표현

- 하나의 종속성은 하나의 릴레이션으로 표현

→ 정규화(Normalization) 과정

4) 스키마 설계와 변환

◆ 스키마 설계 : 데이터베이스의 논리적 설계

① 애트리뷰트들과 이들의 제약조건(종속성)들을 수집

② 수집된 결과를 명시된 제약조건에 따라 바람직한 여러 개의 릴레이션으로 분할

→ 스키마 변환(Schema Transformation) 성이나

◆ 스키마 변환의 원리

① 정보 표현의 무손실(Nonloss Representation of Information) : 개선된 구조

② 최소의 데이터 중복(Minimal Data Redundancy)

③ 분리의 원칙(Principle of Separation) : 독립된 관계성은 별도의 릴레이션으로 분리시켜 표현

2. 함수 종속(FD)

1) 함수 종속(FD)

◆ 함수 종속(Functional Dependency : FD)의 정의

- 어떤 릴레이션 R에서, 애트리뷰트 X의 값 각각에 대해 애트리뷰트 Y의 값이 최대 하나만 연관

  ↔ 애트리뷰트 Y는 애트리뷰트 X에 함수 종속 X → Y

- 애트리뷰트 X는 Y를 (함수적으로) 결정

  - X를 결정자(Determinant)

  - Y를 종속자(Dependent)

- X나 Y는 각각 두 개 이상의 애트리뷰트 집합이 될 수 있음

◆ Note

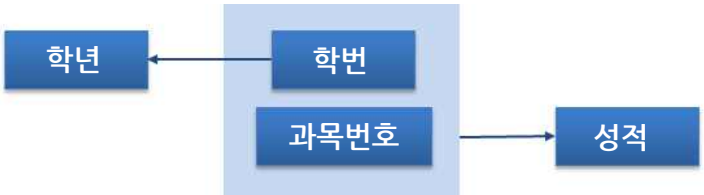
- 릴레이션 R에서 애트리뷰트 X가 키이면, R의 모든 애트리뷰트 Y에 대해 X → Y 성립
- 함수종속 X → Y의 경우 애트리뷰트 X가 반드시 키(유일 값)라는 것을 요건으로 하지 않음
  즉, X의 어느 한 값에 대응되는 Y 값을 갖는 투플이 둘 이상 존재 가능

2) 함수 종속 다이어그램

◆ 함수 관계의 표현

◆ 수강 릴레이션 : 수강(학번, 과목번호, 학년, 성적)

- 기본 키 : { 학번, 과목번호 }
  {학번, 과목번호} → 성적
  학번 → 학년



3) 완전 함수 종속과 부분 함수 종속

- 2개 이상으로 구성된 애트리뷰트 집합 X에 대하여 X → Y가 성립할 때
- 완전 함수 종속(Full Functional Dependency)
  - X' ⊂ X 이고 X' → Y 가 성립되는 애트리뷰트 X'이 존재하지 않는 경우
- 부분 함수 종속(Partial Functional Dependency)
  - X' ⊂ X 이고 X' → Y 가 성립되는 애트리뷰트 X'이 존재하는 경우
- 예 : 수강 릴레이션의 함수 종속
  - 학번 → 학년
    {학번, 과목번호} → 성적

◆ Note

① "학년"은 "학번"에 완전 함수 종속,

② "학년"은 {학번, 과목번호}에 부분 함수 종속,

③ "성적"은 {학번, 과목번호}에 완전 함수 종속

- 함수 종속에 대한 추론 규칙(Inference Rules)

  R1: (반사, Reflexive) A ⊇ B이면 A → B이다.

  또한 A → A이다.

  R2 : (첨가, Augmentation) A → B이면 AC → BC이고 AC → B이다.

  R3 : (이행, Transitive) A → B이고 B → C이면 A → C이다.

  R4 : (분해, Decomposition) A → BC이면 A → B이다.

  R5 : (결합, Union) A → B이고 A → C이면 A → BC이다.

◆ Note

- 함수 종속은 데이터의 의미(Data Semantics)를 표현
  - 예 : "학번 → 학년"의 의미는 "학생은 한 학년에만 속한다"
  - 의미적 제약조건(Semantic Constraint)
- DBMS는 함수 종속을 유지하기 위하여 함수 종속을 스키마에 명세하는 방법과 함수 종속을 보장하는 방법을 제공해야 함

4) 기본 정규형

◆ 정규형(Normal Form)

- 어떤 일련의 제약조건(Constraints)을 만족하는 릴레이션
- 정규화(Normalization) 즉 스키마 변환 (S → S')으로 정규형을 만듦

◆ 정규화(Normalization)의 원칙

① 정보 표현의 무손실
  - 같은 의미의 정보 유지
  - 효율적인 구조

② 최소의 데이터 중복

③ 분리의 원칙
  - 독립적인 관계는 별개의 릴레이션으로 표현
  - 릴레이션 각각에 대해 독립적 처리가 가능

7주차 - 2차시	관계 데이터 연산 2

1. 제1정규형(1NF)

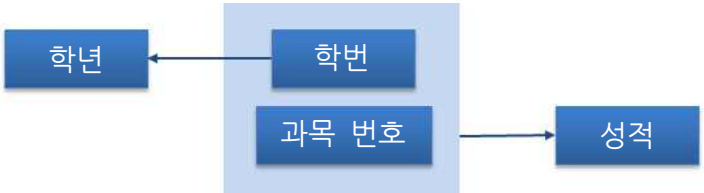
1) 제1정규형(1NF)

◆ First Normal Form(1NF)의 정의

- 모든 도메인이 원자 값(Atomic Value)만으로 된 릴레이션

◆ 예 : 수강지도 릴레이션

- 수강지도 (학번, 지도교수, 학과, 과목번호, 성적)
- 기본 키 : {학번, 과목번호}
- 함수 종속 : {학번, 과목번호} → 성적
  학번 → 지도교수
  학번 → 학과
  지도교수 → 학과



- 수강 지도

학번	지도교수	학과	과목번호	성적
100	P1	컴퓨터	C413	A
100	P1	컴퓨터	E412	A
200	P2	전기	C123	B
300	P3	컴퓨터	C312	A
300	P3	컴퓨터	C324	C
300	P3	컴퓨터	C413	A
400	P1	컴퓨터	C312	A
400	P1	컴퓨터	C324	A
400	P1	컴퓨터	C413	B
400	P1	컴퓨터	E412	C

◆ 1NF에서의 이상

- 삽입이상 : 500번 학생의 지도교수가 P4라는 사실을 삽입하려 할 때 어떤

과목을 등록하지 않는 한 삽입은 불가능

- 삭제이상 : 200번 학생이 C123의 등록을 취소하여 이 투플이 삭제되는 경우에 지도교수가 P2라는 정보까지 손실됨
- 갱신이상 : 400번 학생의 지도교수를 P1에서 P3로 변경하는 경우에 학번이 400인 4개의 투플에 대한 지도교수를 모두 P3로 변경해야 함

◆ 1NF 이상의 원인

- 기본 키에 부분 함수 종속된 애트리뷰트가 존재

→ 기본 키로 식별되는 개체와 무관한 애트리뷰트가 존재

→ 두 가지 독립적인(상이한) 정보가 하나의 릴레이션으로 표현

- 1NF 이상의 해결

- 프로젝션으로 1NF 릴레이션을 분해(부분 함수 종속을 제거)

→ 2NF

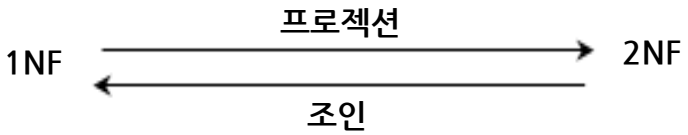
2. 제2정규형(2NF)와 제3정규형(3NF)

1) 제2정규형(2NF)

◆ Second Normal Form(2NF)의 정의

- 1NF이고, 키에 속하지 않는 애트리뷰트들은 모두 기본 키에 완전 함수 종속

◆ 무손실 분해(Nonloss Decomposition)



- 프로젝션으로 분해된 릴레이션들은 자연 조인을 통해 원래의 릴레이션으로 복귀 가능
- 원래의 릴레이션에서 얻을 수 있는 정보는 분해된 릴레이션들로부터도 얻을 수 있음

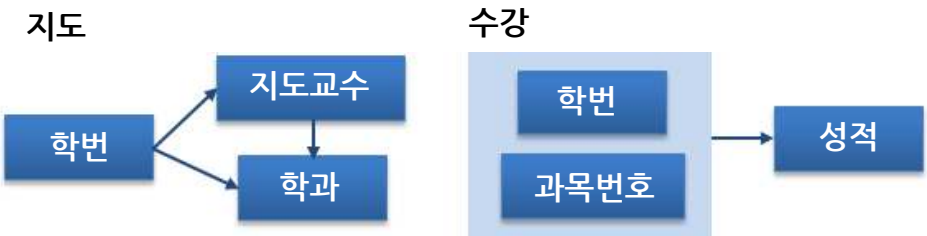
  그러나, 그 역은 성립하지 않음 (500번 학생의 지도교수가 P4라는 정보는 원래의 1NF 릴레이션에서는 표현할 수 없음)

◆ Note : Heath의 무손실 분해 정리

- R(A,B,C)에서 함수 종속 A → B가 성립하면
  ⇒ R1(A,B), R2(A,C) 로 무손실 분해가 가능
- 정보의 손실 없이 릴레이션을 분해

◆ 예 : 수강지도 → 지도, 수강 릴레이션으로 분해

- 지도 (학번, 지도교수, 학과)
  기본 키 : {학번}
- 수강 (학번, 과목번호, 성적)
  기본 키 : {학번, 과목번호}
  외래 키 : {학번} 참조 : 지도



- 지도

학번	지도교수	학과
100	P1	컴퓨터
200	P2	전기
300	P3	컴퓨터
400	P1	컴퓨터

- 수강

학번	과목번호	성적
100	C413	A
100	E412	A
200	C123	B
300	C312	A
300	C324	C
300	C413	A
400	C312	A
400	C324	A
400	C413	B
400	E412	C

◆ 2NF(지도 릴레이션)에서의 이상

- 삽입이상 : 어떤 지도교수가 특정 학과에 속한다는 사실의 삽입이 불가능

- 삭제이상 : 300번 학생의 투플을 삭제하면 지도교수 P3가 컴퓨터공학과에 속한다는 정보를 상실
- 갱신이상 : 지도교수 P1의 소속을 컴퓨터공학과에서 전자과로 변경한다면 학번이 100과 400인 두개의 투플을 모두 변경해야 함

◆ 2NF 이상의 원인

- 이행적 함수 종속이 존재
  - 프로젝션으로 2NF 릴레이션을 분해(이행적 함수 종속을 제거) ⇒ 3NF

◆ Note

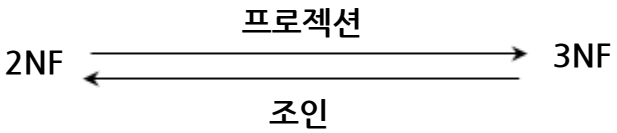
- 이행적 함수 종속 (TD, Transitive Dependency) 릴레이션 R에 함수 종속 A → C가 존재하고 함수 종속
  A → B와 B → C가 성립할 때 애트리뷰트 C는 애트리뷰트 A에 이행적 함수 종속이라고 한다.
- 2NF 이상의 해결

2) 제3정규형(3NF)

◆ Third Normal Form(3NF)의 정의

- 2NF이고, 키가 아닌 모든 애트리뷰트들은 기본 키에 이행적 함수 종속이 되지 않음

◆ 무손실 분해



- 원래의 릴레이션에서 얻을 수 있는 정보는 분해된 릴레이션들로부터도 얻을 수 있으나 그 역은 성립하지 않음(지도교수 P4가 수학과에 속한다는 정보를 3NF에서 표현 가능)

◆ 예 : 지도 → 학생지도와 지도교수학과로 분해

- 학생지도(학번, 지도교수)
  기본 키 : {학번}
  외래 키 : {지도교수} 참조 : 지도교수학과
- 지도교수학과(지도교수, 학과)

기본 키 : {지도교수}

- 학생지도

학번	지도교수
100	P1
200	P2
300	P3
400	P1

- 지도교수학과

지도교수	학과
P1	컴퓨터
P2	전기
P3	컴퓨터

◆ Note

- 키가 아닌 애트리뷰트 값의 갱신 시 불필요한 부작용(이상)은 발생하지 않음
- 모든 이원 릴레이션은 3NF에 속함

◆ 3NF의 약점

- 적용할 수 없는 경우

① 복수의 후보 키를 가지고 있고

② 후보 키들이 두 개 이상의 애트리뷰트들로 구성되고

③ 후보 키들의 애트리뷰트가 서로 중첩되는 경우

⇒ 보다 일반적인 Boyce/Codd Normal Form(BCNF)을 제안

◆ 예(3NF) : 수강과목(학번, 과목, 교수) 릴레이션

- 제약조건

- 한 학생은 각 과목에 대해 오직 한 교수의 강의만 수강
- 각 교수는 한 과목만 담당

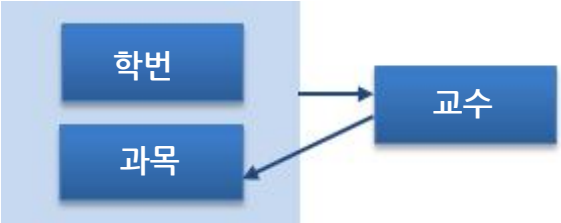
- 같은 과목을 여러 교수가 담당할 수 있음
  - ▸ 수강과목 (학번, 과목, 교수)

    후보 키 : {학번, 과목}, {학번, 교수}

    기본 키 : {학번, 과목}

    함수종속 : {학번, 과목} → 교수

    교수 → 과목



학번	과목	성적
100	프로그래밍	P1
100	자료구조	P2
200	프로그래밍	P1
200	자료구조	P3
300	자료구조	P3
300	프로그래밍	P4

◆ 3NF(수강과목 릴레이션)에서의 이상

- 삽입이상 : 교수 P5가 자료 구조 과목을 담당한다는 사실의 삽입은 학번(수강학생)이 있어야 가능
- 삭제이상 : 100번 학생이 자료 구조를 취소하여 투플을 삭제하게 되면 P2가 담당교수라는 정보도 삭제됨
- 갱신이상 : P1이 프로그래밍 과목 대신 자료 구조를 담당하게 되면 P1이 나타난 모든 투플을 변경하여야 함 → 원인 : 교수가 결정자이지만 후보 키가 아님

3) 보이스/코드 정규형(BCNF)

◆ Boyce/Codd Normal Form(BCNF) 정의

- 릴레이션 R의 결정자(Determinant)가 모두 후보 키이면 릴레이션 R은

BCNF에 속한다.

- 강한 제3정규형(Strong 3NF)이라고도 함
- 릴레이션 R이 BCNF에 속하면 R은 제1, 제2, 제3 정규형에 속함

◆ 예(BCNF) : 수강과목 → 수강교수, 과목교수

수강교수(학번, 교수) 기본 키 : {학번, 교수} 외래 키 : {교수} 참조 : 과목교수 과목교수(교수, 과목) 기본 키 : {교수}

- 수강교수

학번	교수
100	P1
100	P2
200	P1
200	P3
300	P3
300	P4

- 과목교수

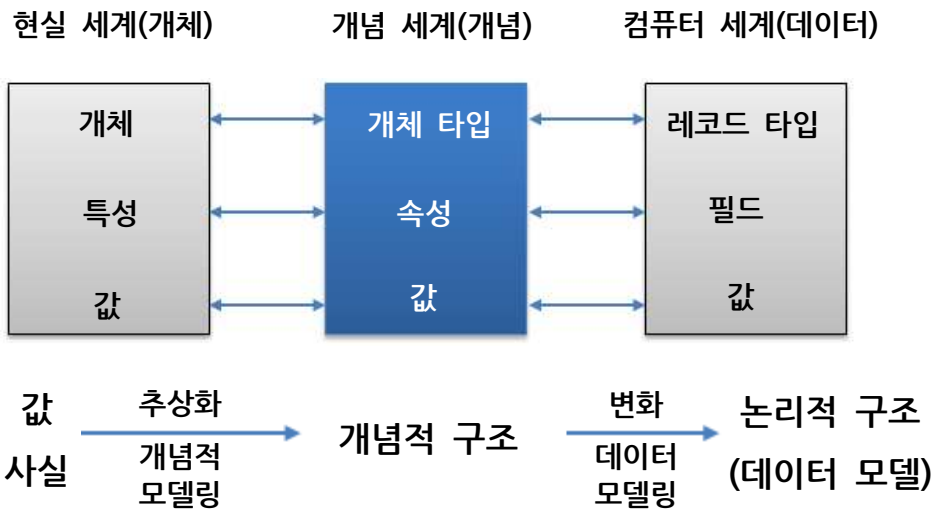
성적	과목
P1	프로그래밍
P2	자료구조
P3	자료구조
P4	프로그래밍

9주차 - 1차시	데이터 모델링

1. 데이터의 세계

1) 데이터의 세계 1

◆ 데이터에 대한 3개의 세계



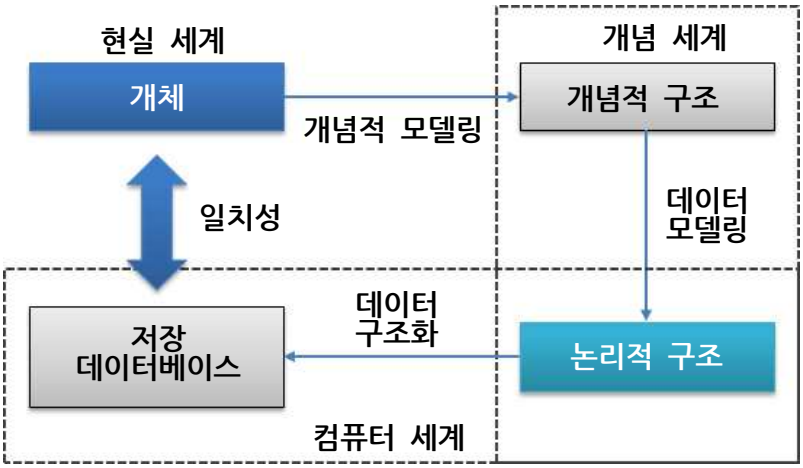
2) 데이터의 세계 2

◆ 현실 세계(Real World)의 구조적 표현 : 데이터베이스 설계(Database Design)

- 개념적 모델링(Conceptual Modeling)
  - 현실 세계를 추상적 개념(Abstract Concepts), 개체 타입(Entity Type)으로 표현
  - 개념적 구조(Conceptual Structure)
  - 개념적 설계(Conceptual Design)
- 데이터 모델링(Logical Modeling)
  - 개념적 구조를 논리적 개념(Logical Concepts), 레코드 타입(Record Type)으로 표현
  - 논리적 구조(Logical Structure), 데이터 모델(Data Model)

- 논리적 설계(Logical Design)
- 데이터 구조화(Data Structuring)
  - 논리적 구조를 물리적 구조(Physical Structure)로 표현
  - 물리적 설계(Physical Design)
  - 저장 장치에서의 데이터 표현

3) 데이터의 세계 3



2. Data Modeling, Entity Type, Relationship Type

1) Data Modeling

◆ 데이터베이스 설계(Database Design) 과정의 일부
- 설계의 핵심은 개념적인 구조와 논리적인 구조를 거쳐 실제 데이터를 저장할 수 있는 물리적 구조로 변환하는 것

◆ 개념적 데이터 모델링(Conceptual Data Modeling)
- 애트리뷰트(Attribute)들로 기술된 개체 타입(Entity Type)과 이 개체 타입들 간의 관계(Relationship)를 이용하여 현실 세계를 표현하는 방법
- 가장 대표적인 모델이 개체-관계 모델(E-R Model : Entity-Relationship Model)

◆ 논리적 데이터 모델링(Logical Data Modeling)
- 데이터 필드(Field)로 기술된 레코드 타입(Record Type)과 이 레코드

타입들 간의 관계를 이용하여 개념 세계를 표현하는 방법

- 관계 데이터 모델(Relational Data Model), 네트워크 데이터 모델(Network Data Model), 계층 데이터 모델(Hierarchical Data Model), 객체 지향 데이터 모델(Object-oriented Data Model), 객체-관계 데이터 모델(Object-relational Data Model)

2) Data Model 1

◆ 데이터 모델(Data Model : D)

- D = 〈S, O, C〉
- S : 구조(Structure)
  - 데이터의 정적 성질(Static Properties)
  - 개체 타입과 이들 간의 관계를 명세
- O : 연산(Operation)
  - 데이터의 동적 성질(Dynamic Properties)
  - 개체 인스턴스(Instance)를 처리하는 작업에 대한 명세
  - 데이터의 조작 기법
- C : 제약 조건(Constraints)
  - 데이터의 논리적 제약(Logical Constraints)
    - ▸ 개체 인스턴스의 존재 조건
    - : 구조(S)로부터 파생
    - : 의미상 제약
  - 데이터 조작의 한계를 표현하는 규정

3) Data Model 2

◆ 논리적 데이터 모델 중에서 가장 많이 사용되고 있는 모델은 관계 데이터 모델(Relational Data Model)

◆ 그 이전에는 네트워크 데이터 모델(Network Data Model)과 계층 데이터 모델(Hierarchical Data Model)이 많이 사용됨

◆ 최근에는 객체 지향 데이터 모델(Object-oriented Data Model)과 객체-관계 데이터 모델(Object-relational Data Model)이 사용

◆ DBMS는 하나의 논리적 데이터 모델만을 구현

- 모델 간의 주요 차이점은 데이터 요소 간의 관계(Relationships)를 표현하는 방식

4) Entity Type 1

◆ 개체(Entity) : 단독으로 존재할 수 있으며 다른 것과 구별되는 객체(Object)

◆ 개체 타입(Entity Type)

- 이름(Name)과 애트리뷰트(Attribute)들로 정의됨
- 개체 집합(Entity Set)
  - 개체 타입에 대한 개체 인스턴스(Instance)들의 집합

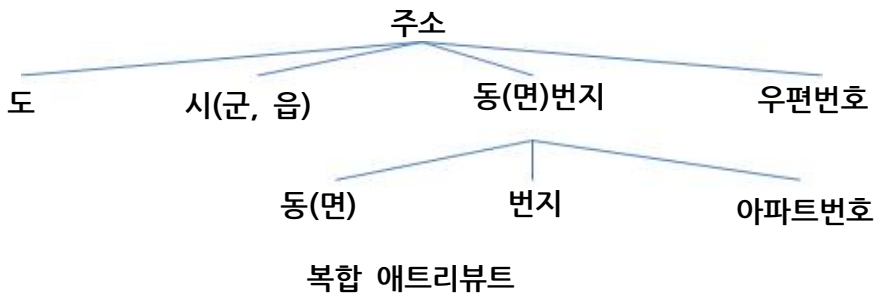
◆ 애트리뷰트(Attribute)의 유형

- 단순 애트리뷰트와 복합 애트리뷰트
- 단일 값 애트리뷰트와 다중 값 애트리뷰트
- 유도 애트리뷰트와 저장 애트리뷰트
- 널(Null) 애트리뷰트

5) Entity Type 2

◆ 단순 애트리뷰트와 복합 애트리뷰트

- 단순 애트리뷰트(Simple Attribute)
  : 더 이상 작은 구성원소로 분해할 수 없는 애트리뷰트
- 복합 애트리뷰트(Composite Attribute)는 몇 개의 기본적인 단순 애트리뷰트로 분해할 수 있는 애트리뷰트



6) Entity Type 3

◆ 단일 값 애트리뷰트와 다중 값 애트리뷰트

- 단일 값 애트리뷰트(Single-valued Attribute)
  - 대부분의 애트리뷰트들은 특정 개체에 대해 하나의 값을 가짐
  - “학생” 개체 타입의 Name(이름) 애트리뷰트 : 각 개체에 대해 하나의 값만 가짐
- 다중 값 애트리뷰트(Multivalued Attribute)
  - 한 개체에 대해서 몇 개의 값을 가지고 있을 수 있음
  - “학생” 개체 타입에 Hobby(취미) 애트리뷰트 : 학생에 따라서는 두 개 이상의 취미를 가질 수 있음
  - 현실적으로 다중 값 애트리뷰트로 처리하기 위해서 값의 최대 수 제한 가능

7) Entity Type 4

◆ 유도 애트리뷰트와 저장 애트리뷰트

- 유도 애트리뷰트(Derived Attribute)
  - 애트리뷰트의 값이 다른 애트리뷰트나 개체가가지고 있는 값으로부터 유도되어 결정되는 경우
- 저장 애트리뷰트(Stored Attribute)
  - 유도 애트리뷰트를 생성하는데 사용된 애트리뷰트
- 과목별 평균 성적을 표현하는 “과목-성적” 개체 타입의 “평균 성적” 애트리뷰트인 경우
  - “평균 성적” 애트리뷰트의 값은 등록 개체 타입의 “성적” 애트리뷰트 값을 평균한 결과로 결정
  - “평균 성적” 애트리뷰트 : 유도 애트리뷰트
  - 이를 위해 사용된 “성적” 애트리뷰트 : 저장 애트리뷰트

8) Entity Type 5

◆ 널 애트리뷰트(Null Attribute)

- 널 값(Null Value)을 갖는 애트리뷰트
- 널 값은 어떤 개체 인스턴스가 어느 특정 애트리뷰트에 대한 값을 가지고 있지 않을 때 이를 명시적으로 표시하기 위해 사용

◆ 널(Null) 값을 갖는 경우

- 그 애트리뷰트 값이 그 개체에 “해당되지 않는”(Not Applicable) 경우
- 그 애트리뷰트 값을 “알 수 없는”(Unknown) 경우
  - 그 값이 존재하지만 값이 “누락”(Missing)인 경우
  - 그 값이 존재하고 있는지 조차 알 수 없어 “모르는”(Not Known) 경우

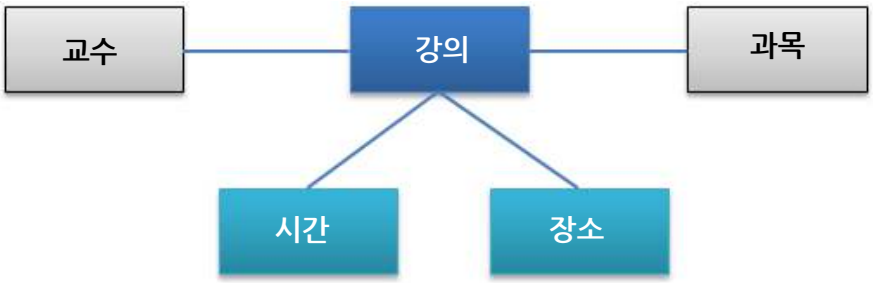
9) Relationship Type

◆ 관계 타입(Relationship Type)

- 개체 집합(타입)들 사이의 대응(Correspondence), 즉 사상(Mapping)을 의미

“등록” Relationship Type

10) Relationship Type의 특성 1



Attributes를 가진 Relationship Type

11) Relationship Type의 특성 2

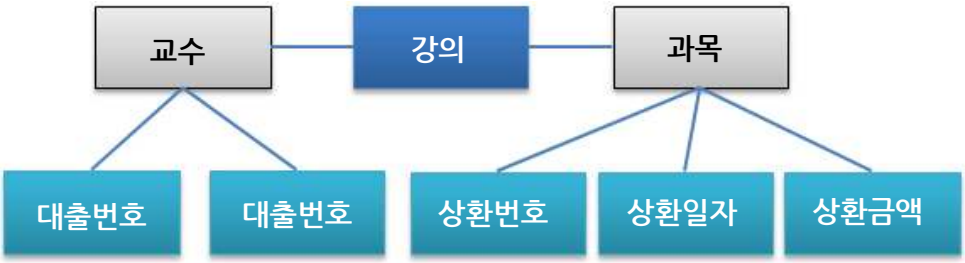
◆ 관계 타입의 참여 제약조건(Participation Constraint)

- 전체 참여(Total Participation)
  - A-B 관계에서 개체 집합 A의 모든 개체가 A-B 관계에 참여
  - Ex) 교수-학과 관계
- 부분 참여(Partial Participation)
  - A-B 관계에서 집합 A의 일부 개체만 A-B 관계에 참여
  - Ex) 학생-과목 관계에서 학생(휴학생 제외) 일부만 참여

12) Relationship Type의 특성 3

◆ 존재 종속(Existence Dependence)

- 어떤 개체 b의 존재가 개체 a의 존재에 종속됨
- b는 a에 존재 종속
- a : 주 개체(Dominant Entity) b : 종속 개체(Subordinate Entity)



대출상환 관계 (주 개체 : 대출, 종속 개체 : 상환)

9주차 - 2차시	ERD

1. 데이터의 세계

1) Entity-Relationship model(E-R model)

- 개체-관계 모델(Entity-Relationship Model)
  - 개념적 데이터 모델(Conceptual Data Model)
- 현실 세계의 개념적 표현
- 개체 타입(집합)과 관계 타입(집합)을 기본 개념으로 현실 세계를 개념적으로 표현하는 방법
- 개체 집합(Entity Set) : 한 개체 타입에 속하는 모든 개체 인스턴스(Entity Instance)
- 관계 집합(Relationship Set) : 한 관계 타입에 속하는 모든 관계 인스턴스(Relationship Instance)

2) E-R Diagram 1

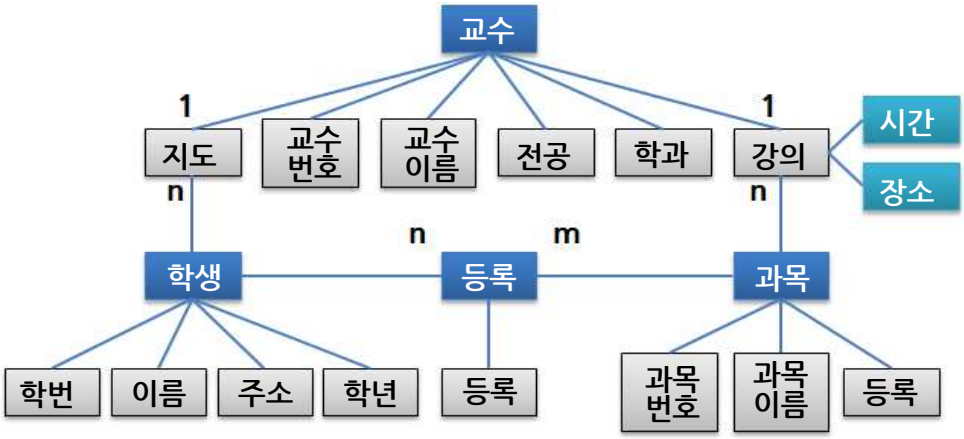
- 1976. Peter Chen
- E-R 모델의 그래픽 표현

개체 타입(Entity Type)	
관계 타입(Relationship Type)	
속성(Attribute)	
링크(Link)	

- 레이블(Label) : 관계의 사상 원소 수를 표현
  - 일대일(1:1), 일대다(1:n), 다대일(n:1), 다대다(n:m)

3) E-R Diagram 2

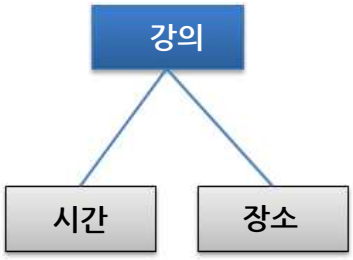
◆ Example



4) E-R Diagram 3

◆ 특징

- 다 대 다(m : n) 관계 표현
- 다원 관계(N-ary Relationship) 표현 : 두개 이상의 개체 타입이 하나의 관계에 관련 가능
- 다중 관계(Multiple Relationship) 표현 : 두 개체 타입 사이에 둘 이상의 관계가 존재 가능
- 관계 타입도 속성(Attribute)을 가질 수 있음



5) Entity Type과 Key Attributes

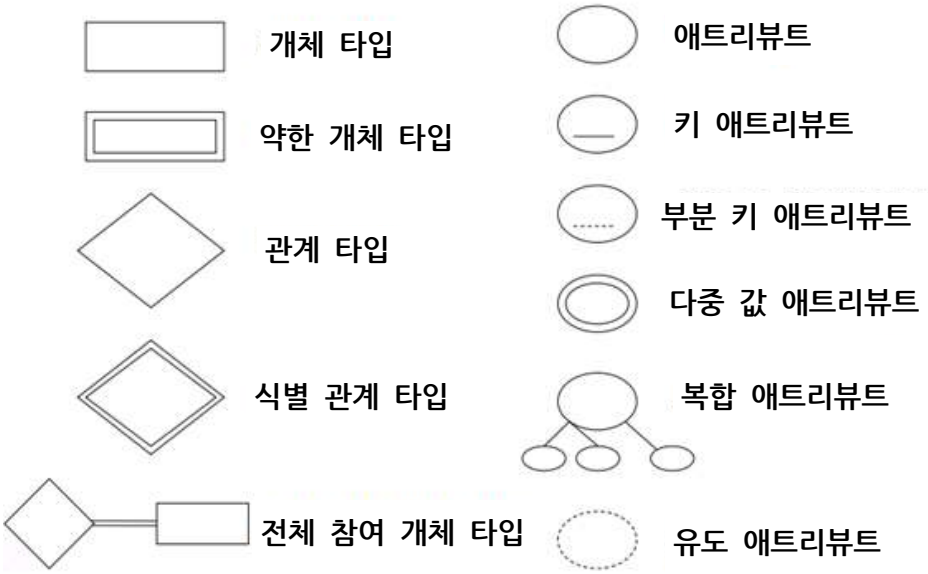
◆ 키 애트리뷰트(Key Attributes)

- 한 개체 집합 내에서 각 개체마다 상이한 값을 갖는 애트리뷰트나 애트리뷰트 집합
- 키(Key)
  - 개체 타입 내의 모든 개체 인스턴스들을 유일하게 식별
  - 동일한 키 값을 갖는 두 개의 개체 인스턴스는 없음
- E-R 다이어그램 상에서 밑줄로 표시

6) Weak Entity Type

- 약한 개체 타입(Weak Entity Type)
  - 자기 자신의 애트리뷰트만으로는 키를 명세할 수 없는 개체 타입 ↔ 강한 개체 타입(Strong Entity Type)
  - 주 개체(Dominant Entity) - 강한 개체 타입, 종속 개체(Subordinate Entity) - 약한 개체 타입
- 구별자(Discriminator)
  - 강한 개체와 연관된 약한 개체 집합 내에서만 이들을 서로 구별할 수 있는 애트리뷰트
  - 부분 키(Partial Key)
- 식별 관계 타입(Identifying Relationship Type)
  - 약한 개체를 강한 개체에 연관시켜 개체를 유일하게 식별하게 하는 관계

7) E-R Diagram 표기법

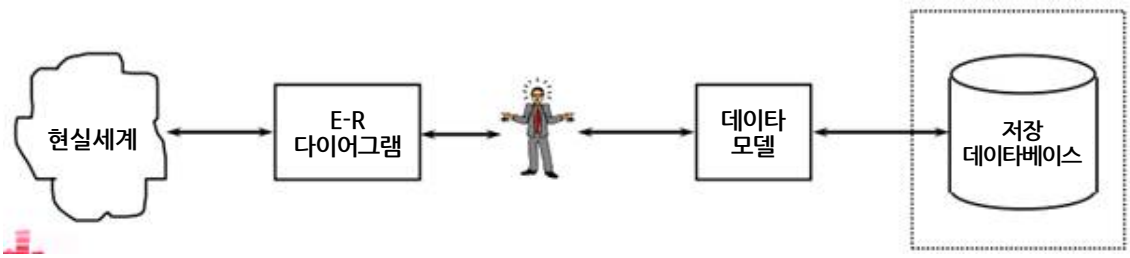


2. Data Model

1) Logical Data Model

◆ 논리적 데이터 모델(Logical Data Model)

- 개념적 구조를 데이터베이스로 구현하기 위한 중간 단계로 논리적 개념인 레코드(개체) 타입과 관계로 표현
- 개념적 데이터 모델(Conceptual Data Model)
  - 개체-관계 데이터 모델
  - 현실 세계를 추상적 개념인 개체 타입과 관계 타입으로 표현



2) Data Model

- 네트워크 데이터 모델(Network Data Model)
- 계층 데이터 모델(Hierarchical Data Model)

- 관계 데이터 모델(Relational Data Model)
- 객체지향 데이터 모델(Object-oriented Data Model)
  - 객체 데이터 모델(Object Data Model)
- 객체-관계 데이터 모델(Object-relational Data Model)
  - 확장 관계 데이터 모델(Extended Relational Data Model)

3) Relational Data Model

◆ 데이터베이스
- 릴레이션(Relation) - 테이블(Table)의 집합
- 개체 릴레이션, 관계 릴레이션

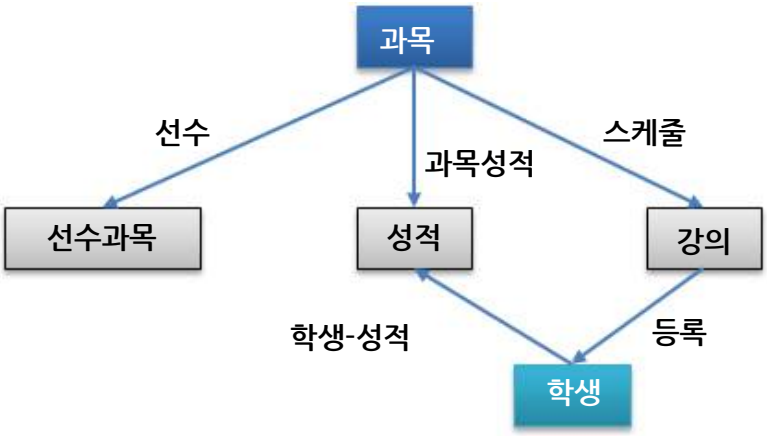
◆ 릴레이션 스키마(Relation Schema)
- 개체와 관계성을 모두 릴레이션(테이블)으로 정의

학생	**학번**	**이름**	**주소**	**학년**
교수	**교수번호**	**교수이름**	**전공**	**학과**
과목	**과목번호**	**과목이름**	**학점**	
지도	**교수번호**	**학번**		
등록	**학번**	**과목번호**	**성적**	
강의	**교수번호**	**과목번호**	**시간**	**장소**

4) Data Structure Diagram 1

◆ 자료구조도(Data Structure Diagram)
- 레코드 타입 간의 관계를 도형으로 표현

◆ Bachman 다이어그램(1969)



5) Data Structure Diagram 2

◆ 구성 요소

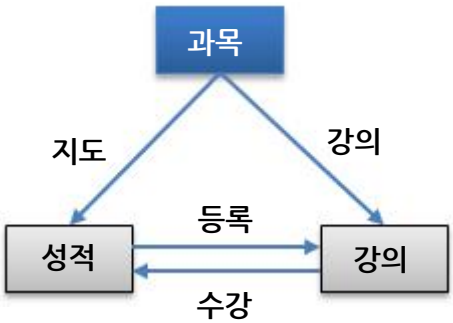
- 사각형 노드 : 레코드 타입 (개체 타입)
- 링크(link; arc, →) : 레코드 타입 간의 일대다 (1:n) 관계
- 레이블 : 관계 이름

◆ 스키마 다이어그램(Schema Diagram)

- 데이터베이스 스키마를 데이터 구조도로 표현 : 스키마의 도형적 표현
- 트리(Tree) 형태 : 계층 데이터 모델
- 그래프(Graph) 형태 : 네트워크 데이터 모델

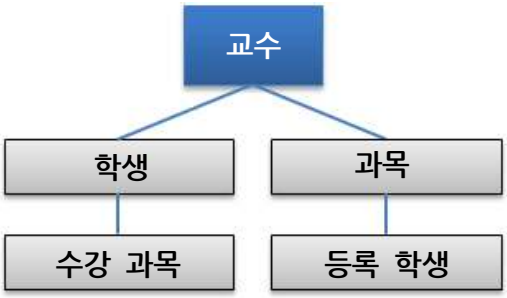
6) Network Data Model

- 스키마 다이어그램이 Network(그래프)
- 허용되는 레코드 타입, 관계성을 명세
- 허용되는 레코드 타입, 관계성을 명세 : 두 레코드 타입간의 1:n 관계(→)



7) Hierarchical Data Model

- 스키마 다이아그램이 Tree
- No Cycle
- 루트 레코드, 자식 레코드, 레벨
- 자식-부모(Parent-child)관계
- 1:n 관계의 두 레코드 타입



10주차 - 1차시	데이터베이스 설계

1. 데이터베이스 설계 및 요구조건 분석

1) 데이터베이스 설계 단계 및 설계 시 고려사항

◆ 데이터베이스 설계 단계

- 단계별 주요 작업 내용

◆ 데이터베이스 설계 전략

- 데이터 중심(Data-driven) DB 설계
  - DB 내용과 구조에 치중
- 처리 중심(Processing-driven) DB 설계
  - 데이터의 처리와 응용(트랜잭션)에 치중
    - ▸ 병행적으로 추진

◆ 데이터베이스 설계 고려사항

- 무결성(Integrity) : 제약조건
- 일관성(Consistency) : 데이터 간, 응답간의 일치성
- 회복(Recovery) : 장애 복구

- 보안(Security) : 불법 접근의 방지
- 효율성(Efficiency) : 응답시간, 저장 공간, 처리도
- 데이터베이스 성장(Database Growth) : 응용과 데이터의 계속적 확대

2) 요구조건 분석

◆ 요구조건 분석(Requirements Analysis)

- 잠정적인 사용자(Potential User)의 식별
  - 수행 업무
  - 데이터의 종류
  - 데이터의 용도 및 처리 형태
  - 데이터의 흐름
  - 처리 과정에서의 요구조건이나 제약조건
  - 트랜잭션의 입력과 출력 데이터

◆ 요구조건 내용

- DB의 정적 정보 구조 요소
  - 개체(Entities)
  - 애트리뷰트(Attributes)
  - 관계성(Relationships)
  - 제약조건(Constraints)
- 동적 DB 처리 요구조건
  - Transaction 유형
  - Transaction 실행 빈도
- 범 기관적 제약조건
  - 경영 목표, 정책, 규정, …
  - 데이터베이스는 경영 목표 달성을 위한 효율적 도구

◆ 요구조건 분석 과정

- 정보 내용과 처리 요구조건의 수집
  - 방법 : 서면 조사, 인터뷰
  - 내용 : 업무, 데이터, 처리형태
- 범기관적 경영 목표와 제약조건을 식별

- 장래 정보 전략(Future Information Strategy)

• 공식적 요구조건 명세(Requirement Specification)의 작성
- 데이터(Data)
- 트랜잭션(Transactions)
- 작업-데이터(Task-data) 간의 관계
- 제약조건(Constraints)

• 요구조건 명세의 검토 및 확인
- 중복, 누락, 보완
- 잠정적 확정

◆ 작업-데이터(Task-data) 간의 관계 예

작업 / 데이터	**학생명부**	**성적표**	**과목표**	**교수명부**
학번	○	○		
주소	○	○		
학과	○			○
과목이름	○	○	○	○
과목번호		○	○	
성적		○	○	
학점		○		
교수이름		○	○	○
전공			○	○
직급				○
전화번호	○			○

◆ 요구조건 명세 작성 기법

• Diagram 방식
- 정보 처리 요구조건을 조직하여 표현
  ▸ HIPO (Hierarchical Input Process Output)
  ▸ SADT (Structured Analysis and Design Technique)
  ▸ DFD (Data Flow Diagram)

• Computer Tool 이용

- PSL/PSA(Problem Statement Language/Problem Statement Analyzer)
- 일관성(Consistency)과 완전성(Completeness)을 검사

2. 개념적 스키마 모델링

1) 개념적 설계(Conceptual Design)

◆ Conceptual Schema Modeling
- Data의 조직과 표현에 초점

→ Data Oriented Design

◆ Transaction Modeling
- Application을 위한 데이터 처리에 초점

→ Processing Oriented Design

2) 개념적 스키마 모델링

◆ 개념적 모델링(Conceptual Modeling)
- 요구조건 분석 결과로 나온 명세를 개념적 데이터 모델(Conceptual Data Model) 즉, DBMS에 독립적이고 고차원적인 표현 기법으로 기술
- 기술 결과를 개념적 구조(Conceptual Structure), 즉 개념적 스키마(Conceptual Schema)라 함
- 스키마 구성 요소 : 개체타입(Entity Type), 속성(Attribute)
- 관계성(Relationship)을 식별해서 결정

→ 설계자의 설계 결정(Design Decision)

- E-R Diagram으로 표현

→ Conceptual Data Model

→ High Level Data Model

3) 개념적 데이터 모델(Conceptual Data Model)

◆ High-level Data Model

◆ 특성

- 표현력(Expressiveness) : 개체 타입, 관계성, 제약조건
- 단순성(Simplicity) : 단순한 이해와 사용
- 최소성(Minimality) : 작은 수의 기본 개념만 사용
- 다이어그램식 표현 : 시각적, 종합적
- 공식성(Formality) : 공식적 명세를 위해 모호하지 않고 정확함
- 독립성(Independency) : DBMS에 독립적
- 융통성(Flexibility) : 변경 수용 능력

4) 개념적 스키마 모델링의 기본 원리

◆ 추상화

- 자세하고 복잡한 것들을 단순하면서 전체적인 추상적 객체, 즉 개념화(Conceptualization)를 이용해 표현함

◆ 추상화 방법

- 집단화(Aggregation)
- 일반화(Generalization)

5) 개념적 스키마 설계 방법

◆ 뷰 통합 방법(View Integration Approach)

◆ 애트리뷰트 합성 방법(Attribute Synthesis Approach)

6) 뷰 통합 방법(View Integration Approach)

◆ 하향식 방법(Top-down Approach)

- Divide and Conquer
- 요구조건 명세로부터 먼저 몇 개의 응용 부문별 뷰(View)를 식별하고 모델링
  - 개체를 식별, 개체에 대한 키 애트리뷰트(Key Attribute)를 결정
  - 개체들 간의 관계성을 식별하고 명세
  - 각 개체에 대해 그 특성을 표현하는 설명 에트리뷰트(Descriptive Attribute)들을 첨가

◆ 부문별 뷰들을 통합해서 하나의 전체적 개념 스키마 구성

- 동일성 통합(Identity Integration)
  - 동일 요소나 동의어들을 통합
- 집단화(Aggregation)
  - 개체 원소들을 그룹핑
- 일반화(Generalization)
  - 개체들의 공통 성질을 기초로 대분류
- 상호 모순 해결
  - 이름, 타입, 도메인, 제약조건, 키

7) 애트리뷰트 합성방법(Attribute Synthesis)

◆ 상향식(Bottom-up)
- 애트리뷰트 리스트에서 출발

◆ 작업-데이터(Task-data) 간의 관계에 기초
- 애트리뷰트들을 식별, 분류
  - 유일성(Uniqueness) 여부에 따라 구분
- 개체 정의
  - 키(Key), 애트리뷰트, 설명(Descriptive) 애트리뷰트
- 일반화(Generalization)
  - 개체들의 공통 성질을 기초로 대분류
- 관계성 정의
  - 개체 간, 개체-애트리뷰트, 애트리뷰트 간 관계
  - 그래프(다이어그램) 표기법으로 표현(ERD)
- 개념적 구조(ERD)를 분석, 확인
  - Cardinality
  - 종속 정보(Dependency Information)
  - 누락 정보(Missing Information)

10주차 - 2차시	DBMS의 선정

1. 트랜잭션 모델링 및 DBMS의 선정

1) 트랜잭션 모델링(Transaction Modeling)

◆ 응용을 위한 Transaction을 식별하고 명세

- 주요 Transaction을 선정하고 기능적 특성을 DB 설계 초기에 명세
- 스키마에 Transaction이 필요로 하는 정보가 모두 포함되고 있는지 확인
- Transaction의 상대적 중요성, 예상 실행 빈도수 파악
  - 물리적 DB 설계의 자료

◆ I/O와 기능적 행태를 명세

- 입력 데이터, 출력 데이터, 내부 제어 흐름

◆ Transaction 유형

검색(Retrieval)	갱신(Update)	혼합(검색, 갱신)

2) 논리적 설계(Logical Design)

◆ 개념적 설계 단계로부터 생성된 Conceptual Schema로부터 목표 DBMS가 처리할 수 있는 논리적 스키마(Logical Schema)를 생성

- Requirements에 부합
- Integrity와 Consistency Constraints도 만족

3) DBMS의 선정 - 비용 중심

◆ 기능 : 기본적(Mandatory)인 기능과 추가적(Optional)인 기능을 평가

◆ Software 구입비용 : Options

◆ 유지비용 : Maintenance and Version Up

◆ Hardware 구입비용

- 추가적인 Main Memory, Disk 주변 장치

◆ DB 생성과 변환 비용

- 구형에서 신형 시스템
- From Scratch

◆ 인건비 : 새로운 직책과 조직

◆ 교육 훈련비용

◆ Operational Cost

◆ 논리적 설계 세부 단계

- 논리적 데이터 모델(Logical Data Model)로 변환
- 트랜잭션 인터페이스(Transaction Interface) 설계
- 스키마의 평가(Schema Evaluation) 및 정제

◆ 논리적 데이터 모델(Logical Data Model)로 변환

- 개념적 스키마(개념적 구조)를 목표 DBMS의 논리적 데이터 모델에 맞는 스키마로 변환
  - 논리적 데이터베이스 구조(논리적 데이터 모델)
  - Relational, Hierarchical, Network, Object-Oriented, Object-Relational

결과	목표 DBMS의 DDL로 기술된 스키마

  - DDL에 포함되는 물리적 설계 매개변수(Physical Design Parameter)가 있는 경우에는 물리적 설계 단계까지 보류

◆ 트랜잭션 인터페이스 설계

- 전체적 트랜잭션 골격(Skeleton) 및 인터페이스(Interface)를 정의
  - 트랜잭션은 입출력과 기능적 행태로 정의
- 데이터 접근 방법 및 인터페이스를 절차적으로 명세

◆ 스키마의 평가 및 정제

- 정량적 정보와 성능 평가 기준에 따라 평가

정량적 정보 (Quantitative Information)	데이터의 양, 처리 빈도수, 처리 작업량

성능 평가 기준 (Performance Evaluation Criteria)	논리적 레코드 접근, 데이터 전송량, DB 크기

- 정제(Refinement)

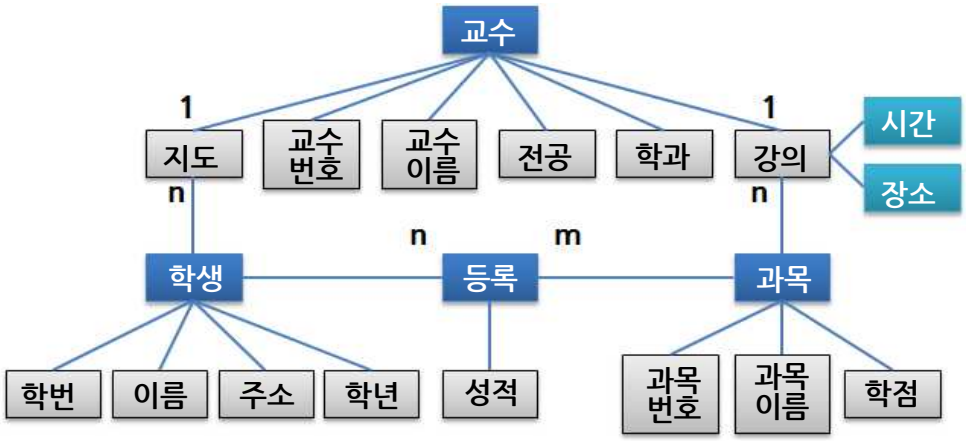
◆ 관계 데이터 모델로의 변환 예

- 개체 타입(Entity Type)
  - 개체 릴레이션(Entity Relation)
- 관계 타입(Relationship Type)
  - 연관된 개체 타입들의 키를 포함시켜 표현하는 관계 릴레이션(Relationship Relation)
  - 두 릴레이션에 공통되는 애트리뷰트(키)를 포함시켜 관계(Relationship)를

표현 : 묵시적 표현

- 설계 선택(Design Alternatives)
  - Data Redundancy
  - Efficient Processing

◆ 학사 관계 데이터베이스 스키마 다이어그램



- 독립된 릴레이션으로 관계(Relationship)를 표현

교수

교수번호	교수이름	전공	학과

학생

학번	이름	주소	학년

과목

과목번호	과목이름	학점

Entity 릴레이션

지도

교수번호	학번

강의

교수번호	과목번호	시간	장소

등록

학번	과목번호	성적

Relationship 릴레이션

◆ 교수와 학생간의 지도 관계의 표현

- 공통 애트리뷰트로 관계(Relationship)를 표현

교수

교수번호	교수이름	전공	학과

학생

학번	이름	주소	학년	교수번호

(a)

교수

교수번호	교수이름	전공	학과	학번

학생

학번	이름	주소	학년

(b)

2. 물리적 설계 및 DB 설계 과정

1) 물리적 설계(Physical Design)

- Logical Schema로부터 효율적인 Internal Schema 설계
  - 물리적 데이터베이스 구조를 생성
- 구현을 위한 Transaction의 내부 구조를 결정
- 물리적 Database의 기본적인 데이터 단위는 저장레코드(Stored Record)
- Contents of Physical Design
  - 저장 레코드 양식 설계(Stored Record Format Design)
  - 레코드 집중화(Record Clustering)
  - 저장 공간 할당(Storage Space Allocation)

2) 저장 레코드의 양식 설계

- 양식(Format) 설계 고려 사항
  - Data Type
  - Distribution of Data Values

- Applications to be Used
- Access Frequency

• 데이터 표현(Data Representation) 및 압축(Compression) 양식도 설계
• 접근 빈도에 따라 상이한 양식으로 저장의 효율성을 증진

3) 레코드 집중의 분석 및 설계

• 레코드 집중(Record Clustering)
- 물리적 순차성(Physical Contiguity) 지원

• Record Size와 Physical Storage Device의 특성에 의존
• 순차 처리(Sequential Processing)
- Large Block

• 임의 접근 처리(Random Processing)
- Small Block

4) 접근 방법 설계 1

• 접근 경로(Access Path)
- 저장 레코드의 접근을 위한 절차
- 저장 구조 : (인덱스를 통한) 접근 방법과 저장 레코드를 정의
- 탐색 기법 : 응용에 적절한 접근 경로 정의

• 기본 접근 경로와 보조 접근 경로의 설계

기본 접근 경로(Primary Access Path)

- 기본 키를 이용한 기본 인덱스 이용
- 초기 레코드 적재, 레코드의 물리적 위치, 기본 키에 의한 검색
- 주요 응용의 효율적 처리

보조 접근 경로(Secondary Access Path)

- 보조 키에 의한 인덱스 이용

5) 접근 방법 설계 2

• 상용 DBMS는 성능 향상을 위해 여러 가지 Tool을 지원
• Physical Design 옵션 선택 시 고려사항

- Response Time
- Efficient Storage Space
- Transaction Throughput

• Simulation이나 Prototype으로 평가

• 추후 분석을 위한 Monitoring Utility 이용
- System Catalog에 성능 통계를 수집, 저장
- Design Tune-up

6) DB 구현

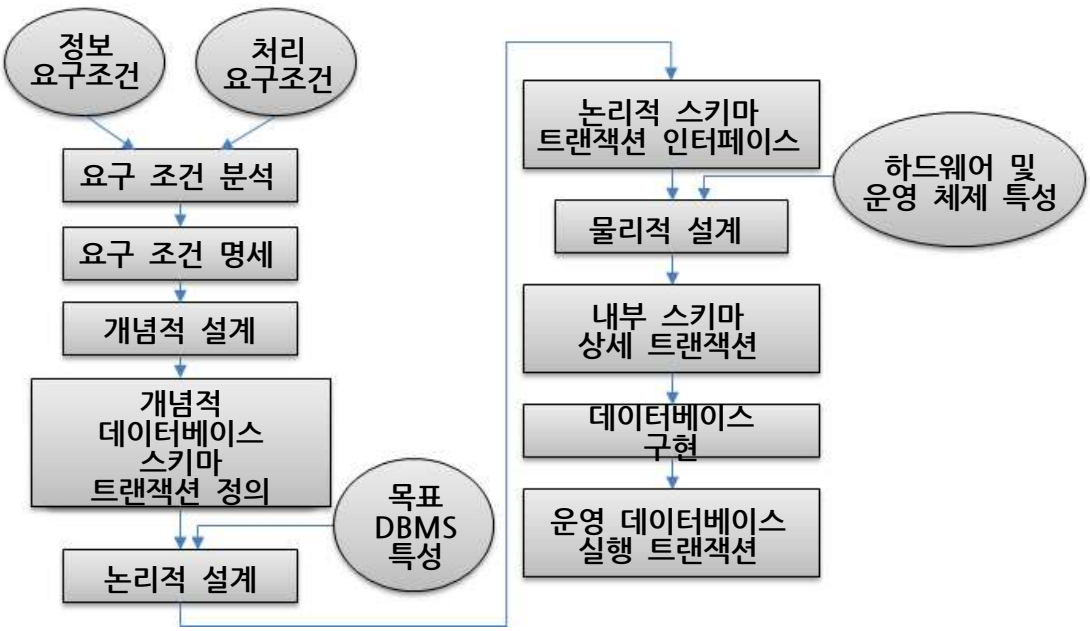
• 목표 DBMS의 DDL로 기술된 Schema의 Compile 및 Execution
- DB Schema Creation
- Empty DB File Creation

• Data Loading
- Transformation Routine이나 Utility를 이용

• Transaction 구현
- Executable Transaction

• Operational Database의 완성

7) DB 설계 과정 요약



11주차 - 1차시	무결성 규정

1. 무결성 규정

1) 무결성 규정

◆ 무결성 규정

- 데이터의 정밀성(Accuracy), 정확성(Correctness)
- 초기 적재 이후에 일어나는 잘못된 갱신으로부터 데이터베이스 보호
  - 갱신 : 정식허가를 받은 사용자에 의한 것
  - 보호의 내용 : 의미적 오류(Semantic Error)

◆ 무결성 서브 시스템

◆ 무결성 서브 시스템(Integrity Subsystem)

- DBMS의 한 구성 요소
- 무결성 규정을 관리 유지
- 트랜잭션이 수행하는 갱신 연산이 무결성 규정에 위반되지 않는가를 감시 · 위반하는 경우
  - 연산을 취소시키거나 거부하여 무결성이 파괴되는 것 예방

2) 무결성 규정 요건

- 무결성 규정(Integrity Rule) 또는 무결성 제약(Integrity Constraint) - 도메인, 키, 종속성, 관계성 등
  - 규정 이름 : 규정 참조 시 사용하는 식별자
  - 검사 시기 : 트랜잭션의 접근 유형 및 데이터와 함께 검사해야 될 시기 명세
  - 제약 조건 : 데이터가 만족해야 할 조건
  - 위반 조치 : 대응 조치

```
R1: AFTER UPDATING STUDENT.Year:
      CHECK( STUDENT.Year > 0 )
      ELSE
        DO;
          PRINT"R1 violated";
          REJECT;
        END;
```

- 특징
  ① 고급 언어를 사용하여 기술
  ② 시스템 카탈로그(사전)에 저장 : 질의할 수 있음
  ③ 사용자가 아니라 시스템에 의해 자동적으로 시행
- 무결성 규정의 대상
  ① 도메인, 키, 종속성, 관계성 등 여러 가지
  ② 이들에 대한 무결성 규정은 그 정의 자체가 묵시적 무결성 규정으로 취급
  ③ 별도의 명시적 규정이 필요 없는 것도 있음

3) 무결성 규정의 종류

◆ 도메인 무결성 규정(Domain Integrity Rules)
- 주어진 애트리뷰트의 값이 그 애트리뷰트가 정의된 도메인에 속한 값이어야 한다는 것을 규정
- 삽입, 갱신 연산에 적용

◆ 릴레이션 무결성 규정(Relation Integrity Rules)
- 릴레이션을 조작함에 있어서 의미적 관계(Semantic Relationship)에 관한 것
- 연산 수행 전/후에 대한 관계 규정(삽입, 갱신, 삭제)
- 분류

상태 제약과 과도 제약	집합 제약과 투플 제약	즉시 제약과 지연 제약

4) 상태 제약과 과도 제약

◆ 상태 제약(State Constraint)

- 데이터베이스가 일관성 있는 상태가 되기 위한 조건 명세
- 데이터베이스 상태(Database State)
  - 어느 특정 시점에 데이터베이스에 저장되어 있는 데이터의 값
- 정적 제약의 성질
  - 각 데이터베이스 상태가 모두 만족해야 되는 규정

◆ 과도 제약(Transition Constraint)

- 데이터베이스의 한 상태에서 다른 상태로 변환되는 과정에 적용되는 규정
- 데이터베이스 상태의 변환 직전과 직후의 비교
  - 예 : 월급은 감소될 수 없다
- 동적 제약
  - 데이터베이스 상태의 변경 전후가 모두 만족해야 함

5) 집합 제약과 투플 제약

집합 제약(Set Constraint)
어떤 투플 집합 전체에 관련된 규정

투플 제약(Tuple Onstraint)
처리되고 있는 투플에만 적용되는 규정

6) 즉시 제약과 지연 제약

즉시 제약(Immediate Constraint)
삽입, 삭제, 갱신 연산이 수행된 즉시 적용되는 규정

지연 제약(Deferred Constraint)
트랜잭션이 완전히 수행된 뒤에 적용되는 규정

2. 무결성 규정의 명세 및 SQL에서의 무결성 규정

1) 무결성 규정의 명세

◆ 트리거(Trigger)

- 데이터베이스 무결성을 유지하기 위한 프로시저
- 디먼(Demon) 개념과 비슷
- SQL을 확장하여 트리거 명세
  - 조건 : 조치를 유발하는 사건
  - 조치 : 조건이 만족되는 경우

```
DEFINE TRIGGER 트리거 이름 ON 트리거 조건 : 명령문
```

◆ 트리거의 예

```
DEFINE TRIGGER STUDINS
  ON INSERTION OF STUDENT:
  (UPDATE DEPARTMENT
   SET Tstdn = Tstdn + 1
   WHERE Dno = NEW_STUDENT.Dno );

DEFINE TRIGGER STUDDEL
  ON DELETION OF STUDENT:
  (UPDATE DEPARTMENT
   SET Tstdn = Tstdn - 1
   WHERE Dno = OLD_STUDENT.Dno );
```

- 조건 만족되면 UPDATE 명령 자동 실행

2) SQL에서의 무결성 규정

◆ 도메인 제약 조건

```
[CONSTRAINT constraint] CHECK (cond-exp)
```

- Constraint : 제약 조건의 이름
- Cond-exp : 제약 조건을 표현하는 조건식

- 예 : 도메인 DYEAR는 정수 {1, 2, 3, 4}로 정의

```
CREATE DOMAIN DYEAR SMALLINT
CONSTRAINT CONST-YEAR
CHECK (VALUE IN (1, 2, 3, 4));
```

◆ 기본 테이블 제약 조건

```
[CONSTRAINT constraint]
    { PRIMARY KEY | UNIQUE } (column-list)
    | [CONSTRAINT constraint]
      FOREIGN KEY (column-list)
                REFERENCE base-table[(column-list)]
            [ON DELETE {NO ACTION | CASCADE}]
            [ON UPDATE {NO ACTION | CASCADE}]
    | [CONSTRAINT constraint] CHECK (cond-exp)
```

```
CREATE TABLE ENROL
   ( Sno   DSNO NOT NULL,
     Cno   DCNO NOT NULL,
     Grade INTEGER NOT NULL,
     PRIMARY KEY (Sno,Cno),
     FOREIGN KEY (Sno) REFERENCES STUDENT
           ON DELETE CASCADE
           ON UPDATE CASCADE,
     FOREIGN KEY (Cno) REFERENCES COURSE
           ON DELETE CASCADE
           ON UPDATE CASCADE,
     CHECK (Grade ≥ 0 AND Grade ≤ 100) );
```

◆ 일반 제약 조건

```
제약 조건 명세 : CREATE ASSERTION constraint CHECK (cond-exp)
제약 조건 제거 : DROP ASSERTION constraint;
```

- 예

```
CREATE ASSERTION CONST_YEAR
        CHECK ( (SELECT MIN(STUDENT.Year)
                    FROM STUDENT) >0 );

CREATE ASSERTION CONST_CREDIT
        CHECK ( NOT EXISTS (SELECT * FROM COURSE
                    WHERE NOT (COURSE.Credit <5) ) );
```

11주차 - 2차시	HTML/XML

1. 구조적/비구조적 문서 및 HTML 문서의 특징

1) 구조적/비구조적 문서

◆ 데이터베이스와 인터넷

- 전자 상거래와 인터넷 데이터베이스 응용들은 Web Page를 디스플레이 하는 웹 인터페이스를 통해서 사용자와 정보 교환
- Web Page의 내용과 포맷팅을 위해서는 Hyperlink 문서를 사용
- 문서 작성 언어
  - HTML(HyperText Markup Language)

디스플레이를 위한 웹 문서(텍스트 데이터)의 작성과 포멧팅에 사용
DB로부터 추출되는 구조화 데이터를 명세하는 것은 부적절

  - XML(Extensible Markup Language)

웹 페이지에 있는 데이터의 구조와 의미에 관한 정보를 제공

◆ 구조화 데이터와 반구조화 데이터

- 구조화 데이터(Structured Data)
  - 데이터베이스에 저장되는 데이터

엄격한 포맷으로 표현	모두 동일한 구조의 투플

- Company 데이터베이스

Employee	EmpName	EmpId	Address	Salaty	DeptId
Department	DeptName	DeptId	MgrId		
project	ProjName	ProjId	DeptId		
WorksOn	EmpId	ProjId	Hours		

2) HTML 문서의 특징

◆ 반구조화 데이터 개념과 특성 (1)

- 반구조화 데이터(Semistructured Data)
  - 데이터가 저장되고 관리될 방법이 결정되기 전에 과도적인 방법으로 수집 저장되는 데이터

수집된 데이터는 모두 다양한 구조를 가짐
- 애트리뷰트(Attribute)가 여러 개체(Entity)들 간에 공용 - 어떤 애트리뷰트는 일부 개체들만 가짐 - 새로운 데이터들이 수집되면 또 다른 애트리뷰트를 가질 수 있음 → 스키마를 미리 정해놓을 수 없음

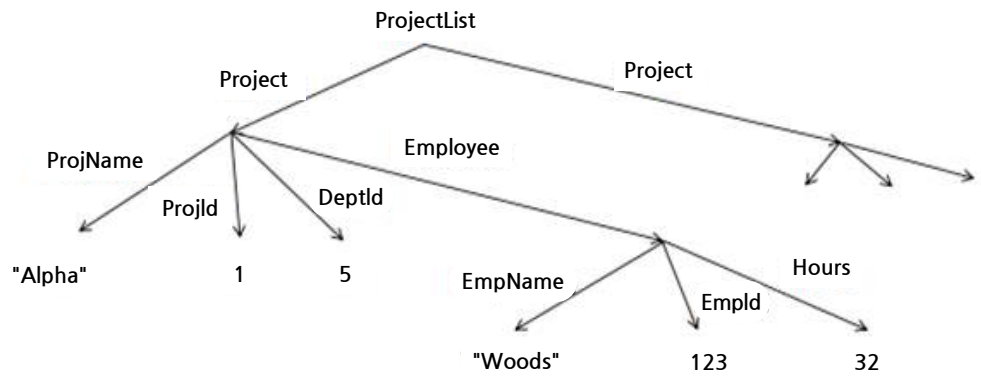
  - 애트리뷰트와 데이터 값을 함께 표현(<Attribute, Value>)

◆ 반구조화 데이터 개념과 특성 (2)

- 구조화 데이터와 반구조화 데이터의 차이점
  - 애트리뷰트(Attribute), 관계(Relationship), 개체타입(Entity Type) 이름과 같은 스키마 구성요소(Schema Constructs)의 취급 방법
  - 반구조화 데이터는 데이터 객체마다 서로 다른 애트리뷰트들을 가질 수 있음
  - 반구조화 데이터는 스키마 정보가 데이터 값들과 혼합되어 있기 때문에 자기 기술 데이터(Self-describing Data)라 함

◆ 반구조화 데이터 표현

- 반구조화 데이터를 위한 데이터 모델
  - 트리(Tree)나 그래프(Graph) 자료 구조에 기초
- 방향 그래프로 표현한 반구조화 데이터 예



레이블(Label)	**애트리뷰트, 개체 타입, 관계**
내부 노드(Internal Node)	**객체, 복합 애트리뷰트**
리프노드(Leaf Node)	**단순 애트리뷰트의 데이터 값**

◆ 비구조화 데이터

- 비구조화 데이터(Unstructured Data)
  - 텍스트 문서(Text Document)

모든 정보가 문서 속에 포함

  - HTML(HyperText Markup Language)로 만들어진 웹 페이지

데이터가 디스플레이 정보와 혼합

- HTML 문서의 구성
  - 시작 태그(Start Tag) : <텍스트>
  - 끝 태그 (End Tag) : </텍스트>
  - 시작 태그와 끝 태그 사이에 있는 텍스트를 디스플레이하는 방법을 표기(Markup)해서 HTML 처리기에 지시
  - HTML 태그는 문서 디스플레이를 위한 포맷팅 정보(폰트, 스타일, 색깔 헤딩 레벨 등)를 명세
- HTML의 특성
  - 많은 수의 시스템 정의 태그(System-defined Tag)가 있음

몇 권의 책 분량

- 각 태그는 각자 정해진 의미(Semantics)를 가짐
- 웹 브라우저(Web Browser)는 각 태그의 의미대로 HTML 문서를 포맷팅해서 디스플레이
- 포맷해야 할 텍스트는 반드시 시작 태그(Start Tag)와 끝 태그(End Tag) 쌍으로 명세

◆ HTML Tag

• 〈html〉 … 〈/html〉
  - HTML 문서의 시작(〈html〉) 과 끝을 명세 (〈/html〉)
• 〈head〉 … 〈/head〉
  - 문서 헤더(Document Header) 정보를 명세
  - 문서에서 사용될 여러 가지 명령을 명세
  - 문서의 Title이나 디스플레이 할 필요가 없는 정보도 포함
• 〈body〉 … 〈/body〉
  - 문서의 몸체(Body)
  - 텍스트와 텍스트의 포맷팅, 디스플레이 정보를 명세하는 표기 태그 (Markup Tag)를 포함
  - 다른 객체(이미지, 비디오, 음성 등)나 문서에 대한 참조(Reference)도 포함

◆ HTML의 특성

• 많은 수의 시스템 정의 태그(System-defined Tag)가 있음
  - 몇 권의 책 분량
• 각 태그는 각자 정해진 의미(Semantics)를 가짐
• 웹 브라우저(Web Browser)는 각 태그의 의미대로 HTML 문서를 포맷팅해서 디스플레이
• 포맷해야 할 텍스트는 반드시 시작 태그(Start Tag)와 끝 태그(End Tag) 쌍으로 명세

◆ HTML의 한계

• HTML 문서의 포맷을 잘 설계하면
  - 문서 내용의 이해가 용이

- 웹 문서의 탐색 및 항해(Navigation)가 용이

• HTML의 한계

- HTML 텍스트 원문은 스키마 정보를 포함하고 있지 않기 때문에 컴퓨터 프로그램에 의한 자동 번역이 곤란

→ XML의 탄생 배경

2. XML 문서의 구성 및 장단점

1) XML 데이터 모델

◆ XML(Extensible Markup Language)

• 문서를 작성하는 규칙을 기술하는 메타언어(Meta-language)

• 인터넷에 의한 문서 교환을 위해 표준으로 설계

◆ HTML/SGML과의 차이점

• HTML : 텍스트 데이터의 디스플레이 방식을 기술

• SGML(Standard Generalized Markup Language)

- 복잡한 기술 문서의 출판이나 배포를 위한 국제 표준

• XML : 문서에 있는 데이터 원소의 의미를 기술

→ 컴퓨터 프로그램이 XML 문서에 있는 데이터 원소를 자동으로 처리 가능

2) XML 문서

◆ XML 문서의 구성요소

• 원소(Element)

• 애트리뷰트(Attribute)

- 원소를 설명하는 보조적인 정보 제공

• 개체(Entity)

• 식별자(Identifier)

• 참조(Reference)

3) XML 문서의 구성

◆ XML 문서의 구성

- XML 선언
  - XML processor에 이 문서가 작성된 xml version 과 기타 정보를 선언
  - 필수적으로 문서의 첫 줄에 <?...?> 형식으로 문서가 작성된 xml version을 선언

```
<?xml version="1.0" standalone="yes"?>
```

→ version은 1.0이며 이 문서가 독립적임
(DTD나 scheme 없이 명세됨)

- <!－－ －－> : comment
  - 태그 속을 제외하고 어디에도 삽입 가능
  - 문서에서만 유효하고 응용에 전달되지는 않음
- XML은 원소를 기술하는데 필요한 태그를 사용자가 정의하여 사용
- XML 문서 작성의 특징
  - 시스템 정의 태그 이름이 없고 사용자 정의 태그 이름만 있음
  - 작성자가 태그 이름을 정의해서 사용해야 함
  - 작성자가 자유롭게 새로운 원소 명세가 가능
  - 프로그램에 의한 문서의 자동 번역에 한계
- 원소(Element)
  - 시작 태그(<…>)와 끝 태그(</…>)로 둘러싼 텍스트

<원소이름>	**원소내용**	**</원소이름>**
시작태그	**스트링 데이터 값**	**끝 태그**

- 원소 내용(Content)
  - 따옴표(“ ”)가 없는 String 데이터
    단순 원소(Simple Element)

```
<ProjName> Alpha </ProjName>
```

- 복합 원소(Complex Element)
  - 원소 속에 또 다른 하위 원소(Subelement)를 포함
  - 계층 구조(Tree 구조)를 형성 : 부모/자식 관계, 조상/자손 관계
  - 원소의 명세 순서는 문서에 출력 순서를 표현

- 애트리뷰트(Attribute)
  - 원소의 특성을 명세

<원소 이름 attribute-name = value>

원소의 시작 태그

  - Value는 반드시 따옴표(" ")로 표시된 스트링

```
<Project City="Seoul" Period="Long">
```

  - 출력이나 디스플레이 되지 않는 묵시적 텍스트
  - 원소에 XML 태그와 같은 특수 문자를 표시해야 될 경우 <![CDATA[…]]>를 사용 : CDATA는 따옴표 기능을 가짐

```
<![CDATA[ <b><i>Wrong element</b></i> ]]>
```

12주차 - 1차시	데이터베이스 보안

1. 데이터베이스 보안 개요

◆ 다중 사용자 환경(Multi-User Environment)

- 불법적인 접근 및 유출 방지를 위해 보안 대책이 필요함
- 오라클에서는 사용자는 자신이 생성한 객체에 대해 소유권을 가지고 데이터에 대한 조작이나 조회가 가능함
- 다른 사용자가 소유한 객체는 소유자로부터 접근 권한을 부여 받지 않는 한 접근이 불가함
- 다중 사용자 환경에서는 데이터베이스 관리자의 암호를 철저하게 관리함

◆ 중앙 집중적인 데이터 관리

- 분산적으로 관리되는 기존의 파일 시스템보다 보안이 취약할 수 있으므로 철저한 보안 대책이 필요함

1) 보안에 대한 개념 알기

◆ 데이터베이스 객체에 대한 보안

- <u>데이터베이스 내의 특정 객체들</u>의 접근 및 이용에 관한 권한을 설정하는 것을 말함
  ↳ 테이블, 뷰 등
- 대개 사용자 단위, 그룹 단위, 역할(Roll) 단위로 정의함

◆ 시스템 객체에 대한 보안

- 시스템 레벨에서 데이터베이스에 대한 접근과 이용에 요구되는 정보들을 부여하는 것을 말함
- <u>데이터베이스 인증 정보</u>부여 및 사용자에게 적절한 테이블 스페이스 할당들의 작업을 말함
  ↳ 사용자 아이디와 패스워드

2. 데이터베이스 보안 관리자 역할

1) 사용자의(User)의 종류 및 역할(Role)에 대하여

◆ 사용자의 종류

- 데이터베이스 관리자
  - DBA(Data Base Administrator)를 말함
  - 데이터베이스 시스템을 전체적으로 관리하는 권한을 가짐
  - 일반 사용자를 생성 및 삭제하는 권한을 가짐
  - 일반 사용자에게 권한을 부여하거나 부여된 권한을 회수하는 권한을 가짐
- 일반 사용자
  - 데이터베이스 관리자에 의해 생성됨
  - 생성될 때 부여 받은 권한 범위 안에서만 데이터베이스 객체에 접근하고 연산을 할 수 있음
  - 일반적으로는 DB 객체인 테이블, 뷰에 대해 SELECT, UPDATE, INSERT, ALTER, DELETE 등의 연산을 주로 수행함

◆ 역할(Role)

- 데이터베이스에 관련된 권한들의 집합을 말하며 일종의 그룹의 개념임
- 역할에는 다수의 사용자나 다른 역할들이 구성원이 될 수 있음
- DBA가 개별 사용자 별로 권한을 부여하고 회수하는 데 따른 권한관리의 불편을 해소하기 위해 사용함
- 역할 단위로 권한을 부여하고 회수하며 역할을 사용자에게 부여함

  → 역할에 권한을 부여 또는 회수하는 경우 역할에 소속된 모든 사용자에게 권한을 부여 또는 회수하는 것과 같음

2) 사용자 생성 및 삭제하기

※ 새 사용자의 생성 및 기존 사용자의 삭제는 DBA 또는 DBA와 동등한 권한을 부여받은 사용자만이 할 수 있음

◆ 새 사용자 생성

- CREATE USER 명령문을 사용함

- DB에 새 사용자를 등록하려 할 때는 사용자명과 패스워드를 지정해야 함
  ↳ 생략이 가능함
- 테이블 스페이스들은 생략하면 기본값으로 설정됨
- 지정한 사용자명과 패스워드 정보는 특정 사용자가 DB 접근 인증을 받으려 할 때 사용함
  → 사용자명이나 패스워드 정보는 대소문자를 구별하여 사용된다는 점에 특히 유의하여야 함
- 구문형식
  - CREATE USER 사용자명 IDENTIFIED [BY 패스워드]
    [DEFAULT TABLESPACE 테이블스페이스]
    [TEMPORARY TABLESPACE 테이블스페이스]
    [PROFILE 프로파일명][QUOTA [정수 | UNLIMITED]
    ON 테이블스페이스];
- CREATE USER 명령은 표준 질의 명령이며 다른 상용 DBMS에서는 지원되지만 SQL Server의 쿼리 분석기에서는 사용이 불가능함
- SQL Server에서는 엔터프라이즈 관리자를 사용하여 새 사용자를 등록하여야 함

◆ 기존 사용자의 패스워드 변경

- ALTER USER 명령문을 사용함
- DBA나 DBA와 동등한 권한을 부여 받은 사용자만이 다른 사용자의 패스워드를 변경할 수 있음
- ALTER USER 명령은 SQL Server에서는 실행되지 않음
- SQL Server에서 ALTER USER 명령 대신 SP_PASSWORD 명령을 사용하여야 함
- 쿼리 분석기에서 암호 변경 명령을 사용해야 함
- 구문형식 1
  - ALTER USER 사용자명 IDENTIFIED BY 새 패스워드(표준 명령)
- 구문형식 2
  - SP_PASSWORD 기존패스워드, 새 패스워드, 사용자명 (SQL Server명령)

◆ 사용자 삭제

- DROP USER 명령문을 사용함
- DB에서 기존의 생성된 사용자가 더 이상 필요 없을 경우 삭제를 하고자 할 때 사용함
- 쿼리 분석기에서는 실행이 되지 않으며 엔터프라이즈 관리자에서만 사용자를 삭제할 수 있음
- 구문형식
  - DROP USER 사용자명(표준 명령)

3) 역할 생성 및 삭제하기

◆ 역할 생성 및 삭제

- 그룹으로 역할(Role)을 생성하고 역할을 이용하여 권한을 부여함
  → 사용자에게 개별적으로 권한을 부여하는 것보다 더 편리함
- 역할의 생성 및 삭제는 DBA 또는 동등한 권한을 부여 받은 사용자만이 역할을 생성하고 삭제할 수 있음
- SQL Server에서는 모든 사용자가 생성될 때 기본적으로 Public 역할의 구성원으로 포함됨
- 역할의 생성 및 삭제 명령은 쿼리 분석기에서는 실행할 수 없기 때문에 엔터프라이즈 관리자에서만 생성과 삭제가 가능함
- 새로운 역할(Role)을 생성하고자 할 때
  - CREATE ROLE 명령문을 사용함
  - CREATE ROLE 역할 이름(표준 명령)
- 기존의 역할을 삭제하고자 할 때
  - DROP ROLE 명령문을 사용함
  - DROP ROLE 역할 이름(표준 명령)

4) 권한(GRANT) 부여하기

◆ 역할 생성 및 삭제

- GRANT 명령문을 사용함
- 데이터베이스를 안전하게 보호하기 위해서 필요함
- DB 내에 존재하는 객체들에 관한 접근 및 작업 권한을 제한하는 것을 말함

└, 테이블, 뷰, Stored Procedure

- 각 사용자의 업무 수행에 있어서 적당한 범위까지만 권한을 부여함
- 권한의 허가는 개별 사용자 별로 부여할 수도 있지만 일반적으로 역할 단위로 부여함
- DBA 또는 동등한 권한을 부여 받은 사용자 또는 객체의 소유주에 의해 권한 부여가 가능함
- SQL Server의 엔터프라이즈 관리자 또는 쿼리 분석기를 사용하여 권한을 부여함
- 구문형식 1

  - GRANT {[객체권한 목록들] | ALL} ON 객체명
    TO {사용자 [,사용자] | 역할명 | PUBLIC}
    [WITH ADMIN(GRANT) OPTION]

5) 암호화

◆ 암호화

- 전치 기법
- 대체 기법
- DES
  - 발신자과 수신자만이 데이터 암호화에 사용된 키를 알고 있음

- RSA

- 전자 서명에 많이 쓰이는 알고리즘

12주차 - 2차시	데이터베이스 권한

1. 데이터베이스 권한 개요

1) 권한의 역할과 종류

◆ 권한

- 사용자가 특정 테이블을 접근할 수 있도록 하거나 해당 테이블에 SQL(SELECT/INSERT/UPDATE/DELETE)문을 사용할 수 있도록 제한을 두는 것
- 데이터베이스 보안을 위한 권한
  - 시스템 권한(System Privileges)
  - 객체 권한(Object Privileges)

◆ 시스템 권한

- 주로 DBA에 의해 부여됨
- 사용자의 생성과 제거, DB 접근 및 각종 객체를 생성할 수 있는 권한 등 권한의 수가 80가지가 넘음

  → 본 강의에서는 대표적인 시스템 권한만 다룸

◆ 대표적인 시스템 권한

시스템 권한	기능
CREATE USER	새롭게 사용자를 생성하는 권한
DROP USER	사용자를 삭제하는 권한
DROP ANY TABLE	임의의 테이블을 삭제할 수 있는 권한
QUERY REWRITE	함수 기반 인덱스를 생성하는 권한
BACKUP ANY TABLE	임의의 테이블을 백업할 수 있는 권한

◆ 데이터베이스를 관리하는 권한

- 시스템 관리자가 사용자에게 부여하는 권한

시스템 권한	기능
CREATE SESSION	**데이터베이스에 접속할 수 있는 권한**
CREATE TABLE	**사용자 스키마에서 테이블을 생성할 수 있는 권한**
CREATE VIEW	**사용자 스키마에서 뷰를 생성할 수 있는 권한**
CREATE SEQUENCE	**사용자 스키마에서 시퀀스를 생성할 수 있는 권한**
CREATE PROCEDURE	**사용자 스키마에서 함수를 생성할 수 있는 권한**

◆ 대표적인 시스템 권한

- 객체 권한
  - 객체를 조작할 수 있는 권한

  → 현재까지 학습한 것 중에서 테이블, 뷰, 시퀀스, 인덱스 등을 예로 들 수 있음

  → 본 강의에서 학습하게 될 동의어도 모두 객체에 해당됨

2. 데이터베이스 권한 관리

1) 사용자 생성

◆ 사용자 생성

- 회사에 새로운 사원이 입사하는 경우 시스템에 접속하도록 관리자가 계정을 발급해 주어야 함
- 권한은 사용자에게 부여함

  → 먼저 사용자를 생성해야 함

- 사용자 생성을 위한 CREATE USER 명령어의 형식

```
CREATE USER user_name
IDENTIFIED BY password;
```

2) 권한 부여하는 GRANT 명령어

◆ 시스템 권한 부여

- 사용자에게 시스템 권한을 부여하는 GRANT 명령어 형식

```
GRANT privilege_name, …
TO user_name;
```

- 모든 사용자에게 해당 시스템 권한을 부여하려면 user_name 대신 PUBLIC을 기술함
  ↳ DB 내에 있는 모든 계정을 의미함

① 데이터베이스 관리자로 접속함

② 새로 생성된 User01에 데이터베이스에 접속할 수 있는 권한인 CREATE SESSION를 부여함

③ 다시 User01 사용자로 접속을 시도하면 데이터베이스에 성공적으로 접속 가능함

◆ WITH ADMIN OPTION 옵션

- 시스템 권한을 WITH ADMIN OPTION과 함께 부여 시
  - 데이터베이스 관리자가 아닌데도 불구하고 시스템 권한을 다른 사용자에게 부여할 수 있는 권한도 함께 부여 받게 됨

① 데이터베이스 관리자로 로그인해서 사용자 USER02와 USER03를 생성함

② 데이터베이스 관리자에서 User02와 User03에 데이터베이스에 접속할 수 있는 권한인 CREATE SESSION 권한을 부여함

→ USER02는 WITH ADMIN OPTION을 지정함

→ USER03은 WITH ADMIN OPTION을 지정하지 않은 채 권한 부여를 함

③ USER02 사용자로 로그인하면 USER01 사용자에게 CREATE SESSION

권한을 부여할 수 있음

→ USER02에게 WITH GRANT OPTION과 함께 CREATE SESSION 권한을 부여하여 다른 사용자에게도 권한을 부여할 수 있도록 허용하였기 때문임

→ USER03는 CREATE SESSION 권한만을 부여 받았으므로 다른 사용자에게 권한을 부여할 수 없음

3) 객체 권한

◆ 객체 권한

- 특정 객체에 조작을 할 수 있는 권한으로 객체의 소유자는 객체에 대한 모든 권한
- 객체와 권한 설정을 위한 명령어(1/2)

권 한	TABLE	VIEW	SEQUENCE	PROCEDURE
ALTER	v		v	
DELETE	v	v		
EXECUTE				v
INDEX	v			

- 객체와 권한 설정을 위한 명령어(2/2)

권 한	TABLE	VIEW	SEQUENCE	PROCEDURE
INSERT	v	v		
REFERENCES	v			
SELECT	v	v	v	
UPDATE	v	v		

- 테이블, 뷰, 시퀀스, 함수 등과 같은 객체 별로 DML문(SELECT, INSERT, DELETE)을 사용할 수 있는 권한을 설정함
- 객체에 권한을 부여하기 위한 형식

```
GRANT privilege_name [(column_name)] | ALL ①
ON object_name | role_name | PUBLIC ②
TO user_name; ③
```

→ GRANT 명령어의 형식은 어떤 객체(②)에 어떠한 권한(①)을 어느 사용자(③)에게 부여하는가를 설정함

→ ON 옵션이 추가된다는 것이 시스템 권한과의 차이점임

→ ON 다음에 테이블 객체나 뷰 객체 등을 기술함

- 특정 객체에 대한 권한은 기본적으로 해당 객체를 만든 사용자에게만 주어짐
- 본 강의의 학습 과정에서 사용했던 EMP 테이블은 SCOTT 사용자 소유의 테이블임

→ SCOTT 사용자로 로그인해서 USER01 사용자가 테이블 객체 EMP를 조회할 수 있도록 권한 부여를 해야 함

◆ 사용자에게 부여된 권한 조회

- 현재 사용자와 관련된 권한을 조회해 봄
- 사용자 권한과 관련된 데이터 딕셔너리 중 USER_TAB_PRIVS_MADE 데이터 딕셔너리는 현재 사용자가 다른 사용자에게 부여한 권한 정보를 알려줌
- 자신에게 부여된 사용자 권한을 알고 싶을 때는 USER_TAB_PRIVS_RECD 데이터 딕셔너리를 조회함
- USER01과 SCOTT 사용자가 부여한 권한과 부여된 권한을 살펴봄

◆ 사용자에게서 권한을 뺏기 위한 REVOKE 명령어

- 사용자에게 부여한 객체 권한을 데이터베이스 관리자 혹은 객체 소유자로부터 철회하기 위해서는 REVOKE 명령어를 사용함
- REVOKE 명령어의 형식

```
REVOKE {privilege_name | all}
ON object_name
FROM {user_name | role_name | public};
```

→ REVOKE 명령어 다음에는 철회하고자 하는 객체 권한을 기술함

→ ON 다음에는 어떤 테이블에 부여된 권한인지 해당 테이블 명을 기술함

→ FROM 다음에는 어떤 사용자에게 부여한 권한인지 사용자명을 기술함

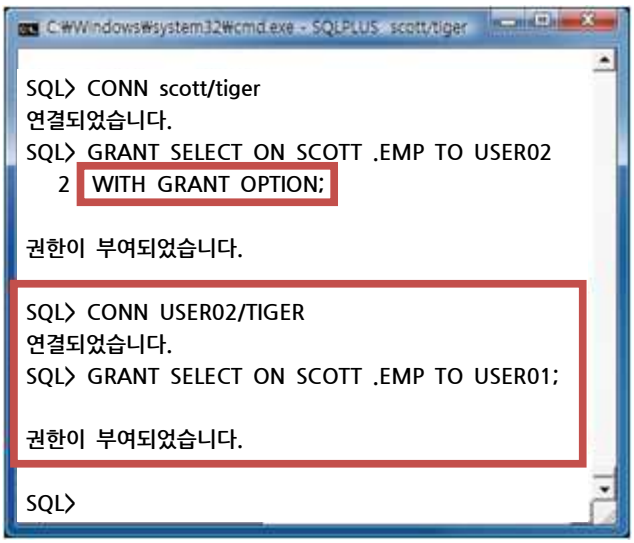
◆ WITH GRANT OPTION

- 객체 권한을 WITH GRANT OPTION과 함께 부여하면 객체에 대한 접근 권한과 함께 권한을 다른 사용자에게 부여할 수 있는 권한도 부여 받음

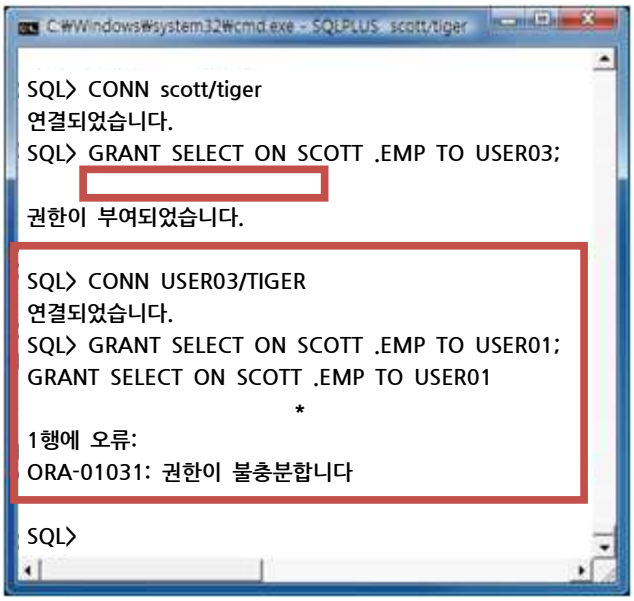
① SCOTT 사용자로 로그인해서 사용자 USER02와 USER03에게 EMP 테이블 객체를 SELECT할 수 있는 권한을 부여함

→ USER02는 WITH GRANT OPTION을 지정하고 USER03은 WITH GRANT OPTION을 지정하지 않음

② 차이점을 확인해 봄



→ USER02는 WITH GRANT OPTION을 지정하였으므로 USER02로 로그인해서 또 다른 사용자에게 객체 권한을 부여할 수 있음



→ USER03는 WITH GRANT OPTION을 지정하지 않았기에 USER03으로 로그인해도 객체 권한을 또 다른 사용자에게 부여할 수 없음

13주차 - 1차시	빅데이터의 개요

1. 빅데이터의 정의

1) 빅데이터의 개념

◆ 빅데이터란?

- 어마어마한(Big) 데이터(Data)?
- 데이터의 규모에 초점을 맞춘 정의
  - 기존 데이터베이스 관리도구의 데이터 수집, 저장, 관리, 분석 역량을 넘어서는 데이터

(출처) 맥킨지, 2011년 5월

- 업무 수행 방식에 초점을 맞춘 정의
  - 다양한 종류의 대규모 데이터로부터 저렴한 비용으로 가치를 추출하고, 데이터의 빠른 수집, 발굴, 분석을 지원하도록 고안된 차세대 기술 및 아키텍처

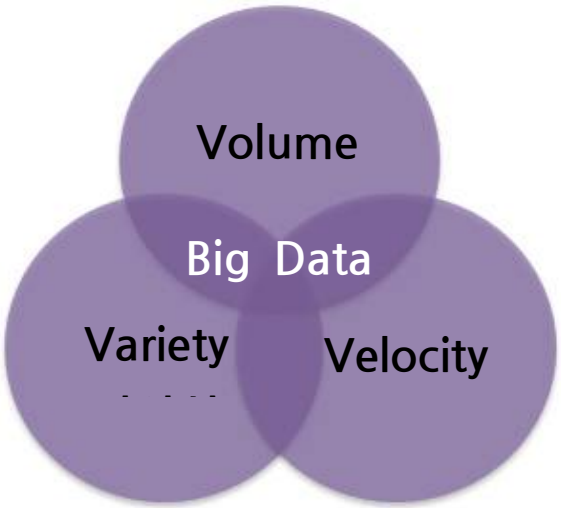
(출처) IDC, 2011년 6월

2) 빅데이터의 특징

2. 빅데이터의 3대 요소

1) 빅데이터의 3대 요소

◆ 3대 요소 가운데 두 가지 이상의 요소만 충족된다면 빅데이터라고 볼 수 있음



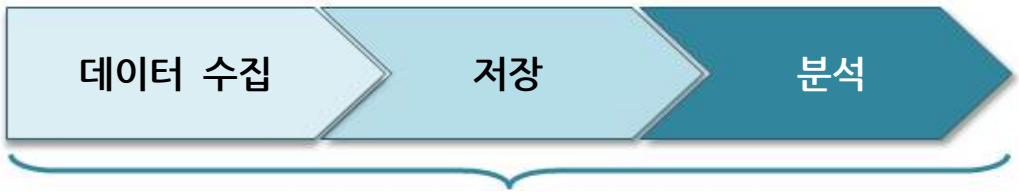
(출처) BI/DW 리서치 기관 TDWI

◆ Volume(규모)

- 일반적으로 수십 테라바이트(Terabyte) 혹은 수십 페타바이트(Petabyte) 이상이 빅데이터의 범위에 해당함
  - 1 테라바이트 = 1024 기가바이트(Gigabyte)
    ↳ 6기가바이트 DVD 영화를 17만 4천 편 담을 수 있는 용량
  - 제타바이트 시대(MB, GB 단위 규모)

◆ Velocity(속도)

- 빅데이터의 속도적인 특징
  - 실시간 처리
  - 장기적인 접근
- 오늘날 디지털 데이터는 매우 빠른 속도로 생성됨



실시간으로 처리돼야 함

- 모든 데이터는 실시간 처리뿐 아니라 수집된 대량의 데이터를 다양한 분석 기법과 표현 기술로 분석해야 함

  → 장기적이고 전략적인 차원에서 접근할 필요가 있음
- 통계학과 전산학에서 사용하는 데이터 마이닝, 기계 학습, 자연어 처리, 패턴 인식 등이 분석 기법에 해당함
- 브리티시텔레콤 : 1초 60기가의 데이터 전송

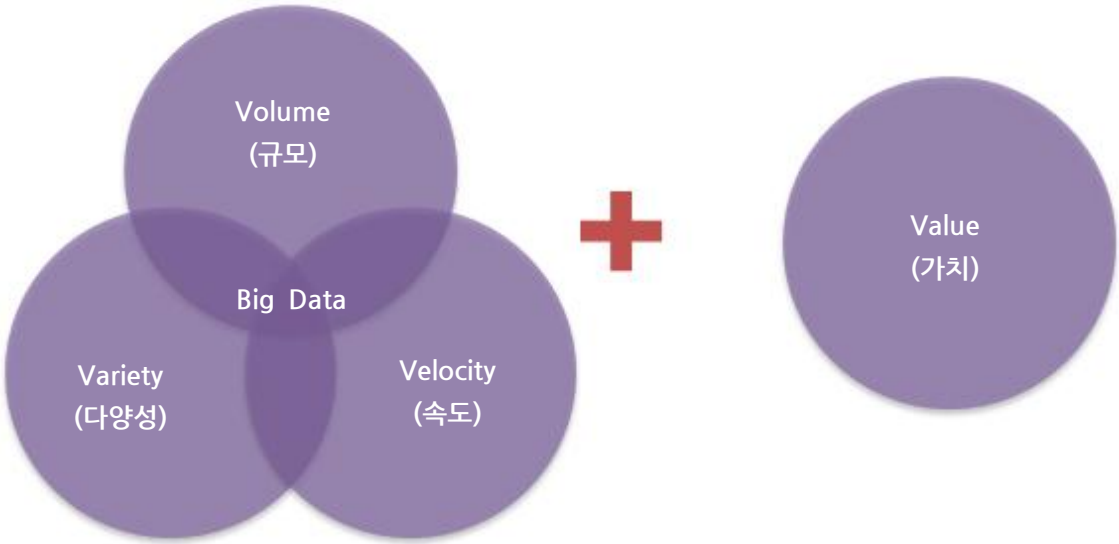
◆ Variety(다양성)

① 정형 데이터

- 온라인 쇼핑몰에서 제품을 주문할 때의 이름, 주소, 연락처, 배송주소, 결제정보 등

② 비정형 데이터

- 동영상 데이터, 사진과 오디오 데이터, 메신저로 주고받은 대화 내용, 위치 정보, 통화 내용 등



3. 빅데이터의 출현 배경

1) 빅데이터의 출현 배경

◆ 기업

- 빅데이터를 적극적으로 활용하는 기업은 비즈니스 성과를 개선할 수 있음
  - 글로벌 패션기업인 자라(ZARA)는 빅데이터 분석을 통해 상품 공급 프로세스를 혁신하고 재고 수량을 최적화함
    → 매출과 영업이익을 크게 증대시켰음

◆ 의료

- 빅데이터는 의료 분야에서도 적용이 확대되고 있음
  - 최근 일본 IBM과 도호대학은 의료 정보를 분석해 미래 질병을 예측할 수 있는 시스템을 개발했다고 발표함
    → 의료 기록에 텍스트 분석 기술을 적용함

◆ 교육

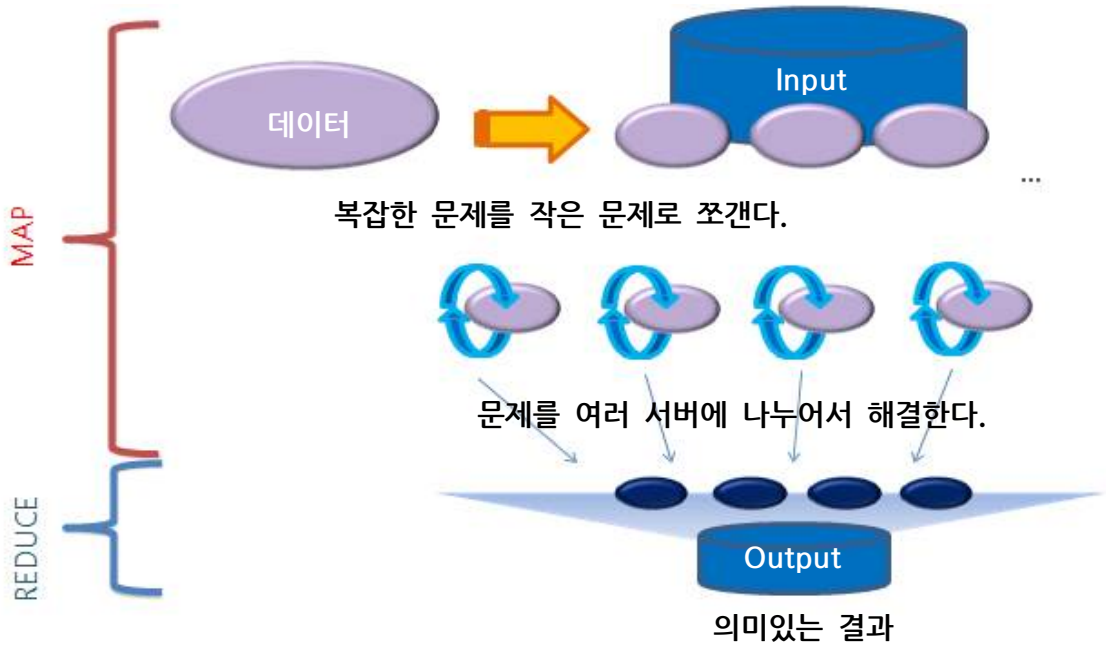
- DNA 연구 분야에서도 빅데이터 기술이 활발하게 적용되고 있음
  - 인간의 DNA를 구성하는 염기 서열 1쌍당 4테라바이트의 이미지 데이터가 생성됨

◆ 정부

- 국가 차원에서도 빅데이터에 대응하기 위한 다양한 정책이 발표되고 있음
  - 미국 백악관과 일본 총무성의 국가전략회의

2) 빅데이터의 처리방식

◆ 대표적인 빅데이터 처리 프로그램

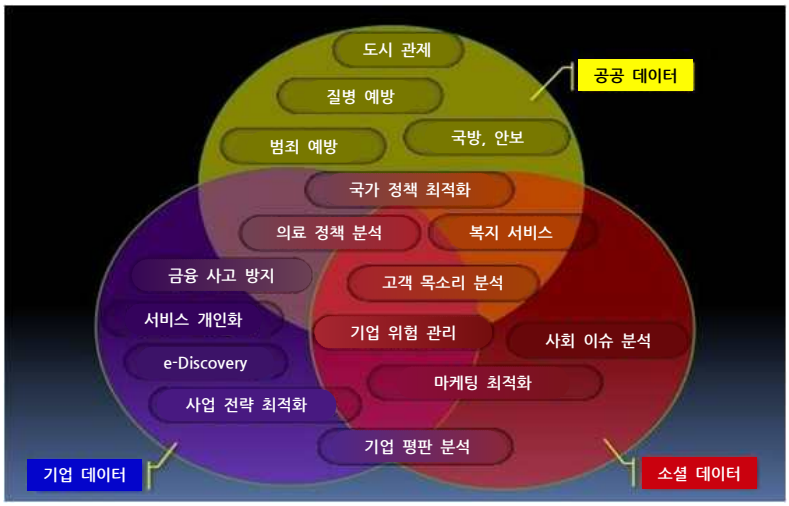


13주차 - 2차시	빅데이터의 실제

1. 빅데이터의 허와 실

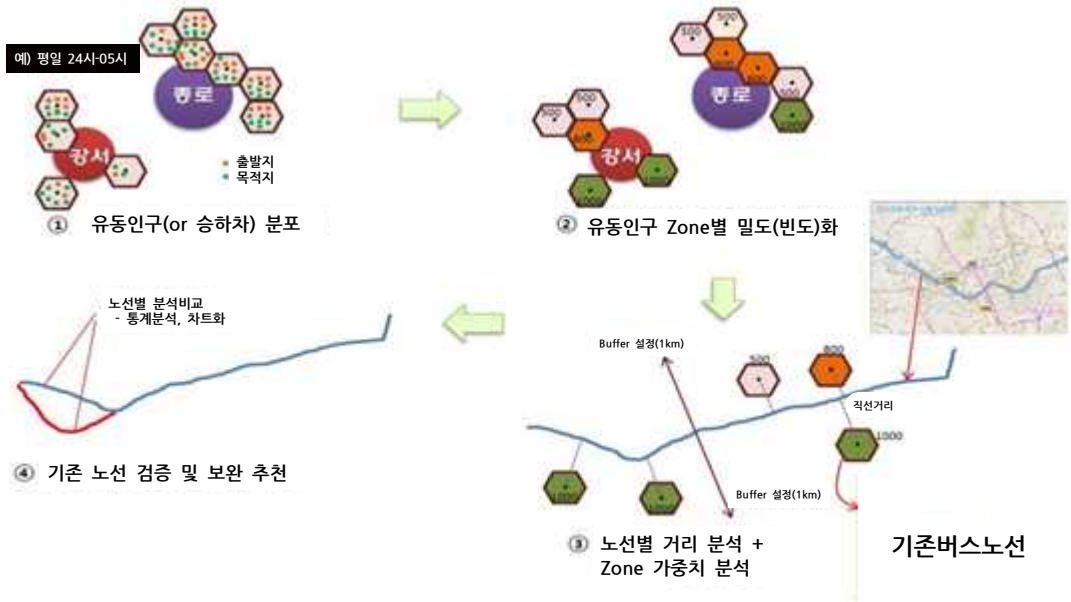
1) 빅데이터 적용 사례

◆ 빅데이터의 활용 분야



◆ 서울시 심야버스

- 승차거부 하루 약 7000여 명
- 9개의 노선 변경 승차거부 8%감소
- 밤에 다니는 여성들이 11% 증가

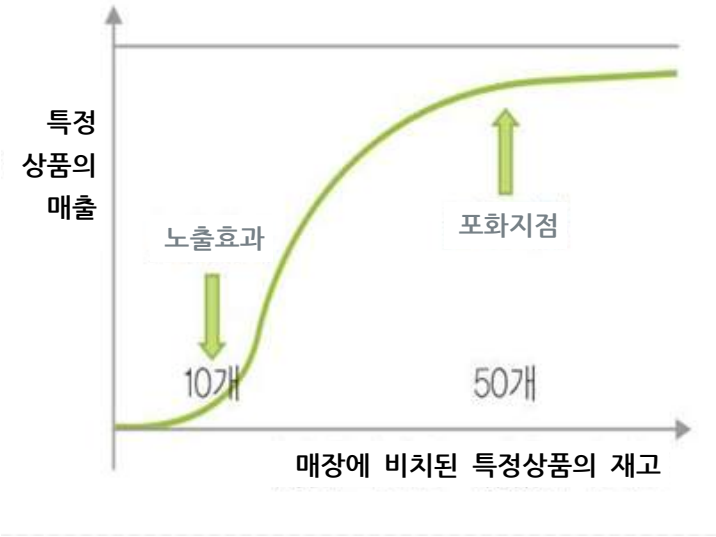


◆ 아마존의 도서 추천 시스템

- 고객들의 모든 구매 관련 데이터 → 고객 구매 정보들 추출 → 책들 간의 연관성을 파악 → 다양한 분야의 책을 추천
- 아이템 기반 필터링(Item-item Collaborative Filtering) → 매출의 1/3 차지

◆ 자라(ZARA) 실시간 데이터 분석

- 패스트 패션(Fast Fashion) 지향
  - 저가격, 스피드, 패션성, 고품질로 차별화, 글로벌화 된 업체가 소비자로부터 각광받게 됨
    → ZARA는 전 세계 400여 개 도시에 진출한 각 매장의 실시간 판매, 재고 데이터를 분석
    → 최대 매출을 달성할 수 있는 재고 최적 분배 시스템을 통해 유행을 선도
- 최대 매출을 달성할 수 있는 재고 최적 분배 시스템 → 전체 매출 극대화



- Problem
  - ▸ 본사에서 A제품 100개를 50개 매장에 공급계획
  - ▸ A제품 10개 이상이 노출되지 않을 시 판매가 미미
  - ▸ A제품이 50개 노출되어야 매출이 극대화됨
- Solution
  - ▸ A제품 100개를 2개 매장에만 50개씩 공급
  - ▸ 2개 매장의 매출 극대화를 통해 전체 매출 극대화
  - ▸ 모든 매장에 유통할 수 있을 만큼의 물량이 없을 경우, 상위 500곳 위주로 제품을 분배

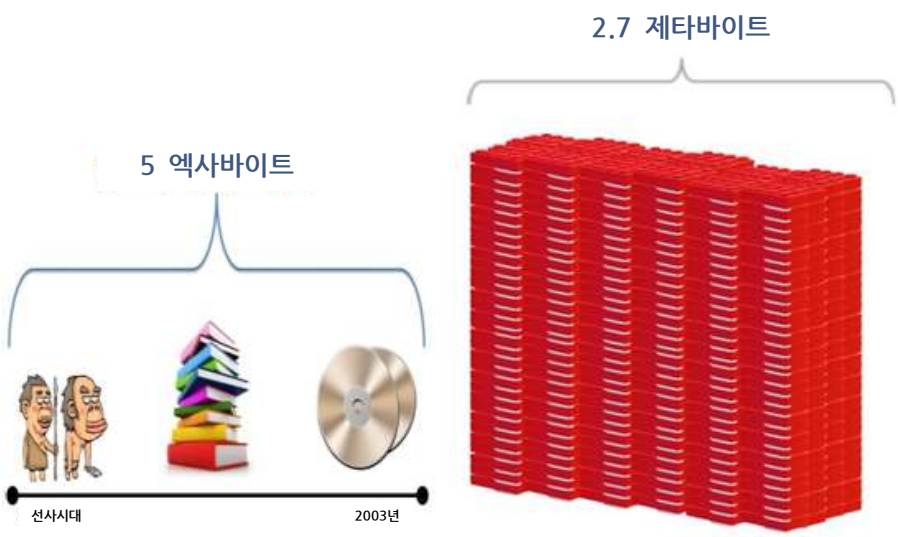
2) 빅데이터 예측 모순점

◆ 구글 독감 프로젝트

- 전 세계 구글 사용자 2003-2007
- 독감 환자수와 가장 잘 맞는 키워드 50개 선정
- 구글 접속 IP주소로 검색 위치 추측 → 전 세계 독감 환자 수 실시간 업데이트 → 미국 질병통제예방센터보다 2주차 빠른 예측
- 그러나 2013년 구글 독감 프로젝트는 언론의 힘 때문에 실패함
  → 상황판단, 내용 검토 없이 숫자만 맹신하는 것은 금물

2. 빅데이터의 규모

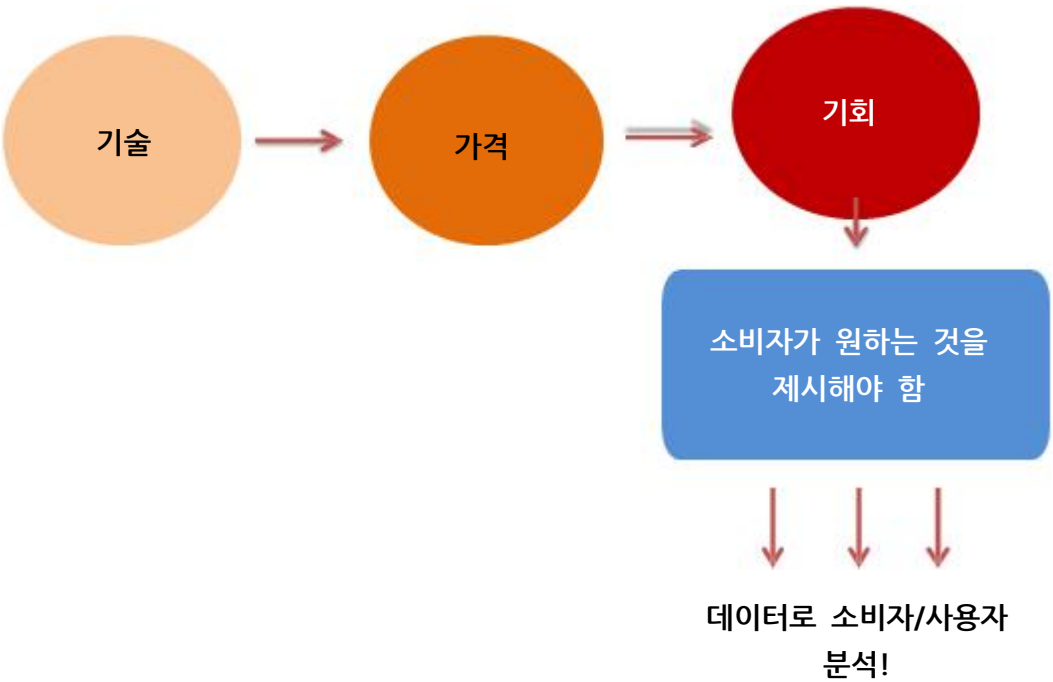
1) 지금은 제타바이트 시대



◆ 거대한 데이터 속에서 인사이트를 발굴하기 위한 움직임이 시작됨

└→ 소비자의 의식이나 행동을 깊이 있게 탐구함으로써 소비자 스스로도 자각하지 못하는 속마음, 또는 무의식을 꿰뚫어보는 일

2) 빅데이터를 통한 경영 혁신



14주차 - 1차시	빅데이터 분석기법

1. 기술 통계

1) 기술 통계 개념

◆ 기술통계(Descriptive Statistics)

- 분석할 데이터의 전반적인 데이터 형태를 나타내는 것을 의미함
- 수집한 데이터를 '요약, 묘사, 설명'하는 통계적인 방법임
- 표본 자체의 속성을 파악하는데 주안점을 두는 데이터 분석방법임
- 주로 표본에 속한 대상자들의 인구통계학적 속성과 함께 연구문제나 가설에 포함된 개별적인 변수에 대한 표본 대상자의 응답임
  - 즉, 데이터 속성을 특정한 통계량을 사용해 요약해 줌
- 추론 통계 전 데이터의 형태를 파악하기 위해 사용됨
  - 추리통계의 기초작업을 수행하기 위한 과정임

2) 기술 통계 해석

◆ 기술통계는 크게 '집중경향' 통계 값과 '분산도'로 나누어짐

- 집중경향(Central Tendency)
  - 분석자료가 몰려 있는 정도를 의미함
    ① 중앙값(Median)
      · 데이터 값 중 가운데 있는 값
    ② 최빈값(Mode)
      · 데이터들 중 가장 많이 존재하는 값
    ③ 평균(Mean)
      · 데이터들의 전체 합을 기준으로 한 평균값
- 분산도(Variance)
  - 자료가 퍼져 있는 정도를 의미함
    ① 범위(Range)
    ② 사분편차(Quartile Deviation)

③ 분산(Variance)

④ 표준편차(Standard Deviation)

2. 상관분석

1) 분산과 공분산

◆ 상관분석 개념

- 자연 및 사회현상의 규명에 있어서 관련된 변수들 간의 상호관련성을 갖게 되는 경우가 흔히 있음

(예) 가계소득과 저축, 흡연량과 폐암발병률 등

◆ 상관분석(Correlation Analysis)

- 두 변수간의 선형관계가 존재하는지 또는 존재하지 않는지를 분석함
  - 상관분석은 변수들 간의 선형성의 강도에 대한 통계적 분석이라 할 수 있음
  - 변수들 간의 구체적인 함수관계를 파악하는 것이 아님
- 회귀분석(Regression Analysis)
  - 한 변수의 값으로부터 다른 변수의 값을 예측하고자 하는 경우에 사용됨
    ↳ (예) 가계소득으로부터 저축을 예측하는 경우
  - 즉 변수들 간의 구체적인 함수관계를 파악하고자 하는 경우에 사용됨

◆ 분산

- 분산은 평균에 대한 편차 제곱의 평균을 구한 값임

◆ 공분산

- 공분산은 두 변수 X, Y가 서로 어떤 패턴(Pattern)을 보여주는가를 나타냄
  - Cov(X, Y) > 0이면, X가 증가(감소)할 때 Y도 증가(감소)
  - Cov(X, Y) < 0이면, X가 증가(감소)할 때 Y는 감소(증가)
  - Cov(X, Y) = 0이면, 두 변수는 아무런 상관이 없음

2) 상관계수

◆ 상관계수(Correlation Coefficient)

- 공분산은 두 변수의 측정단위에 따라 그 값이 달라지기 때문에 두 변수 간

관계의 방향은 알 수 있지만, 관계의 정도는 알 수 없음

→ 이러한 문제를 해결하기 위해 측정단위에 관계없이 관계의 정도를 비교할 수 있도록 표준화한 것이 '상관계수'임

- 공분산을 두 변수 X와 Y의 각 표준편차로 나누면 상관계수를 구할 수 있음

◆ 표본상관계수

- 두 변수의 선형(직선)관계 정도를 나타냄

◆ 표본상관계수 특징

- 표본상관계수(r)는 항상 -1과 1 사이에 있음(-1<=r<=1)
- 표본상관계수의 절대값의 크기는 선형(직선)관계에 가까운 정도를 나타내고, 표본상관계수의 부호는 선형(직선)관계의 방향을 나타냄

① r>0

▸ 산점도에서 점들이 좌하방에서 우상방으로 띠를 형성(우상향의 형태)

② r<0

▸ 산점도에서 점들이 좌상방에서 우하방으로 띠를 형성(우하향의 형태)

③ r=1

▸ 모든 점들이 기울기가 양수인 직선상에 위치

④ r=-1

▸ 모든 점들이 기울기가 음수인 직선상에 위치

3) 상관계수 종류

◆ 종류

- 피어슨 상관계수(Pearson Correlation)
  - 두 개의 연속변수 간 상관계수
    (예) 국어점수와 영어점수간의 상관관계
- 스피어만 서열 상관계수(Spearman Rank Correlation)
  - 순위가 있는 서열척도
    (예) 국어점수 등수와 영어점수 등수간의 상관관계
- 양류상관계수, 점이연 상관계수(Point Biserial Correlation)
  - 두 개 변수 중 하나는 명명척도이고 다른 하나는 연속변수
    (예) 성별과 수학점수와의 상관관계

- 다연상관계수(Polyserial)
  - 두 개의 변수 중 하나는 명명척도이고 다른 하나는 연속변수
    (예) 인종과 키와의 상관관계
    우열반 편성여부와 중간고사 점수와의 상관관계

3. 연관분석

1) 지지도, 신뢰도, 향상도

◆ 지지도(Support)
- 전체 거래 건수 중에서 항목집합 X와 Y를 모두 포함하는 거래 건수의 비율

◆ 신뢰도(Confidence)
- 항목집합 X를 포함하는 거래 중에서 항목집합 Y도 포함하는 거래 비율 (조건부 확률)

◆ 향상도(Lift)
- 항목집합 X가 주어지지 않았을 때의 항목집합 Y의 확률 대비 항목집합 X가 주어졌을 때 항목집합 Y의 확률 증가 비율

2) 연관규칙 해석 및 마케팅 전략

◆ 여러 아이템들이 하나의 레코드(트랜잭션)를 구성하는 레코드(트랜잭션) 집합에서, "어떤 아이템들이 나타나면 특정 아이템들도 함께 나타난다"는 형태의 의존 규칙을 찾고자 함

TID	Items
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Rules Discovered:

{Milk} --> {Coke}
{Diaper, Milk} --> {Beer}

◆ 마케팅 및 영업 프로모션
- 다음 형태의 규칙을 찾았다고 가정 : {Bagels, …} → {Potato Chips}
  - 포테이토칩이 결론(Consequent)부에 있으므로, 포테이토칩의 판매 촉진을 위해서 무엇을 해야 하는지 결정할 수 있음

- 베이글이 선행(Antecedent)부에 있으므로, 만일 베이글 판매를 중지한다면 어떤 제품이 영향을 받는지를 유추할 수 있음

• 마트 진열대(Supermarket Shelf) 관리
- 목적
  ▸ 많은 수의 고객이 “함께 구매하는 제품들”을 식별함
- 접근법
  ▸ 바코드 스캔을 통해 축적된 POS 데이터를 분석하여 판매 제품들 간의 의존성을 알아냄
- 잘 알려진 규칙
  ▸ 기저귀와 우유를 사는 고객은 맥주를 함께 구매할 가능성이 높음

• 재고(Inventory) 관리
- 목적
  ▸ 가전제품 서비스 회사는 제품 수리를 위한 부품들을 각 지역에 적절히 분배함
- 미국은 넓어서 부품 배달, 가정 방문 등의 최소화가 경쟁력에 해당함
- 참고
  ▸ 한번 방문 시 여러 부품들을 함께 교체, 수리하는 경우가 많음
- 접근법
  ▸ 수리 이력 데이터를 분석하여, 어떤 부품들이 함께 필요한지를 파악함

14주차 - 2차시	빅데이터와 인공지능

1. 인공지능 개요

1) 빅데이터를 이용한 인공지능 연구 및 개발 사례

◆ 빅데이터를 이용한 인공지능 연구

• 서술적 분석(Descriptive Analytics)
- 데이터로 지금 일어나는 현상을 설명하는데 초점을 맞추는 영역임

- SNS에 올라오는 글들을 수집하여 최근 많이 언급되고 있는 단어가 무엇인지 파악하고, 이를 통해 현 트렌드를 분석하는 것임

• 예측 분석(Predictive Analytics)
- 빅데이터 분석의 꽃으로, 미래에 무슨 일이 일어날지 사전에 파악하는 영역임
- 머신러닝 기술이 많이 적용됨
- 이메일이 왔을 경우 해당 텍스트를 분석하여 스팸메일인지 아닌지 예측하여 알아서 구별해주는 것임

• 처방적 분석(Prescriptive Analytics)
- 앞으로 무엇을 해야 하는지를 분석하는 영역임
- 책 판매 업체나 음악 스트리밍 사이트에서 상품을 추천하는 것임
- 대부분의 처방적 분석이 추천 시스템의 내용이고 추천 시스템도 역시 딥러닝 기술이 중요함

◆ 빅데이터 활용 분야

• 공공 분야
- 국가적 차원에서 방대한 양의 데이터로 수자원 관리, 스마트 그리드, 재난 방재 영역 등에서 포괄적으로 활용됨

• 과학 분야
- 산발적으로 흩어진 과학 데이터를 국가 차원에서 수집, 가공, 유통, 재활용할 수 있는 기반을 마련함

• 의료 분야
- 의료 기록의 전자화, 병원 간 연구 데이터 공유로 빅데이터 도입과 활용이 확대됨

• 도소매 분야
- 이미 데이터를 활용 중이며 빅데이터 분석으로 수요 예측 및 선제적 경영 지원에 초점을 둠

• 제조 분야
- 보유 데이터양이 많음
- 불량품 개선비용 등 적용 효과를 계량화하여 빅데이터의 유용성을 확인할 수 있는 분야임

• 정보 통신 분야

- 이동통신의 발전과 개인 단말기의 증가로 생성된 디지털 공간의 개인 데이터로 목표 마케팅, 개인화 서비스를 확대함

◆ 빅데이터 활용 사례

• 구글의 독감 트렌드와 자동 번역 서비스, 검색 통계 등과 월마트의 소비자 패턴분석, 리츠칼튼호텔의 고객맞춤형 서비스, T-Mobile의 고객이탈 사전감지 등은 이미 잘 알려진 대표적인 빅데이터 활용 사례임

① 구글 독감트렌드

- 시간・지역별 검색어에 기반을 둔 독감 예상수치를 구글 독감트렌드 사이트를 통하여 제공함

② 구글 자동번역 서비스

- 유럽연합 20여 개 국의 언어로 번역되어 있는 수억 건의 문서를 이용하여 통계적 자동번역 시스템 개발을 성공함

③ 구글 검색통계

- 검색통계를 이용해 오바마 정부의 노후 차량 보상 프로그램의 호응도를 분석하여 정부 예산이 부족할 것을 예측함

④ 월마트의 소비자 패턴분석

- 월마트의 웹사이트에서 발생하는 거래데이터를 이용한 재고예측 조사시스템을 마련하여 고객선호도 및 수요데이터를 분석하여 점포 운영에 반영함

⑤ 리츠칼튼호텔의 고객맞춤형 서비스

- 투숙 고객의 특성을 관찰하여 얻어진 데이터를 종합 분석하여 친절한 서비스를 제공함
- 기존 고객이 제기한 불만사항을 수집하여 서비스를 개선하고 모든 단계에서 품질개선을 위한 프로세스를 추진함

⑥ T-Mobile의 고객이탈 사전감지

- 타 통신사로 회선을 옮기는 고객이 보이는 이용패턴을 분석하여 실시간 고객이탈 사전감지 시스템을 구축함
- 시스템 구축 후 이탈 고객 수가 9만9천 명에서 5만 명으로 감소함

⑦ BoA 사례

- 5천만 건, 약 65PB(petabytes)의 고객 데이터를 보유하고 있으며 이를 분석하여 고객에게 적합한 상품을 제안함

⑧ 오비츠 사례

- 미국의 온라인 여행사이트인 오비츠는 고객의 온라인 활동을 추적하여 특성을 파악해 고객기호나 지출습관을 예측하여 최적화된 제품/상품을 제시함

⑨ 허츠 사례

- 글로벌 렌터카 기업인 허츠는 다양한 채널에 퍼져 있는 VOC(고객의 소리, Voice of Customer)를 실시간으로 분석해 고객의 요구 사항에 대해 빠르게 대응할 수 있는 시스템을 운영함
- 필라델피아 지점에서 고객에 대한 서비스 지연이 발생하는 가장 큰 요인은 차량 반납에 걸리는 시간 때문이며, 하루 중 구체적으로 어떤 시간에 이런 지연이 발생하는지 파악함
- 이러한 정보를 바탕으로 해당 지점의 고객이 몰리는 시간대에 직원 수를 늘리고 고객 불만을 원활하게 해결할 수 있는 지점 매니저를 배치하는 등의 대응책을 실행함

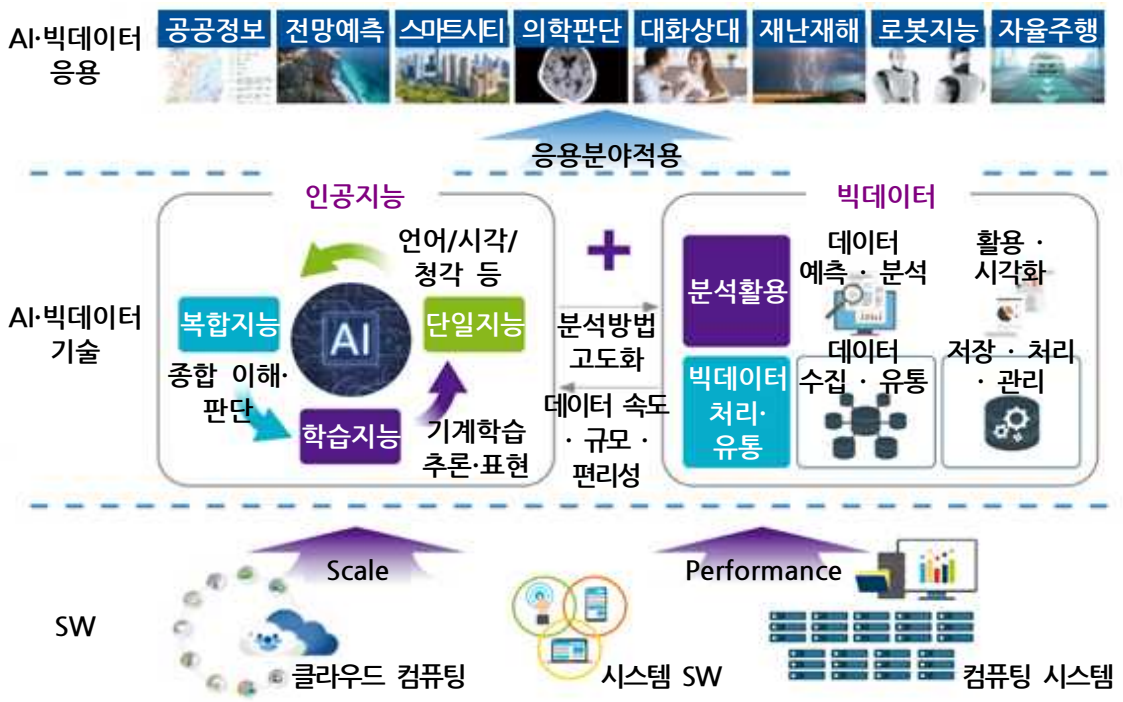
⑩ UNC 헬스케어 사례

- 빅데이터를 적극 활용해 선진화된 의료 서비스를 갖추고, 맞춤형 건강관리와 같은 다양한 환자관리 프로그램을 마련함
- 유방암과 자궁경부암 부문에서 암진단 건수를 10%이상 증가시켰으며, 결장암과 같은 다른 암의 진단에 확대 적용함

⑪ GE 사례

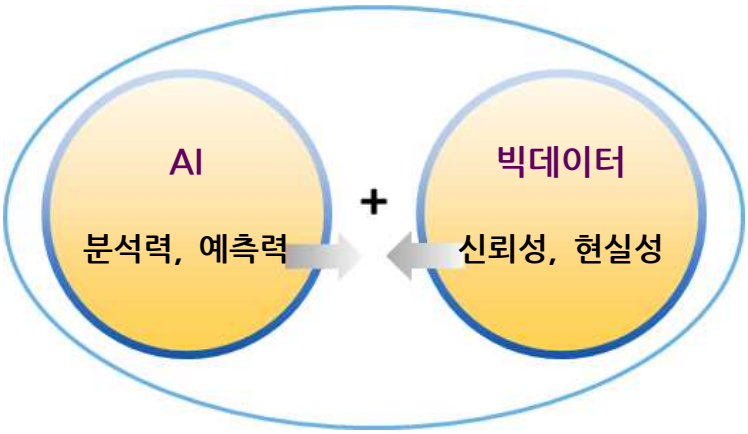
- 지능형 시스템 기반의 '지능형 항공 운영(Intelligent Operation)' 실현으로 운영 효율성을 극대화함
- 예측하지 못한 항공정비로 인해 발생하는 비행지연을 사전에 예방하여 항공기 운영 효율성의 증가, 정시 운영의 증가, 정비 효율의 증가, 유지보수 비용의 감소를 실현함

◆ 인공지능과 빅데이터 관계



(출처) https://modulabs.co.kr/blog/ai-bigdata/

◆ 빅데이터와 AI의 상호보완효과



- AI와 빅데이터가 만나면 서로의 약점을 보완하면서 윈윈효과를 창출할 수 있음
  - AI 분야는 마치 앞이 안 보이는 외길을 걷는 분야와 같다는 평가를 받아오면서 실현가능성에 대한 의문이 지속적으로 제기되어 옴
  - 그러나 어마어마한 양의 빅데이터가 등장하면서 새로운 국면을 맞이하고 있음
- 방대한 양의 데이터가 AI의 실현가능성 및 연구의 신뢰성을 높여주고 있음
- 방대한 양의 데이터 습득과 학습, 분석, 추론 등을 통해 알고리즘의 정확성

및 정교성을 높일 수 있음

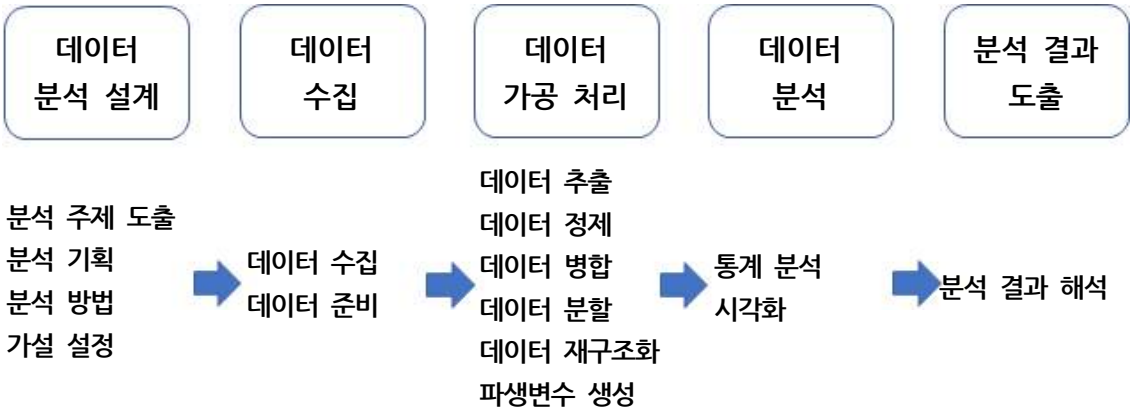
- 빅데이터 분야 역시 AI의 분석력, 추론력, 예측력 등을 활용하면 더욱 지능화된 가치 있는 서비스 창출이 가능함

→ AI와 빅데이터는 상호보완적인 역할을 수행하면서, 서로에게 시너지효과를 만들어낼 수 있음

2) 빅데이터 수집 및 전처리, 저장관리

◆ 빅데이터 분석 과정

- 데이터 분석은 주어진 데이터로부터 인사이트를 찾아내고, 의사결정을 지원하기 위해 데이터를 전처리, 변환, 모델링하는 과정을 의미함



① 데이터 분석 설계

- 분석 주제 설정
  - ▸ 분석하려는 주제를 명확하고 구체적으로 설정함
- 가설 설정
  - ▸ 브레인스토밍 등을 활용하여 주제와 연관된 내용의 가설을 설정함
- 분석 가능 변수 구성
  - ▸ 가설에 따른 분석 가능 변수를 구성하고, 파생 변수도 함께 고려함
- 분석 항목 결정

② 데이터 수집

- DB에 저장되어 있는 데이터를 불러옴
- 데이터원으로부터 TXT, CSV 파일 등 다운로드함
- 웹 크롤링을 이용하여 데이터를 수집함
- 데이터 구성을 파악함

③ 데이터 가공처리
- 원시 데이터를 분석에 적합한 형태로 변환함
- 데이터를 병합 또는 분할함
- 데이터를 분석 목적에 따라 재구조화함

④ 데이터 분석
- 기술통계(빈도, 평균, 최댓값, 이상치 등)를 조사함
- 다양한 그래프 생성 및 데이터를 탐색함
- 데이터 패턴 및 분포를 확인함
- 적용 가능한 분석 방법을 적용함

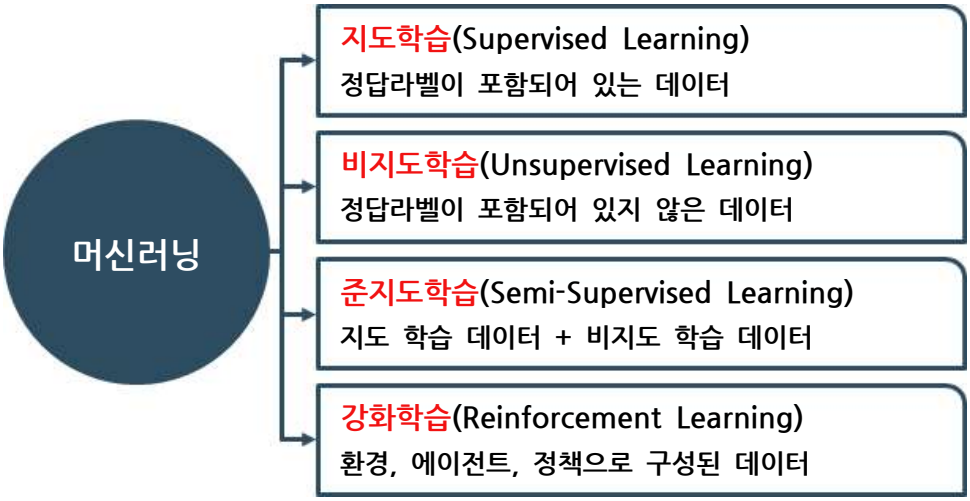
⑤ 데이터 분석 결과 도출
- 다양한 분석 방법을 통해 가설을 검정함
- 분석 결과를 정리 및 시각화함

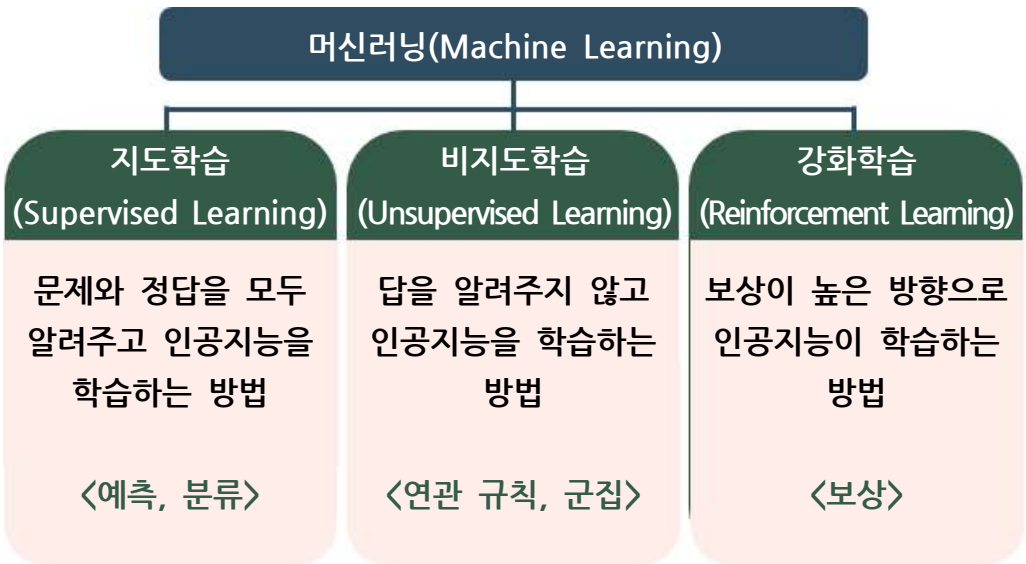
2. 머신러닝과 딥러닝

1) 머신러닝 알고리즘

◆ 머신러닝(Machine Learning)
- 규칙을 일일이 프로그래밍하지 않아도 자동으로 데이터에서 규칙을 학습하는 알고리즘을 연구하는 분야임
- 인공지능의 하위 분야 중에서 지능을 구현하기 위한 소프트웨어를 담당하는 핵심 분야임
- 통계학에서 유래된 머신러닝 알고리즘이 많으며 통계학과 컴퓨터 과학 분야가 상호 작용하면서 발전하고 있음
- 컴퓨터 과학 분야의 대표적인 머신러닝 라이브러리는 사이킷런(Scikit-learn)이 사용됨

◆ 머신러닝 종류
- 머신러닝은 학습 방법에 따라서 4가지 유형으로 구분됨





① 지도 학습(Supervised Learning)

- 정답이 있는 데이터를 활용해 데이터를 학습시키는 방법임
  - ▸ 종류 : 분류(Classification), 회귀(Regression)

② 비지도 학습(Unsupervised Learning)

- 정답이 없는 데이터를 군집화하여 새로운 데이터에 대한 결과를 예측하는 방법임
  - ▸ 종류 : 클러스터링, k-means

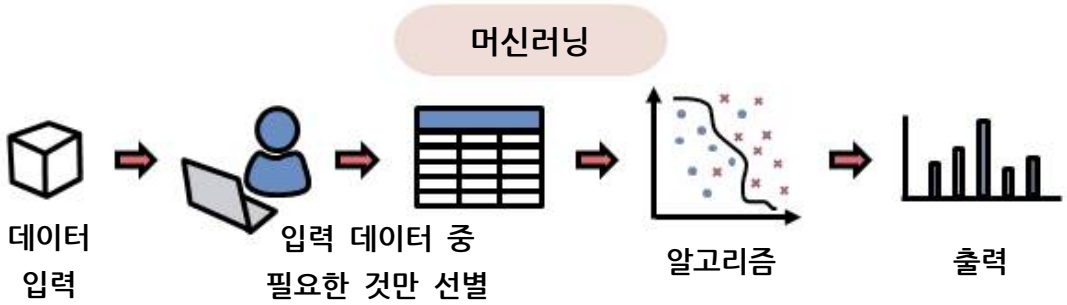
③ 준지도 학습(Semi Supervised)

- 지도 학습과 비지도 학습의 중간에 해당하는 기술로, 명확한 정답이 존재하나 정답이 있는 데이터를 구하기 힘들 때 사용함

④ 강화 학습(Reinforcement Learning)

- 주어진 환경에서 어떤 행동을 취하고 이로부터 어떤 보상을 얻으면서 학습을 진행함
  - ▸ 개념 : 에이전트, 환경, 상태, 행동, 보상

2) 딥러닝 알고리즘

◆ 딥러닝(Deep Learning)

- 머신러닝보다 더 작은 개념으로 '신경망'을 통해 인공지능을 만드는 머신러닝의 한 종류임
- 딥러닝은 신경망을 여러 층 쌓아서 만들어 냄
- 딥러닝은 합성 함수처럼 동물의 신경세포들의 합성인 신경망을 따라 만든 인공신경망을 여러 계층 쌓아서 만든 깊은 신경망임